hexure

FireLight Developer Guide

firelight

FIRELIGHT DEVELOPER GUIDE

Document Version: 5.3

Published: 1/26/2023

Hexure.com

Website: hexure.com

Copyright © 2019-2023 Hexure, all rights reserved.

Hexure, Insurance Technologies, ForeSight® and FireLight® are registered or unregistered trademarks of Insurance Technologies, LLC (IT) in the USA and/or other countries.

ACORD, ACORD ObjX, ACORD OLifE, AL3, ACORD Advantage, ACORD XML, and "Association for Cooperative Operations Research and Development" are registered or unregistered trademarks of ACORD Corporation in the USA and/or other countries.

Microsoft, Microsoft SQL Server, Microsoft Internet Information Server, Windows, and other Microsoft names and logos are either registered or unregistered trademarks of Microsoft Corporation in the U.S.A. and/or other countries.

All other trademarks are the property of their respective owners.

The information contained in this document is current as of the date of the publication. Because Insurance Technologies, LLC must respond to changing market conditions and technology advances, Insurance Technologies, LLC cannot guarantee the accuracy of any information presented after the date of publication.

INSURANCE TECHNOLOGIES, LLC MAKES NO WARRANTIES, EXPRESSED OR IMPLIED, IN THIS DOCUMENT AND HEREBY DISCLAIMS ANY AND ALL SUCH WARRANTIES.

The material contained in this document is considered confidential and the intellectual property of Insurance Technologies, LLC. The recipient is given access to this material on the condition that the recipient (1) will keep the information confidential at all times, and (2) will not copy or modify or share the materials, except as expressly authorized by Insurance Technologies, LLC. The recipient should limit its disclosure of the information and materials only to its employees who have a clear business purpose and need to receive such information and materials and who are bound by confidentiality obligations to the recipient that are at least as protective of such information and materials as those contained herein.

Hexure

Two South Cascade Avenue

Colorado Springs, CO 80903

Phone: 719.442.6400

Internet E-Mail: info@hexure.com

Table of Contents

API Introduction

Target Audience 12

CORS 12

Third Party Cookies and the Authorization Proxy 13

Base URL 13

API Token Service

Overview 14

Token Service Endpoint 14

Pre-Configuring Your Credentials 14

Supported Services Record 15

Organization Carrier Code 16

Associate User with Token (optional) 16

Simple Example 16

Tx1228 Definition 17

Getting the Token 18

Required Headers 18

Body Content 19

Generating the Hash 20

Generating the Signature 21

Response Format 22

Sample Request and Response 22

Request 23

Response 23

Using the Token 24

Code Sample 24

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

FireLight REST API

Overview 26

Endpoint Root URLs 26

Activity Methods 27

CreateActivity 27

DeleteActivity 28

GetActivities 29

GetActivityDetails 32

GetActivityPDF 35

GetAvailableActions 36

GetDocumentList 38

GetProductAvailability 39

ProcessEDeliveryCase 41

SignatureRequest 47

SubmissionConfirmation 53

Submit 53

UnlockActivity 54

UploadDocument 57

ValidateActivityData 58

Illustration - Methods 83

GetCalculationResults 83

GetIllustrationPDF 87

Review Queue Methods 90

GetReviewQueueStates 90

UpdateReviewQueueState 92

Tele-Sign - Methods 93

CompleteSignature 94

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

ValidateSignature 95

FireLight Embedded Client API

Overview 98

Font Awesome Installation 98

Getting Started 98

Common API Interface 101

Events 102

getRuleMessages 103

init 103

loadModule 104

navigateToPage 104

navigateToPageNext 105

navigateToPagePrevious 105

renderPDF 105

save 106

setShowAllValidationMessages 106

showUnlockDialog 106

toggleShowAllValidationMessages 106

Code Sample 107

E-Application Client API Interface 110

Events 110

loadEmbeddedEApp 110

proceedToSignatures 110

showDocumentsDialog 111

Code Sample 111

E-Sign Client API Interface 114

Events 114

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

getAvailableActions 114
loadEmbeddedESign 114
Code Sample 115
Illustration Client API Interface 117
Events 117
createApplication 117
openQuickView 117
updateApplication 117
Code Sample 118
JSON Models
Rest API Models 120
ActivitiesRequest 120
Activity 120
ActivityAttachment 121
ActivityAttachmentDocumentFormat 122
ActivityAttachmentDocumentType 122
ActivityAuditEntity 123
ActivityDataItem 124
ActivityDetailRequest 124
ActivityPDFResponse 124
ActivityValidationRequest 125
AvailableActionsReturn 126
AvailableProductResponse 127
CalculationErrorResult 128
CalculationResponse 128
CompleteSignatureRequest 129
CompleteSignatureResponse 130

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

CreateActivityRequest 130
CreateActivityResponse 132
DocumentInfoDocumentType 133
DocumentInfoResponse 133
DutyType 134
GeneralAPIResponse 136
GetActivities 136
GetActivityDetails 137
GetActivityDetailsDataItem 137
GetActivityPDFRequest 138
GetActivityPDFResponse 138
GetDocumentListResponse 139
GetIllustrationRequest 139
GetReviewQueueStatesRequest 139
OutputField 140
OutputModel 140
OutputSet 141
OutputTable 142
ProductAvailabilityRequestModel 142
ProductAvailabilityResponse 143
ReportResponse 143
ResponseMessage 144
ReviewQueueGetStatusResponseModel 145
ReviewQueueState 145
ReviewQueueUpdateStatusResponseModel 146
TokenResponse 146
UpdateReviewQueueStateRequest 146
UploadDocumentRequest 147

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

ValidateActivityDataResult 147
ValidateSignatureRequest 148
ValidateSignatureResponse 148
ValidationMessage 149
WizardErrorMessage 150
Embedded Client API Event Models 151
ErrorEvent 151
ESignStatusEvent 151
LinkedApplicationEvent 152
NavigationEvent 152
NotificationsEvent 153
RuleMessagesEvent 154
SignatureStatusEvent 154
StatusEvent 155
Embedded Client API Models 155
ActivityBundle 156
AvailableActions 157
BundlePage 157
Error 158
Message 159
MessageError 160
Lists 161
AcordTypeCode & AcordSubTypeCode 161
Display Status Types 162
E-Sign Status Types 168
Jurisdictions 169
Last Action Types 172
Notification Severity Types 172

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

QueueState 172
Return Status 173
Status Types 173
Transaction Types 174

Salesforce Code Samples

CreateActivity Example 176
GetToken Example 176
VisualForce Page Code 177

Embedded Workflows

Declining e-Signature and Capturing Wet-Signed Documents 182
E-Sign Workflows 183
Steps for Using FireLight E-Sign within an Embedded Application 183
Steps for Using Embedded E-Sign within an Embedded Application 183

Common Issues Using the FireLight Embedded System

Sign On Methods

Introduction 187
FireLight External Integration 187
Summary 187
Create New Application 187
ACORD TX103 Transaction Request 189
Submitting Compliant Illustration 190
Creating a Fee Inquiry Transaction 191
Submitting a Third Party Needs Determination 192
Submitting TX103 Transaction 193
ACORD Tx103 Response 196

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Post ACORD TX 1228 Transaction 196
"Can Sell" Checks for Agents 197
Home Office Users Can Enter Applications for an Agent 197
Sample HTML Form 197
Edit Existing Application in FireLight 200
Launch into FireLight Dashboard 202
Launch into FireLight Dashboard - Delayed Use of 103 203
ACORD Tx103 Requirements and Usage 204
Overview 204
Requirements 204
Usage 206
ACORD Tx1228 Requirements and Usage 208
Overview 208
TX1228 Examples 208
Simple Example 208
Extended Example 209
User Nickname Example 211
Elements within the Tx1228 213
Usage 216
POST ACORD Tx1228 to FireLight 216
Generating the <CryptPswd> Contents 216
Deriving the IV from the Shared Key 217
Tx1228 in an SSO 217
FireLight SSO Requirements 218
SAML 2.0 218
Required Assertions 218
User Role 218
User Rights 218

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Organization ID 219
Name Identifier 219
Optional Assertions 219
SSO Session Info 219
SSO New Business Transaction 220
External Role Code 220
Branding 221
Sample SAML 2.0 Response 221
IDP-Initiated SSO Endpoint 223
Possible Redirects 223
Managing Firms and Product Availability 224
Details 224
Groups 225
Requirements 225
Sample 1228 228
Token SSO 230
Overview 230
Admin Setting 231
Redirect Options within FireLight 231
Endpoints 233

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

API Introduction

The FireLight® API is a service that can be used to get information about FireLight activities. Activities are defined as pre-sale, illustration, application, e-Delivery, and post issue.

The API topics in the FireLight Developer Guide walk through aspects of our API:

- API Token Service authentication and how to use our token service
- FireLight REST API and the actions that we support
- FireLight Embedded Client API that allows you to render FireLight wizards directly into your application
- JSON Models that are used with the FireLight REST API
- Salesforce Code Samples for using FireLight with Salesforce
- Embedded Workflows using FireLight APIs
- Common Issues Using the FireLight Embedded System
- Spotlight Feature Videos about FireLight APIs

Target Audience

This documentation is designed for people familiar with programming and object-oriented programming concepts. You should also be familiar with FireLight and have access to FireLight and the FireLight Admin tool.

CORS

Access to the FireLight API has been restricted due to security concerns surrounding the integrity and safeguarding of our clients' data. Before you will be able to make a successful API call you will need to work with your FireLight Project Manager to give your development environment access to the API.

By default we have the following hosts setup to allow remote requests:

http://localhost:4000

https://localhost:4000

https://localhost:44355

https://localhost:44356

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Third Party Cookies and the Authorization Proxy

In order to support embedded scenarios, FireLight uses third party cookies to send information and stylesheets to the client web page. However, the latest versions of Firefox and Safari currently disable third-party cookies, as will an upcoming release of Chrome. As a result, an authorization proxy has been added to the Embedded FireLight. This allows the system to use cookies and css that reside on the FireLight server from within an embedded application being hosted on a client server. The proxy server must be manually enabled by setting its corresponding flag within FireLightAPI.Init() and FireLightAPI.startSession().

This also means that you can no longer transition to the FireLight standalone E-Sign by using the URL returned from FireLightAPI.proceedToSignatures(). Instead, SSO (Single Sign On) must be used to navigate to the FireLight E-Sign application. See the separate document for information on SSO. Alternatively, Embedded E-Sign may be used, but this version of E-Sign is currently lacking numerous features available to FireLight E-Sign, such as external signing. See the corresponding sections in this document for more information on Embedded E-Sign.

Base URL

All of the endpoints defined in this document, should reference these base URL's. The base URL is environment specific. Your code should be configurable depending on which FireLight environment your application is using.

Base URLs

Environment	URL
QA	https://flqa.insurancetechnologies.com/EGApp/
QA NEXT	https://flqanext.insurancetechnologies.com/EGAppnext/
UAT	https://uat.firelighteapp.com/EGApp/
STAGING	https://staging.firelighteapp.com/EGApp/
PRODUCTION	https://www.firelighteapp.com/EGApp/

** NEXT environments are early release environments and may not always be available.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

API Token Service

Step through the following topics to find information about the API Token Service.

Overview

The FireLight rest-based API services utilize JSON web tokens for security and access. Please see the documentation for each service in order to determine any differences in security.

Accessing a FireLight API service endpoint requires a JWT token first received from the FireLight token endpoint. Getting access to a service endpoint is a two-step process:

1. A call to the FireLight Token endpoint to get a token. Note this requires credentials pre-defined for your organization. The token will be returned in an encoded string format.

2. Embedding the received encoded token string into the subsequent service endpoint call's "Authorization" header.

    a. FireLight API tokens may be used multiple times within their time limit window. A FireLight API token expires after 15 minutes, after which, you will need to get a new one.

For more information on JSON web tokens, see the open standard documentation at https://tools.ietf.org/html/rfc7519.

Token Service Endpoint

The following token endpoint requires POST requests only, as getting a token requires credential details carried in the body of the request. The following URL is the endpoint for the token service:

{BASE URL}/api/Security/GetToken

Pre-Configuring Your Credentials

Before being able to accomplish the first step of the process, your organization needs to configure specific security options via the FireLight Admin tool. Please contact Insurance Technologies if you do not have access to the FireLight Admin tool.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Supported Services Record

Part of the formation of a request to the token endpoint is a SHA256 hash generated off a secret, placed in the request as a hex-formatted string. A request to the token endpoint additionally requires a digital signature made with the private key of a certificate. This certificate is owned and controlled by your organization, and FireLight will need the exported public key of that certificate to verify the signature on your token requests. The digital signature is generated using the private key of the configured certificate, using either a SHA1 or SHA256 algorithm. Other algorithms are not supported.

In the Admin tool, Organization Configuration > Organization Settings > Security tab: near the end of the tab, add a new supported service record in the supported services table to create a secret and enter a certificate for each supported service. The supported services table displays all certificates and supported services data that are used for authentication within the system. For API/Embedded access, the table can have multiple certificates and keys defined so that the organization can let vendor A in with one certificate/key combination and let vendor B in with another certificate/key combination.

NOTE: To enable one or more extra services for your organization, contact your FireLight project manager to ensure the correct billing is set up and the database is updated to enable the Supported Services check boxes for your organization to add one or more new supported services.

1. Below the table, click the **Add New Supported Service Record** link to open the Supported Services Configuration dialog box.
2. Enter a **Name** for the new service certificate.
3. In **Supported Services**, select the check box for the type of service (IDP, Embedded, Illustration API, API Access).
4. In **Client Secret**, click **New** to generate a new Client Secret. The generated secrets are randomly generated GUID/UUID strings.
5. In the **Certificates** section, enter a certificate. To enter additional certificates for the record, click the plus (+) sign to open another tab. The table can have up to 4 certificates per selected supported service.
6. Copy and paste the certificate information into the text box.
7. Click **Save** and close the Supported Services Configuration dialog box.
8. In the supported services table **Enabled** column, select the check box for the

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

record to enable the service.

9. Click the **Save Organization** button.

Organization Carrier Code

Insurance Technologies assigns your organization with a specific acronym for FireLight. Please contact Insurance Technologies for this acronym as needed.

Associate User with Token (optional)

If you will be using an end point that requires a user associated to the token, an ACORD-formatted Tx1228 xml is required for us to identify the user.

Simple Example

The simple example is used to provide basic user information.

Simple Example

```
<TXLife xmlns="http://ACORD.org/Standards/Life/2">
  <TXLifeRequest>
    <TransRefGUID>26C1A4FC-C22C-4027-9BF6-575AF38AAE2A</TransRefGUID>
    <TransType tc="1228">OLI_TRANS_TRNPRODINQ</TransType>
    <OLifE>
      <SourceInfo>
        <CreationDate>2010-11-02</CreationDate>
        <CreationTime>12:20:03.5876024-06:00</CreationTime>
        <SourceInfoName>Carrier Portal Name</SourceInfoName>
      </SourceInfo>
      <Party id="Agent_Party">
        <PartyTypeCode tc="1">OLI_PT_Person</PartyTypeCode>
        <FullName>USER Full Name</FullName>
        <EMailAddress>
          <AddrLine>email@insurancetechnologies.com</AddrLine>
        </EMailAddress>
        <Producer>
          <CarrierAppointment PartyID="Agent_Party">
            <CompanyProducerID>XXX AGENT ID</CompanyProducerID>
            <CarrierCode>XXX DEFINED CARRIER CODE</CarrierCode>
          </CarrierAppointment>
        </Producer>
      </Party>
      <OLifEExtension VendorCode="25">
 <UserRoleCode tc="InsTech">InsTech</UserRoleCode>
      </OLifEExtension>
      <Relation OriginatingObjectID="Agent_Party">
        <RelationRoleCode tc="11">OLI_REL_AGENT</RelationRoleCode>
      </Relation>
    </OLifE>
  </TXLifeRequest>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
</TXLife>
```

Tx1228 Definition

The following table describes each element within the Tx1228.

Elements within the TX1228

Element	Description
<TXLifeRequest>	Required aggregate containing information about the actual user of the system.
<TransRefGUID>	Required and must be unique for each transaction.
<TransType>	Required and “tc” value must be “1228” (OLI_TRANS_TRNPRODINQ).
<OLifE>	Required aggregate.
<SourceInfo>	Optional aggregate used by FireLight logging to indicate the actual referring system identifier. For instance, if you have multiple ways into FireLight, you can use the <SourceInfoName> to identify which launch mechanism (e.g., portal) was used.
<Party>	Required aggregate containing information used by FireLight to identify the actual user of the system. Optionally, used to identify Firms the user is associated with.
<FullName>	Optional if <Person><FirstName> and <LastName> are supplied. Required if <Person> aggregate is not used.
<Producer>	Required aggregate used to uniquely identify the user within the FireLight system.
<CarrierAppointment>	Required aggregate containing an identifier used by

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Element	Description
	FireLight to identity the user and/or firm's distribution agreement.
<CompanyProducerID>	Unique identifier from the external system that is used by FireLight to uniquely identify the user.
<CarrierCode>	Used by FireLight to identify the organization subscription to FireLight.
<Relation>	Required aggregate used to identify the role of the <Party> aggregate.
<UserRoleCode>	User role code that controls access rights for the user. These role codes are defined within the Groups page in the FireLight Admin tool.

Getting the Token

The first step of the process is to receive the token. Multiple requirements regarding headers and the body content follow, as well as the process to do so.

Required Headers

The following HTTP headers are required to be on the request to the token endpoint.

Required Headers

Header Name	Expected Value	Description
Method	POST	Method of the request.
ContentType	text/plain	Media Type of the request.
ContentLength	Integer Variable	Length of the content contained within the body of the request.
Date	Date-Formatted	Date/time of the request. Must be

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Header Name	Expected Value	Description
	Variable	formatted according to common specification, including UTC/GMT requirement. Note this item additionally gets used to verify the signature, as defined in the 'Generating the Signature' section below.

Body Content

The body content is formatted in a plain-text string with a specific format. If you are planning to call out to an API service endpoint that needs a user, you must include an ACORD-formatted 1228 xml in the request for a token.

If a user is not required for the API's you will be using, you can exclude that part from the body string, as shown below:

- With user 1228:
  - "grant_type=" + **grant_type** + "id=" + **org** + "&secret=" + **hash** + "&sig=" + **signed64** + "&xml=" + **user1228**
- Without user 1228:
  - "grant_type=" + **grant_type** + "id=" + **org** + "&secret=" + **hash** + "&sig=" + **signed64**

The variable details are detailed in the following table:

Variable Details

Variable Name	Expected Value	Description
grant_type	hashsig	This unchanging value must be placed on every request. It indicates the type of security mechanism to be used.
org	String Variable	This is the unique and unchanging organization acronym attributed to each

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Variable Name	Expected Value	Description
		FireLight client organization. Contact Insurance Technologies for this variable.
hash	Hex String Variable	This hexed string value is generated from the request originator's configured secret, unique to each environment. See the 'Generating the Hash' section below for more details.
signed64	Url-Encoded Base64 String Variable	This base64 string value is generated from the request originator's cert that is configured in the FireLight Admin tool. It must be url-encoded (after being base64-encoded) before placed into the post body. See the 'Generating the Signature' section below for more details.
user1228		This is a conditional ACORD 1228 xml containing the details of the user to be applied to the token, as defined in the "Associate User with Token" section of Pre-Configuring Your Credentials. It must be base64-encoded, and then url-encoded, before being sent in the token request's body.

Generating the Hash

A hash is required with the request's body in order to be verified by the service. This hash is generated off a string 'Secret' configured in the FireLight Admin tool.

The following steps explain the process of generating the hash value:

1. Obtain the configured 'Secret' string value for whichever FireLight environment the request will be sent.
2. Create a SHA256 hash of the 'Secret' string value.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

3. Convert the hashed byte array to upper-cased hex values.
4. This generated hash is the 'hash' variable of the request body content.

Only SHA256 hash values are supported for the 'hash' variable.

Generating the Signature

A base64-encoded digital signature string must be included on the request body. One of the certificates configured in the "Web Service Token Certificate" dialog box in the FireLight Admin tool must be used to create the signature. FireLight uses the configured public key to verify the signature.

The digital signature is generated using the private key of the configured certificate, using either a SHA1 or SHA256 algorithm. Other algorithms are not supported.

The signature is based off the content of a specific string. If a user 1228 is included, that portion is part of the string used to generate the signature. The following text string formats are expected to be the content that is signed:

- If user 1228 is included:
  - **NonHashedSecret** + **RequestDate** + **Org** + **Base64User1228**
- If user 1228 is not included:
  - **NonHashedSecret** + **RequestDate** + **Org**

The following table describes the variables.

Variables

Signature Variable	Expected Value	Description
NonHashedSecret	String Variable	This is the plain text (not the hashed hex representation) of the configured 'Secret' from the FireLight Admin tool.
RequestDate	String Variable of Request Date	This is the exact same value of the request's Date header. When formatted into a string, use this formatting: MMddyyyyHHmmss. If the formatting is different at the time

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Signature Variable	Expected Value	Description
		of generating the signature, or the time is different from the request's Date header, the signature is considered invalid.
Org	String Variable	This is the same value of the 'org' variable placed in the body. Contact Insurance Technologies for this variable.
Base64User1228	Base64 String Variable	This is the base64-encoded string of the user's 1228 xml, if applicable. It is not url-encoded yet. If your signature is based off the url-encoded version, it will not be verified by the FireLight system.

Response Format

A successful request will receive an object of type TokenResponse model, which will be passed back as a JSON object within the body content. It will contain the text value of the token, the type of token, and the expiration (in seconds).

Sample Request and Response

The following shows a successful request and its subsequent response, via Fiddler (highlighted for clarity) (the hex value and signature were slightly changed for security reasons).

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Request

```
POST https://localhost/EGApp/api/Security/GetToken HTTP/1.1
User-Agent: Fiddler
Host: localhost
Content-Length: 2410
Content-Type: text/plain
Date: Wed, 14 Mar 2018 21:31:52 GMT

grant_type=hashsig&id=IT&secret=767F75E18449B0880D68B314A1F9674417CE8E48569D8A46D98DF1A13BE19894&s
ig=F3k41NYsvvhaz0wszl2W0vL7ZDCoDtXSwWa9KEuFMbV0Roa%2FL4cdECFqcmT1Uj5UGG2B85E%2FfF%2F1Rr2zm5JqLbaBl
49l43BJpphYGICbtXhimHeO4NsQkUUSS0%2FLlSIjMzhS%2FxpeY2QcSaMKKDEvLvr1WITyRyjntcrkDYfxpsHTD7aSfArN3tf
XzCMDOJc8Bslki1qMNk9VlYK83pyh1FP2DGw1ffzq4XAz%2FFFxhLZoW%2FuF7PmT0FU1YZDO5jAnZClML74fa5sJUrSKMXrMR
Rkou7bbwPM29YiTbGm6VcA9tlIG6B670F4FqfzaYfyWyMFTzdeg0PZpS7n1yJK%2FKg%3D%3D&xml=PFRYTGlmZSB4bWxucz0i
aHR0cDovL0FDT1JELm9yZy9TdGFuZGFyZHMvTGlmZS8yIj4NCiAgPFRYTGlmZVJlcXVlc3Q%2BDQoqICAgPFRyYW5zUmVmR1VJ
RD4yNkMxQTRGQy1DMjJDLTQwMjctOUJGNi01NzVBRjM4QUFFMkE8L1RyYW5zUmVmR1VJRD4NCiAgICA8VHJhbnNUeXBlIHRjPS
IxMjI4Ij5PTElfVFJBTlNfVFJOUFJPRElOUTWvVHJhbnNUeXBlPg0KICAgIDxUcmFuc1N1YlR5cGUgdGM9IjIyODAwIj5PTElf
VFJBTlNTVUJfUFJPRERFVEFMTDwvVHJhbnNTdWJUeXBlPg0KICAgIDxUcmFucOv4ZURhdGU%2BMjAxNi0xMS0wMjwvVHJhbnNF
eGVEYXRlPg0KICAgIDxUcmFucOv4ZVRpbWU%2BMTI6MjA6MDMuNTg3NjAyNC0wNjowMDwvVHJhbnNFeGVUaE1lPg0KICAgIDxP
TGlmRT4NCiAgICAgIDxTb3VyY2VJbmZvPg0KICAgICAgICA8Q3JlYXRpb25EYXRlPjIwMTAtMTEtMDI8L0NyZWF0aW9uRGF0ZT
4NCiAgICAgICAgPENyZWF0aW9uVGltZT4xMjoyMDowMy41ODc2MDI0LTA2OjAwPC9DcmVhdGlvblRpbWU%2BDQoqICAgICAgID
xTb3VyY2VJbmZvTmFtZT5HcmFudDwvU291cmNlSW5mb05hbWU%2BDQogICAgICA8L1NvdXJjZUluZm8%2BDQogICAgICA8UGFy
dHkqaWQ9IkFnZW5OX1BhcnR5Ij4NCiAgICAgICAgPFBhcnR5VHlwZUNvZGUgdGM9IjEiPk9MSV9QVF9QZXJzb248LiBhcnR5VH
lwZUNvZGU%2BDQogICAgICAgIDxGdWxsTmFtZT5BZ2VudHMgRnVsbCBOYW1lPC9GdWxsTmFtZT4NCiAgICAgICAgPEVNYWlsQW
RkcmVzcz4NCiAgICAgICAgICA8QWRkckxpbmU%2BYWdlbnRzX2VtYWlsQGluc3VyYW5jZXRlY2hub2xvZ2llcy5jb208L0FkZH
JMaW5lPg0KICAgICAgICA8L0VNYWlsQWRkcmVzcz4NCiAgICAgICAgPFBlcnNvbiAvPg0KICAgICAgICA8UHJvZHVjZXI%2BDQ
oqICAgICAgICAgPENhcnJpZXJBcHBvaW50bWVudCBQYXJ0eUlEPSJBZ2VudF9QYXJ0eSI%2BDQoqICAgICAgICAgICA8Q29tcG
FueVByb2R1Y2VySUQ%2BZ3JhbnQ1X2l0XzI8L0NvbXBhbnlQcm9kdWNlckEPg0KICAgICAgICAgICAgPENhcnJpZXJDb2RlPk
lUPC9DYXJyawVyQ29kZT4NCiAgICAgICAgICA8L0NhcnJpZXJBcHBvaW50bWvudD4NCiAgICAgICAgPC9Qcm9kdWNlcj4NCiAg
ICAgIDwvUGFydHk%2BDQogICAgICA8T0xpZkVFeHRlbnNpb24gVmVuZG9yQ29kZT0iMjUiPg0KICAgICAgICA8VXNlcjJvbGVD
b2RlIHRjPSJGawx0ZXJUZXN0Ij5UZXN0MTwvVXNlcjJvbGVDb2RlPg0KICAgICAgICAgPC9PTGlmRUV4dGVuc2lvbj4NCiAgICAgID
xSZWxhdGlvbiBPcmlnaW5hdGluZO9iamVjdElPSJBZ2VudF9QYXJ0eSI%2BDQogICAgICAgIDxSZWxhdGlvbjJvbGVDb2RlIH
RjPSIzNyI%2BT0xJX1JFTF9QUklNQUdFTlQ8L1JlbGF0aW9uUm9sZUNvZGU%2BDQogICAgICA8L1JlbGF0aW9uPg0KICAgIDwv
T0xpZkU%2BDQogIDwvVFhMaWZlumVxdWvzdD4NCjwvVFhMaWZlPg%3D%3D
```

Response

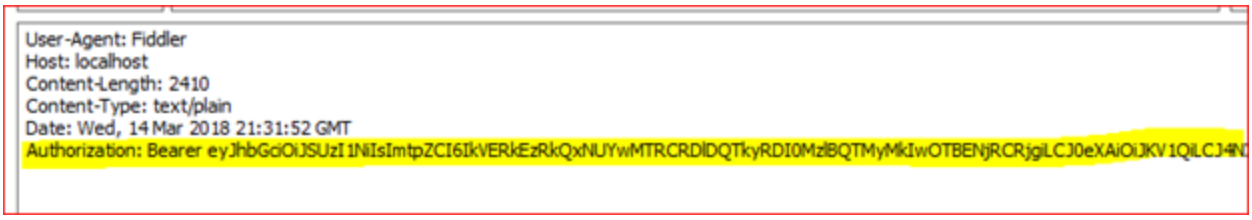
```
HTTP/1.1 200 OK
Cache-Control: no-cache
Pragma: no-cache
Content-Type: application/json; charset=utf-8
Expires: -1
Server: Microsoft-IIS/10.0
X-Powered-By: ASP.NET
X-Content-Type-Options: nosniff
X-XSS-Protection: 1;mode=block
Content-Security-Policy: script-src 'self' 'unsafe-inline' 'unsafe-eval' https://entmax.comm100.com
https://ent.comm100.com https://www.google-analytics.com https://www.google.com/jsapi
Strict-Transport-Security: max-age=31536000;includeSubDomains
X-Frame-Options: SAMEORIGIN
Date: Wed, 14 Mar 2018 21:36:11 GMT
Content-Length: 767

{"access_token":"eyJhbGciOiJSUzI1NiIsImtpZCI6IkVERkEzRkQxNUYwMTRCRDlDQTkyRDI0MzlBQTMyMkIwOTBENjRCR
jqiLCJ0eXAiOiJKV1QiLCJ4NXQiOiI3Zm9fMFY4QlM5bktrdEpEbXFNaXNKRFdTX2cifQ.eyJuYmYiOjE1MjEwNjMzNjYsImV4
cCI6MTUyMTA2NDI2NiwiaXNzIjoiaHR0cDovL2xvY2FsaG9zdDoyODUyNSIsImFlZCI6WyJodHRwOi8vbG9jYWxob3N0OjI4NT
I1L3Jlc291cmNlcyIsImVtYmVkZGVkIl0sImNsawVudF9pZCI6IkluIiwidXNlcl9pZCI6ImdyYW50NV9pdF8yIn0.XNPsxB4O
6EOv3yPd45JHReBWTiMmOvG9fwR9zPAevkNsIxF845HDT4fdz-
Bb6W94rQDrJmpFizc9Oj4YxGq7qOIQsqplG1OC0kqpE2COek9ow6nOm6R5pdY5fK6fL26S7FVJYdIptpsfDK-
y6pwOqd9DHEhYuPNiDB-XYN4rvUPm8U7idD5ZnxZgEDbDQKKaIuVzZsYWK8v1Ip-
BsC_NPQri0nQd38Bc9kSe3vj977ECM61DjxU5IZGQ2EckRyZQ4q6O_bsuLMzDi2WkOEqqje8JO2zz6azeLcPPwPB3co186b5VI
huwGRUc883vK5u5XGBfsZt-4thnOgce2Jv2gg","expires_in":900,"token_type":"Bearer"}
```

Using the Token

For FireLight API service endpoints that use the JWT token security scheme, the token string needs to be placed on any requests made to that service in the 'Authorization' header. The header needs to contain the 'token_type' value received from the response, followed by a space, followed by the 'access_token' value received from the response:

token_type + ' ' + **access_token**

'token_type' will be "Bearer", and so it will look like the following:



Code Sample

Following is a C# code sample of generating the request to get a token.

Code Sample

```
HttpWebRequest req = (HttpWebRequest)WebRequest.Create(
    "https://firelight.insurancetechnologies.com/EGApp/api/Security/GetToken");

req.Method = "POST";
req.ContentType = "text/plain";

string secret = "46098ef35a824170a584bde268586c7b";//this comes from our admin tool
byte[] secretBinary = Encoding.UTF8.GetBytes(secret);
byte[] hashBinary = new SHA256Managed().ComputeHash(secretBinary);

//convert to hex string
StringBuilder builder = new StringBuilder();
for (Int32 idx = 0; idx < hashBinary.Length; idx++)
{
    builder.Append(hashBinary[idx].ToString("X2"));
}
string hashValue = builder.ToString();//this gets appended on the request body

DateTime reqDate = DateTime.UtcNow;
String org = "IT";//this will be different for your organization

//this is the user's 1228 xml - you will need to generate yours as needed
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
string xml1228 = txt1228.Value;
if (!string.IsNullOrWhiteSpace(xml1228))
    xml1228 = Convert.ToBase64String(Encoding.UTF8.GetBytes(txt1228.Value));

//remember, the user 1228 is optional
String textToSign = secret + reqDate.ToString("MMddyyyyHHmmss") + org + xml1228;
byte[] nonSignedBinary = Encoding.UTF8.GetBytes(textToSign);

//load up the test cert - in this example, it comes from a pfx file in a local directory -
you
// may need to get it from your machine's certificate store
X509Certificate2 cert = new X509Certificate2(Path.Combine(
    AppDomain.CurrentDomain.BaseDirectory, txtPfxFile.Text), txtPfxPassword.Text);

RSA rsa = cert.GetRSAPrivateKey();
byte[] signedBinary = rsa.SignData(nonSignedBinary, HashAlgorithmName.SHA256,
    RSASignaturePadding.Pkcs1);

//base64-encoded, then url-encoded
String signed64 = HttpUtility.UrlEncode(Convert.ToBase64String(signedBinary));

//optionally place the user's 1228 on the request if it has value (url-encode it)
String body = $"grant_type=hashsig&id={org}&secret={hashValue}&sig={signed64}"
    + (string.IsNullOrWhiteSpace(xml1228) ? "" : $"&xml={HttpUtility.UrlEncode(xml1228)}");

byte[] bodyBinary = Encoding.UTF8.GetBytes(body);

req.Date = reqDate;
req.ContentLength = bodyBinary.LongLength;

using (Stream post = req.GetRequestStream())
{
    post.Write(bodyBinary, 0, bodyBinary.Length);
}

// Get response
String result = "";
using (HttpWebResponse response = (HttpWebResponse)req.GetResponse())
{
    StreamReader reader = new StreamReader(response.GetResponseStream());
    result = reader.ReadToEnd();
}
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

FireLight REST API

Step through the following topics to find information about the FireLight REST API.

Overview

The REST-based API services utilize JSON web tokens for security and access. Accessing a FireLight API service endpoint requires a JWT token first received from the FireLight token endpoint. Getting access to a service endpoint is a two-step process:

1. A call to the FireLight Token endpoint to get a token. Note this requires credentials pre-defined for your organization. The token is returned in an encoded string format.
2. Embedding the received encoded token string into the subsequent service endpoint call's Authorization header.

    - FireLight API tokens can be used multiple times within their time limit window. A FireLight API token expires after 15 minutes, after which you will need to get a new one.

Please read Getting the Token **for a detailed description on how to retrieve a token.**

Endpoint Root URLs

Endpoint Root URLs

Environment	URL
QA	**Base URL:** https://flqa.insurancetechnologies.com/EGApp/api **Token:** /Security/GetToken
QA NEXT	**Base URL:** https://flqanext.insurancetechnologies.com/EGAppNext/api
UAT	**Base URL:** https://uat.firelighteapp.com/EGApp/api
STAGING	**Base URL:** https://staging.firelighteapp.com/EGApp/api

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Environment	URL
PRODUCTION	**Base URL:** https://www.firelighteapp.com/EGApp/api

** NEXT environments are early release environments and may not always be available.

Activity Methods

Retrieve activities and related information.

CreateActivity

HTTP POST to create an activity with the specified request body.

Requires an API Token with an associated user.

URL

{rootURL}/Activity/CreateActivity

Request body *required

CreateActivityRequest Model. The request body needed to create an activity.

Response

The CreateActivity method returns a CreateActivityResponse Model.

Example

CreateActivty

```
string url =
"https://firelight.insurancetechnologies.com/EGApp/api/Activity/CreateActivity";
string body = "{ 'Id': '2001ec54-8ac9-4c86-a23b-553aa6e5e3de', 'CUSIP': 'FSEFA',
'Jurisdiction': 7, 'ActivityName': 'Smith Illustration', 'AcordTypeCode': 111,
'CarrierCode': 'FLI', 'DataItems': [ {'DataItemId':'Owner_NonNaturalName','Value':'API
Owner'}] }";
string token = GetToken();

try
{
    HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

    request.ContentType = "application/json";
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
        request.Headers.Add("authorization", "Bearer " + token);
        request.Method = "POST";

        Byte[] contentBytes = Encoding.UTF8.GetBytes(body);
        request.ContentLength = contentBytes.LongLength;

        // Send request
        using (Stream post = request.GetRequestStream())
        {
            post.Write(contentBytes, 0, contentBytes.Length);
        }

        using (HttpWebResponse response = (HttpWebResponse) request.GetResponse())
        {
            StreamReader reader = new StreamReader(response.GetResponseStream());
            string result = reader.ReadToEnd();
            Console.WriteLine(result);
        }
    }
    catch (WebException ex)
    {
        if (ex.Response != null && ex.Response.ContentLength > 0)
        {
            using (StreamReader s = new StreamReader(ex.Response.GetResponseStream()))
            {
                string message = s.ReadToEnd();
                Console.WriteLine(message);
            }
        }
    }
```

DeleteActivity

HTTP GET to delete any specific activity.

Requires an API Token with an associated user.

URL

{rootURL}/Activity/DeleteActivity?activityID=XXX

Parameters

- **activityID**: *string* ***required**

  The unique ID of the activity to be deleted.

Response

The GeneralAPIResponse model will be returned. The ResultCode in model will indicate if the request was successful or not. This model will return error message, if there is failure.

Example

DeleteActivity

```
Example
            string url = "https://firelight.insurancetechnologies.com/EGApp/api/Activity/
DeleteActivity?activityID=" + activityId;
            String token = GetToken();
            try
            {
                HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

                request.ContentType = "application/json";
                request.Headers.Add("authorization", "Bearer " + token);
                request.Method = "GET";

                using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
                {
                    StreamReader reader = new StreamReader(response.GetResponseStream());
                    string data = reader.ReadToEnd();
                    Console.WriteLine(data);

                    GeneralAPIResponse result = JsonConvert.DeserializeObject(data, typeof
(GeneralAPIResponse)) as GeneralAPIResponse;
                }
            }
            catch (WebException ex)
            {
                if (ex.Response != null && ex.Response.ContentLength > 0)
                {
                    using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
()))
                    {
                        string message = s.ReadToEnd();
                        Console.WriteLine(message);
                    }
                }
                throw;
            }
```

GetActivities

HTTP GET to query for a list of activities for the user associated to the auth token (IS4) that was passed in the header.

Requires an API Token with an associated user.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

URL

{rootURL}/Activity/GetActivities

Parameters

None

Response

The GetActivities method returns a GetActivities model

Example

GetActivities

```
GetActivitiesResponse result = null;

            string url =
"https://firelight.insurancetechnologies.com/EGApp/api/Activity/GetActivities";
            String token = String.IsNullOrWhiteSpace(Token) ? GetToken() : Token;
            try
            {
                HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

                request.ContentType = "application/json";
                request.Headers.Add("authorization", "Bearer " + token);
                request.Method = "GET";

                using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
                {
                    StreamReader reader = new StreamReader(response.GetResponseStream());
                    string data = reader.ReadToEnd();
                    Console.WriteLine(data);

                    result = JsonConvert.DeserializeObject(data, typeof
(GetActivitiesResponse)) as GetActivitiesResponse;
                }
            }
            catch (WebException ex)
            {
                if (ex.Response != null && ex.Response.ContentLength > 0)
                {
                    using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
()))
                    {
                        string message = s.ReadToEnd();
                        Console.WriteLine(message);
                    }
                }
                throw;
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
            }
```

HTTP POST to get list of activities with specified DataItems for the user associated to the auth token (IS4) that was passed in the header.

Requires an API Token with an associated user.

URL

{rootURL}/Activity/GetActivities

Request body *required

The CreateActivityRequest body needed to get a list of activities.

Response

The GetActivities method returns a GetActivities Model

If DataItems are passed as null or empty, then it will return null in DataItems.

Example

GetActivities

```
GetActivitiesResponse result = null;

            string url =
"https://firelight.insurancetechnologies.com/EGApp/api/Activity/GetActivities";
            String token = String.IsNullOrWhiteSpace(Token) ? GetToken() : Token;
     string body = "{ 'DataItems': [ {'DataItemId':'Owner_FirstName'},{'DataItemId':'Owner_
LastName'}] }";

            try
            {
                HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

                request.ContentType = "application/json";
                request.Headers.Add("authorization", "Bearer " + token);
                request.Method = "POST";
    Byte[] contentBytes = Encoding.UTF8.GetBytes(body);
                request.ContentLength = contentBytes.LongLength;
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
// Send request
    using (Stream post = request.GetRequestStream())
    {
        post.Write(contentBytes, 0, contentBytes.Length);
    }


using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
                {
                    StreamReader reader = new StreamReader(response.GetResponseStream());
                    string data = reader.ReadToEnd();
                    Console.WriteLine(data);

                    result = JsonConvert.DeserializeObject(data, typeof
(GetActivitiesResponse)) as GetActivitiesResponse;
                }
            }
            catch (WebException ex)
            {
                if (ex.Response != null && ex.Response.ContentLength > 0)
                {
                    using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
()))
                    {
                        string message = s.ReadToEnd();
                        Console.WriteLine(message);
                    }
                }
                throw;
            }
```

GetActivityDetails

HTTP GET to query for information about a specific activity.

Requires an API Token with an associated user.

URL

{rootURL}/Activity/GetActivityDetails?activityId=XXX

Parameters

- **activityId**: *string* ***required**

  The unique ID of the activity to be retrieved

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Response

The GetActivityDetails model for the requested activity is returned. If the GetActivityDetails method was not able to locate the activityId that was passed to it, the GetActivityDetails Model is populated with default values.

Example

GetActivityDetails - GET

```
Activity result = null;
            string url =
"https://firelight.insurancetechnologies.com/EGApp/api/Activity/GetActivityDetails?activityI
d=" + activityId;
            String token = GetToken();
            try
            {
                HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

                request.ContentType = "application/json";
                request.Headers.Add("authorization", "Bearer " + token);
                request.Method = "GET";

                using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
                {
                    StreamReader reader = new StreamReader(response.GetResponseStream());
                    string data = reader.ReadToEnd();
                    Console.WriteLine(data);

                    result = JsonConvert.DeserializeObject(data, typeof(Activity)) as
Activity;
                }
            }
            catch (WebException ex)
            {
                if (ex.Response != null && ex.Response.ContentLength > 0)
                {
                    using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
()))
                    {
                        string message = s.ReadToEnd();
                        Console.WriteLine(message);
                    }
                }
                throw;
            }
```

HTTP POST to get information about a specific activity with list of DataItems passed with request.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Requires an API Token with an associated user.

URL

{rootURL}/Activity/GetActivityDetails

Request body *required

ActivityDetailRequest Model.

The request body needed to get an activity details

Response

The GetActivityDetails model for the requested activity is returned. If the GetActivityDetails method was not able to locate the activityId that was passed to it, the GetActivityDetails Model is populated with default values.

Example

GetActivityDetails - POST

```
          Activity result = null;
            string url =
"https://firelight.insurancetechnologies.com/EGApp/api/Activity/GetActivityDetails"

    string body = "{ 'Id': '2001ec54-8ac9-4c86-a23b-553aa6e5e3de',
              'DataItems': [ {'DataItemId':'Owner_FirstName'},{'DataItemId':'Owner_
LastName'}] }";

            String token = GetToken();
            try
            {
                HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

request.ContentType = "application/json";
                request.Headers.Add("authorization", "Bearer " + token);
                request.Method = "POST";

             Byte[] contentBytes = Encoding.UTF8.GetBytes(body);
                request.ContentLength = contentBytes.LongLength;
// Send request
    using (Stream post = request.GetRequestStream())
    {
        post.Write(contentBytes, 0, contentBytes.Length);
    }
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
                using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
                {
                    StreamReader reader = new StreamReader(response.GetResponseStream());
                    string data = reader.ReadToEnd();
                    Console.WriteLine(data);

                    result = JsonConvert.DeserializeObject(data, typeof(Activity)) as
Activity;
                }
            }
            catch (WebException ex)
            {
                if (ex.Response != null && ex.Response.ContentLength > 0)
                {
                    using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
()))
                    {
                        string message = s.ReadToEnd();
                        Console.WriteLine(message);
                    }
                }
                throw;
            }
```

GetActivityPDF

HTTP POST to get PDF streams for the acitivities with the specified request body.

Requires an API Token with an associated user.

URL

{rootURL}/Activity/GetActivityPDF

Request body *required

GetActivityPDFRequest Model. The request body needed to retrieve activity PDFs. If DocumentIds is null or empty, then all documents for the corresponding ActivityId will be returned.

Response

The GetActivityPDF method returns a GetActivityPDFResponse Model

Example

GetActivityPDF

```
            string url =
"https://firelight.insurancetechnologies.com/EGApp/api/Activity/GetActivityPDF";
            string body = "{ 'ActivityId': ['2001ec54-8ac9-4c86-a23b-553aa6e5e3de'],
'DocumentIds': [] }";
            string token = GetToken();
            try
            {
                HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);
                request.ContentType = "application/json";
                request.Headers.Add("authorization", "Bearer " + token);
                request.Method = "POST";
                Byte[] contentBytes = Encoding.UTF8.GetBytes(body);
                request.ContentLength = contentBytes.LongLength;
                // Send request
                using (Stream post = request.GetRequestStream())
                {
                    post.Write(contentBytes, 0, contentBytes.Length);
                }
                using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
                {
                    StreamReader reader = new StreamReader(response.GetResponseStream());
                    string result = reader.ReadToEnd();
                    Console.WriteLine(result);
                }
            }
            catch (WebException ex)
            {
                if (ex.Response != null && ex.Response.ContentLength > 0)
                {
                    using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
()))
                    {
                        string message = s.ReadToEnd();
                        Console.WriteLine(message);
                    }
                }
            }
```

GetAvailableActions

HTTP GET to query for a list of available actions for an application for the user associated to the auth token (IS4) that was passed in the header.

Requires an API Token with an associated user.

URL

{rootURL}/Activity/GetAvailableActions

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Parameters

- **activityId**: string ***required**

  The unique ID of the activity to get the actions for

Response

The GetAvailableActions method returns an AvailableActionsReturn model

Example

GetAvailableActions

```
string url = "https://firelight.insurancetechnologies.com/EGApp/api/Activity/
GetAvailableActions?activityID=" + activityId;
String token = GetToken();
try
{
    HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

    request.ContentType = "application/json"; request.Headers.Add("authorization", "Bearer "
+ token); request.Method = "GET";

    using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
    {
        StreamReader reader = new StreamReader(response.GetResponseStream()); string data =
reader.ReadToEnd();
        Console.WriteLine(data);

        GeneralAPIResponse result = JsonConvert.DeserializeObject(data, typeof
        (GeneralAPIResponse)) as GeneralAPIResponse;
    }
}
catch (WebException ex)
{
    if (ex.Response != null && ex.Response.ContentLength > 0)
    {
        using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
        ()))
        {
            string message = s.ReadToEnd(); Console.WriteLine(message);
        }
    }
    throw;
}
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

GetDocumentList

HTTP GET to query for the documents of a specific activity.

Requires an API Token with an associated user.

URL

{rootURL}/Activity/GetDocumentList?activityId=XXX

Parameters

- **activityId**: *string* ***required**

  The unique ID of the activity to retrieve documents for

Response

The GetDocumentListResponse model for the requested activity is returned. If the GetDocumentList method was not able to locate the activityId that was passed to it, the GetDocumentList Model will contain an empty list.

Example

GetDocumentList

```
            Activity result = null;
            string url =

"https://firelight.insurancetechnologies.com/EGApp/api/Activity/GetDocumentList?activityId="
            + activityId;
            String token = GetToken();
            try
            {
                HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);
                request.ContentType = "application/json";
                request.Headers.Add("authorization", "Bearer " + token);
                request.Method = "GET";
                using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
                {
                    StreamReader reader = new StreamReader(response.GetResponseStream());
                    string data = reader.ReadToEnd();
                    Console.WriteLine(data);
                    result = JsonConvert.DeserializeObject(data, typeof(Activity)) as
Activity;
                }
            }
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
            catch (WebException ex)
            {
                if (ex.Response != null && ex.Response.ContentLength > 0)
                {
                    using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
()))
                    {
                        string message = s.ReadToEnd();
                        Console.WriteLine(message);
                    }
                }
                throw;
            }
```

GetProductAvailability

/ProductAvailability/GetProductAvailability

HTTP Post that returns a list of available products.

Requires API Token, but does not need an API Token with an associated user.

Request body ProductAvailabilityRequestModel

Contains optional filter Parameters. Allowed to be empty ("{}").

Response

If the request was successful, a ProductAvailabilityResponse Model will return a List of AvailableProductResponse which contains the CUSIP, ProductName, CarrierCode, list of Jurisdiction Ids, Locale, and ProductType. ProductAvailabilityResponse will return a ResultCode and ExceptionError if there is a failure.

Example

GetProductAvailability

```
try
            {
          String requestJSON = "{AcordType : '103'}" ;
          String URL = ".../EGApp/api/ProductAvailability/GetProductAvailability";
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
            String token = GetToken("IT", "00000000-0000-0000-0000-000000000001");
                  HttpWebRequest request = (HttpWebRequest)WebRequest.Create(URL);
                  request.Headers["Authorization"] = "Bearer " + token;
                  request.Date = DateTime.UtcNow;

                  // support Json or XML
                  request.ContentType = CustomMediaTypes.JSON;
                  request.Method = "POST";
                  Byte[] contentBytes = Encoding.UTF8.GetBytes(requestJSON);
                  request.ContentLength = contentBytes.LongLength;

                  // Send request
                  using (Stream post = request.GetRequestStream())
                  {
                      post.Write(contentBytes, 0, contentBytes.Length);
                  }
                  // Get response
                  String result = "";
                  using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
                  {
                      StreamReader reader = new StreamReader(response.GetResponseStream());
                      result = reader.ReadToEnd();
                  }


   ProductAvailabilityResponse ProductAvailability =
JsonConvert.DeserializeObject<ProductAvailabilityResponse>(result);
                  if(ProductAvailability.ResultCode == ReturnStatus.RESULT_SUCCESS ||
ProductAvailability.ResultCode == ReturnStatus.RESULT_SUCCESSINFO)
                  {
                      foreach(AvailableProductResponse product in
ProductAvailability.Products)
                      {
                          Debug.Print("Product ID: " + product.CUSIP);
                          Debug.Print("Carrier: " + product.CarrierCode);
                          Debug.Print("Product Name: " + product.ProductName);
                          Debug.Print("Locale: " + product.Locale);
                          Debug.Print("Product Type: " + product.Type);
                          foreach (long Jursidiction in product.Jurisdictions)
                          {
                              Debug.Print("Approved In: " + Jursidiction);
                          }

                      }
                  }
                  else
                  {
                      Debug.Print("Failure " + ProductAvailability.ExceptionError?.Message);
                  }
              }
              catch (WebException ex)
              {
                  txtResponse.Value = ex.Message;
                  if (ex.Response != null && ex.Response.ContentLength > 0)
                  {
                      using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
()))
```

```
                    {
                        txtResponse.Value += Environment.NewLine + Environment.NewLine +
s.ReadToEnd();
                    }
                }
            }
            catch (Exception ex)
            {
                txtResponse.Value = ex.Message;
            }
```

NOTE: Product Availability Calls In the Provider Restrict Choices. With a Value of 99999 for Current Carrier and Product Type, and 0 for Jurisdiction. This is to indicate that these values are not specified. It up to the provider to decide how to restrict Carriers, Jurisdictions, Product Types, and Products based on these values.

ProcessEDeliveryCase

Create or update an eDelivery application and related data and return the response.

Requires API Token, but does not need an API Token with an associated user.

URL

{rootURL}/EDelivery/ProcessEDeliveryCase

Request body *required

The TX121 request body for creating and updating an eDelivery activity

Response

The ProcessEDeliveryCase method returns a TXLifeResponse with a TransResult ResultCode tc value of “1” if the request was successful, otherwise the tc value is a different integer value and the reason for the failed is in the text of the ResultCode tag.

Example

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

ProcessEDeliveryCase

```
public IActionResult ProcessEDelivery()
        {
            ProcessEDeliveryModel result = new ProcessEDeliveryModel();

            string url =
"https://firelight.insurancetechnologies.com/EGApp/api/EDelivery/ProcessEDeliveryCase";
            string body = @"
<TXLife xmlns:xsd=""http://www.w3.org/2001/XMLSchema""
xmlns:xsi=""http://www.w3.org/2001/XMLSchema-instance"" Version=""2.37.00""
xmlns=""http://ACORD.org/Standards/Life/2""">
    <TXLifeRequest id=""_b82a4cd5-4361-a6dc-f780-f3a2fcda7c4e"" PrimaryObjectID=""Holding_
PP6544PJ6590""">
        <TransRefGUID>" + result.CurrentGuid.ToString() + @"</TransRefGUID>
        <TransType tc=""121"">General Requirement Order Request </TransType>
        <TransSubType tc=""12100"">General Requirement Order Request </TransSubType>
        <TransExeDate>2018-11-12</TransExeDate>
        <TransExeTime>10:34:30</TransExeTime>
        <ProcessingInstruction>
            <ProcessingInstructionType tc=""9"">Workflow</ProcessingInstructionType>
            <ProcessingInstructionDesc>Agent Directed</ProcessingInstructionDesc>
        </ProcessingInstruction>
        <TransExeDate/>
        <TransExeTime/>
        <TransMode tc=""2""/>
        <PrimaryObjectType tc=""4""/>
        <OLifE>
            <Holding id=""Holding_PP6544PJ6590"">
                <HoldingTypeCode tc=""2"">Policy</HoldingTypeCode>
                <Policy CarrierPartyID="""">
                    <PolNumber>7550001</PolNumber>
                    <LineOfBusiness tc=""1"">Life</LineOfBusiness>
                    <ProductType tc=""3"">Term</ProductType>
                    <ProductCode>S10P</ProductCode>
                    <CarrierCode>IT</CarrierCode>
                    <PlanName>Flex Term Series X, Renewable Term Life Insurance
Policy</PlanName>
                    <PolicyStatus tc=""8"">Pending</PolicyStatus>
                    <Jurisdiction tc=""1"">Alabama</Jurisdiction>
                    <EffDate>2019-03-28</EffDate>
                    <Life>
                        <InitialPremAmt>50000</InitialPremAmt>
                        <FaceAmt>500000</FaceAmt>
                        <Coverage>
                            <IndicatorCode tc=""""/>
                        </Coverage>
                    </Life>
                    <ApplicationInfo>
                        <SignedDate>2019-05-05</SignedDate>
                        <CWAAmt>50000</CWAAmt>
                        <PremBalDue>450000</PremBalDue>
                    </ApplicationInfo>
                    <RequirementInfo AppliesToPartyID=""Party_77pa65ba9-27eb-4228-962f-
9fda3c14d8ca"">
                        <RequirementInfoUniqueID>SIGN755000101</RequirementInfoUniqueID>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
                        <RequirementDetails/>This may contain extra instructions for the
Fulfiller party or AppliesTo party
                        <ReqStatus tc=""1"">Pending</ReqStatus>
                        <RequestedDate>2019-10-03</RequestedDate>
                        <FulfilledDate>2019-06-08</FulfilledDate>
                        <FulfilledTime>01:01:01</FulfilledTime>
                        <StatusDate>2019-06-08</StatusDate>
                        <StatusTime>01:01:01</StatusTime>
                        <RequirementStatusReason/>
                        <ReqSubStatus tc=""58"">Awaiting Action by Agent</ReqSubStatus>
                    </RequirementInfo>
                    <RequirementInfo AppliesToPartyID=""Party_b27ec71c-7772-4d7e-a89a-
71cb2dcac9a8"">
                        <RequirementInfoUniqueID>SIGN755000102</RequirementInfoUniqueID>
                        <RequirementDetails/>This may contain extra instructions for the
Fulfiller party or AppliesTo party
                        <ReqStatus tc=""1"">Pending</ReqStatus>
                        <RequestedDate>2019-10-03</RequestedDate>
                        <FulfilledDate>2019-06-08</FulfilledDate>
                        <FulfilledTime>01:01:01</FulfilledTime>
                        <StatusDate>2019-06-08</StatusDate>
                        <StatusTime>01:01:01</StatusTime>
                        <RequirementStatusReason/>
                        <ReqSubStatus tc=""58"">Awaiting Action by Agent</ReqSubStatus>
                    </RequirementInfo>
                    <RequirementInfo AppliesToPartyID=""Party_6246b9b2-df6c-43d4-8a41-
f8e4bd72a32a"">
                        <RequirementInfoUniqueID>SIGN755000104</RequirementInfoUniqueID>
                        <RequirementDetails/>This may contain extra instructions for the
Fulfiller party or AppliesTo party
                        <ReqStatus tc=""1"">Pending</ReqStatus>
                        <RequestedDate>2019-10-03</RequestedDate>
                        <FulfilledDate>2019-06-08</FulfilledDate>
                        <FulfilledTime>01:01:01</FulfilledTime>
                        <StatusDate>2019-06-08</StatusDate>
                        <StatusTime>01:01:01</StatusTime>
                        <RequirementStatusReason/>
                        <ReqSubStatus tc=""58"">Awaiting Action by Agent</ReqSubStatus>
                    </RequirementInfo>
                </Policy>
            </Holding>
            <Party id=""Party_00b65ba9-27eb-4228-962f-9fda3c14d8ca"">
                <PartyTypeCode tc=""2"">Organization</PartyTypeCode>
                <FullName>Insurance Technologies</FullName>
                <Carrier>
                    <CarrierCode>IT</CarrierCode>
                    <CarrierForm tc=""1"">Direct Carrier</CarrierForm>
                    <NAICCode>90611</NAICCode>
                    <NAICGroupCode>761</NAICGroupCode>
                </Carrier>
                <Organization>
                    <AbbrName>IT</AbbrName>
                    <DBA>IT</DBA>
                    <OrgCode>IT</OrgCode>
                    <DTCCMemberCode>ABCD</DTCCMemberCode>
                </Organization>
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
            </Party>
            <Party id=""Party_77pa65ba9-27eb-4228-962f-9fda3c14d8ca"">
                <PartyTypeCode tc=""1"">Person</PartyTypeCode>
                <FullName>Mark Agent</FullName>
                <Person>
                    <FirstName>Mark</FirstName>
                    <LastName>Agent</LastName>
                    <BirthDate>1985-01-01</BirthDate>
                </Person>
                <Address>
                    <AddressTypeCode tc=""17""/>
                    <Line1>601 LYONS WAY</Line1>
                    <City>PLACENTIA</City>
                    <AddressStateTC tc=""6"">California</AddressStateTC>
                    <Zip>92870</Zip>
                </Address>
                <EMailAddress>
                    <EMailType tc=""5""/>
                    <AddrLine>email@insurancetechnologies.com</AddrLine>
                </EMailAddress>
                <Producer>
                    <CarrierAppointment id=""CarrierAppointment_4407eb59-45c7-4b54-8ef0-
19a8856968f6"">
                        <CompanyProducerID>00000000-0000-0000-0000-
000000000001</CompanyProducerID>
                        <CarrierCode>IT</CarrierCode>
                        <CarrierName>Insurance Technologies</CarrierName>
                        <CarrierApptTypeCode tc=""""/>
                    </CarrierAppointment>
                </Producer>
            </Party>
            <Party id=""Party_b27ec71c-7772-4d7e-a89a-71cb2dcac9a8"">
                <PartyTypeCode tc=""1"">Person</PartyTypeCode>
                <Person>
                    <FirstName>Simon</FirstName>
                    <LastName>Taylor</LastName>
                    <BirthDate>1955-02-02</BirthDate>
                </Person>
                <EMailAddress>
                    <EMailType tc=""5"">whatever</EMailType>
                    <AddrLine>email@insurancetechnologies.com</AddrLine>
                </EMailAddress>
                <GovtID>222-22-2222</GovtID>
                <GovtIDTC tc=""1"" />
            </Party>
            <Party id=""Party_6246b9b2-df6c-43d4-8a41-f8e4bd72a32a"">
                <PartyTypeCode tc=""1"">Person</PartyTypeCode>
                <Person>
                    <FirstName>Sara</FirstName>
                    <LastName>Taylor</LastName>
                    <BirthDate>1955-03-03</BirthDate>
                </Person>
                <EMailAddress>
                    <EMailType tc=""5"">whatever</EMailType>
                    <AddrLine>email@insurancetechnologies.com</AddrLine>
                </EMailAddress>
                <GovtID>222-55-1287</GovtID>
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
                <GovtIDTC tc=""1"" />
            </Party>
            <Relation id=""Relation_94a85ae3-c7ee-42b6-b787-a259f6acd722""
OriginatingObjectID=""Holding_PP6544PJ6590"" RelatedObjectID=""Party_77pa65ba9-27eb-4228-
962f-9fda3c14d8ca"">
                <OriginatingObjectType tc=""4"">OLI_HOLDING</OriginatingObjectType>
                <RelatedObjectType tc=""6"">OLI_PARTY</RelatedObjectType>
                <RelationRoleCode tc=""11"">OLI_REL_AGENT</RelationRoleCode>
            </Relation>
            <Relation id=""Relation_94a85ae3-c7ee-42b6-b787-a259f6acd723""
OriginatingObjectID=""Holding_PP6544PJ6590"" RelatedObjectID=""Party_b27ec71c-7772-4d7e-
a89a-71cb2dcac9a8"">
                <OriginatingObjectType tc=""4"">OLI_HOLDING</OriginatingObjectType>
                <RelatedObjectType tc=""6"">OLI_PARTY</RelatedObjectType>
                <RelationRoleCode tc=""8"">OLI_REL_OWNER</RelationRoleCode>
            </Relation>
            <Relation id=""Relation_94a85ae3-c7ee-42b6-b787-a259f6acd724""
OriginatingObjectID=""Holding_PP6544PJ6590""  RelatedObjectID=""Party_6246b9b2-df6c-43d4-
8a41-f8e4bd72a32a"">
                <OriginatingObjectType tc=""4"">OLI_HOLDING</OriginatingObjectType>
                <RelatedObjectType tc=""6"">OLI_PARTY</RelatedObjectType>
                <RelationRoleCode tc=""32"">OLI_REL_INSURED</RelationRoleCode>
            </Relation>
            <Relation id=""Relation_94a85ae3-c7ee-42b6-b787-a259f6acd725""
OriginatingObjectID=""Holding_PP6544PJ6590""  RelatedObjectID=""Party_b27ec71c-7772-4d7e-
a89a-71cb2dcac9a8"">
                <OriginatingObjectType tc=""4"">OLI_HOLDING</OriginatingObjectType>
                <RelatedObjectType tc=""6"">OLI_PARTY</RelatedObjectType>
                <RelationRoleCode tc=""31"">OLI_REL_PAYER</RelationRoleCode>
            </Relation>
            <FormInstance id=""FormInstance_60bd354c-b211-423a-8038-9500fb4ff844"">
                <FormName>Policy</FormName>
                <ProviderFormNumber>eDelivery Template Form</ProviderFormNumber>
                <OriginatingTransType tc=""510"">FormInstance Update</OriginatingTransType>
            </FormInstance>
            <FormInstance id=""FormInstance_60bd354c-b111-423a-8038-9500fb4ff844"">
                <FormName>ABC</FormName>
                <ProviderFormNumber>PolicyLife</ProviderFormNumber>
                <OriginatingTransType tc=""510"">FormInstance Update</OriginatingTransType>
            </FormInstance>
        </OLifE>
    </TXLifeRequest>
</TXLife>";
            String token = GetToken("IT", "00000000-0000-0000-0000-000000000001");
            try
            {
                HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);
                request.ContentType = "text/xml";
                request.Headers.Add("authorization", "Bearer " + token);
                request.Method = "POST";

                Byte[] contentBytes = Encoding.UTF8.GetBytes(body);
                request.ContentLength = contentBytes.LongLength;

                // Send request
                using (Stream post = request.GetRequestStream())
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
                {
                    post.Write(contentBytes, 0, contentBytes.Length);
                }

                using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
                {
                    StreamReader reader = new StreamReader(response.GetResponseStream());
                    string res = reader.ReadToEnd();

                    XmlDocument doc = new XmlDocument() { InnerXml = res };

                    if (doc.GetElementsByTagName("ResultCode").Item
(0).Attributes.GetNamedItem("tc").Value == "1")
                    {
                        result.ResponseText = doc.GetElementsByTagName("ResultCode").Item
(0).InnerText;
                    }
                }
            }
            catch (WebException ex)
            {
                if (ex.Response != null && ex.Response.ContentLength > 0)
                {
                    using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
()))
                    {
                        string message = s.ReadToEnd();
                        Console.WriteLine(message);
                    }
                }
                throw;
            }
            return View(result);
        }

    public class ProcessEDeliveryModel
    {
        public ProcessEDeliveryModel()
        {
            CurrentGuid = Guid.NewGuid();
        }
        public String ResponseText { get; set; }
        public Guid CurrentGuid { get; set; }
    }
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

SignatureRequest

Creates a signature request event for the given party information and activity.

Requires API Token, but does not need an API Token with an associated user.

URL

{rootURL}/Activity/SignatureRequest

Request body *required

The TX103 request body for creating a signature request event.

Response

The SignatureRequest method returns a TXLifeResponse with a TransResult ResultCode tc value of “1” if the request was successful, otherwise the tc value is a different integer value and the reason for the failed is in the text of the ResultCode tag. If the request was successful, the response also contains list of “Party” specifying signer information. Each “Party” node contains information about individual signer like first name, last name, signer type and signature url link.

Example of response (if request was successful)

SignatureRequest - Successful


```xml
<?xml version="1.0"?>
<TXLife xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://ACORD.org/Standards/Life/2">
  <TXLifeResponse>
    <TransRefGUID>2e8de4df-4253-4696-b21b-0c3018a06b7f</TransRefGUID>
    <TransType tc="103" />
    <TransExeDate>2020-02-19</TransExeDate>
    <TransExeTime>12:52:26.6744559+05:30</TransExeTime>
    <TransResult>
      <ResultCode tc="1">RESULT_SUCCESS</ResultCode>
    </TransResult>
    <OLifE>
      <Holding id="Holding_4099560a-6af5-45a5-8c8b-8dab3778fba1">
        <HoldingKey>af17234846744109ab6f26b2329f233b</HoldingKey>
      </Holding>
      <Party id="Party_05f86226-9887-40c4-9bac-7f7e1075faa0">
        <PartyTypeCode tc="1">OLI_PT_PERSON</PartyTypeCode>
        <Person>
          <FirstName>Mark</FirstName>
          <LastName>Smith</LastName>
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
        </Person>
        <OLifEExtension VendorCode="25">
          <Signer SignerType="18">Joint Owner</Signer>
          <Url>https://localhost/R1Z</Url>
        </OLifEExtension>
      </Party>
      <Party id="Party_608bbab0-7b13-42ad-a32a-363d323cd16f">
        <PartyTypeCode tc="1">OLI_PT_PERSON</PartyTypeCode>
        <Person>
          <FirstName>Jane</FirstName>
          <LastName>Smith</LastName>
        </Person>
        <OLifEExtension VendorCode="25">
          <Signer SignerType="16">Owner</Signer>
          <Url>https://localhost/m3j</Url>
        </OLifEExtension>
      </Party>
    </OLifE>
  </TXLifeResponse>
</TXLife>
```

The above response example denotes that API has sent two signature request for “Owner” and “Joint Owner”. The example has two “Party” nodes, One for “Owner” and another for “Joint Owner”. “Party” node also contains another information of signer like first name, last name and signature url link.

Example

SignatureRequest

```
public IActionResult SignatureRequest()
        {
            SignatureRequestModel result = new SignatureRequestModel();
            string url =
"https://firelight.insurancetechnologies.com/EGApp/api/Activity/SignatureRequest";
            string body = @"
<TXLife xmlns=""http://ACORD.org/Standards/Life/2"">
  <TXLifeRequest>
    <TransRefGUID>" + result.CurrentGuid.ToString() + @"</TransRefGUID>
    <TransType tc=""103"">OLI_TRANS_NBSUB</TransType>
    <TransExeDate>2010-10-15</TransExeDate>
    <TransExeTime>08:41:17.3373644-06:00</TransExeTime>
    <ProcessingInstruction>
        <ProcessingInstructionType tc=""9"">Workflow</ProcessingInstructionType>
        <ProcessingInstructionDesc>signatures</ProcessingInstructionDesc>
    </ProcessingInstruction>
    <OLifE>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
<SourceInfo>
  <SourceInfoName>ForeSight Enterprise</SourceInfoName>
  <SourceInfoDescription>Version: 5.2.0.344</SourceInfoDescription>
  <FileControlID>6aec9cec-4cd8-48fd-959d-17952d07fa3c</FileControlID>
</SourceInfo>
<Holding id=""9393d31c-7e09-4f4b-a27d-83dafdd542c3"">
  <HoldingTypeCode tc=""6""></OLI_HOLDTYPE_POLICY</HoldingTypeCode>
  <Policy>
    <ProductCode>456</ProductCode>
    <CarrierCode>IT</CarrierCode>
    <Jurisdiction tc=""7""></OLI_USA_CO</Jurisdiction>
    <Life>
      <FaceAmt>500000</FaceAmt>
      <Coverage>
        <DeathBenefitAmt>500000</DeathBenefitAmt>
        <LifeParticipant id=""LP_1"" PartyID=""Party1_1"">
          <FaceAmt>500000</FaceAmt>
          <LifeParticipantRoleCode tc=""1""></Primary Insured</LifeParticipantRoleCode>
          <TobaccoPremiumBasis tc=""2""></non smoker</TobaccoPremiumBasis>
          <PermTableRating tc=""6""></175%</PermTableRating>
          <UnderwritingClass tc=""1""></Standard</UnderwritingClass>
          <UnderwritingSubClass tc=""2""></Better</UnderwritingSubClass>
          <PermRatingType tc=""4""></Occupation</PermRatingType>
          <ClassName></Standard Better</ClassName>
          <PermTableRatingAlphaCode></5</PermTableRatingAlphaCode>
        </LifeParticipant>
      </Coverage>
    </Life>
    <ApplicationInfo>
      <RequestedIssueDate></2010-10-15</RequestedIssueDate>
    </ApplicationInfo>
  </Policy>
</Holding>
<Party id=""d3f24794-c1e1-4d7d-947f-39a47a75d426"">
  <PartyTypeCode tc=""1""></OLI_PT_PERSON</PartyTypeCode>
  <Person>
    <FirstName>Devel</FirstName>
    <LastName>User1</LastName>
    <BirthDate>1970-01-01</BirthDate>
  </Person>
  <Address>
    <AddressTypeCode tc=""1""></OLI_ADTYPE_HOME</AddressTypeCode>
    <Line1>1234 Any Street</Line1>
    <Line2>Suite 1</Line2>
    <City>Colorado Springs</City>
    <AddressStateTC tc=""7""></OLI_USA_CO</AddressStateTC>
    <Zip>80903</Zip>
  </Address>
  <Address>
    <AddressTypeCode tc=""2""></OLI_ADTYPE_BUS</AddressTypeCode>
    <Line1>5555 That Road</Line1>
    <Line2>Apt 222</Line2>
    <City>Denver</City>
    <AddressStateTC tc=""7""></OLI_USA_CO</AddressStateTC>
    <Zip>80210</Zip>
  </Address>
  <Phone>
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
          <PhoneTypeCode tc=""2"">OLI_PHONETYPE_BUS</PhoneTypeCode>
          <AreaCode>719</AreaCode>
          <DialNumber>442-6400</DialNumber>
        </Phone>
        <Phone>
          <PhoneTypeCode tc=""4"">OLI_PHONETYPE_BUSFAX</PhoneTypeCode>
          <AreaCode>719</AreaCode>
          <DialNumber>442-6401</DialNumber>
        </Phone>
        <Producer>
          <CarrierAppointment PartyID=""d3f24794-c1e1-4d7d-947f-39a47a75d426"">
            <CompanyProducerID>00000000-0000-0000-0000-000000000002</CompanyProducerID>
            <CarrierCode>IT</CarrierCode>
          </CarrierAppointment>
        </Producer>
      </Party>
      <Party id=""9c593aab-2ce9-448a-a4c4-bb1eee197113"">
        <PartyTypeCode tc=""1""">OLI_PT_PERSON</PartyTypeCode>
        <Person>
          <FirstName>Jane</FirstName>
          <LastName>Smith</LastName>
          <Gender tc=""2""">OLI_GENDER_FEMALE</Gender>
          <BirthDate>1965-10-15</BirthDate>
          <DriversLicenseState tc=""25""">Maryland</DriversLicenseState>
        </Person>
      </Party>
      <Party id=""e0eb6365-fb16-480e-b21f-7ac4bcbff929""">
        <PartyTypeCode tc=""1""">OLI_PT_PERSON</PartyTypeCode>
        <Person>
          <FirstName>Valued</FirstName>
          <LastName>Client</LastName>
          <BirthDate>1965-10-15</BirthDate>
          <DOBEstimated tc=""1""">OLI_BOOL_TRUE</DOBEstimated>
          <Age>45</Age>
        </Person>
      </Party>
      <Relation OriginatingObjectID=""9393d31c-7e09-4f4b-a27d-83dafdd542c3""
RelatedObjectID=""d3f24794-c1e1-4d7d-947f-39a47a75d426""">
        <OriginatingObjectType tc=""4""">OLI_HOLDING</OriginatingObjectType>
        <RelatedObjectType tc=""6""">OLI_PARTY</RelatedObjectType>
        <RelationRoleCode tc=""37""">OLI_REL_PRIMAGENT</RelationRoleCode>
      </Relation>
      <Relation OriginatingObjectID=""9393d31c-7e09-4f4b-a27d-83dafdd542c3""
RelatedObjectID=""9c593aab-2ce9-448a-a4c4-bb1eee197113""">
        <OriginatingObjectType tc=""4""">OLI_HOLDING</OriginatingObjectType>
        <RelatedObjectType tc=""6""">OLI_PARTY</RelatedObjectType>
        <RelationRoleCode tc=""35""">OLI_REL_ANNUITANT</RelationRoleCode>
      </Relation>
      <Relation OriginatingObjectID=""9393d31c-7e09-4f4b-a27d-83dafdd542c3""
RelatedObjectID=""9c593aab-2ce9-448a-a4c4-bb1eee197113""">
        <OriginatingObjectType tc=""4""">OLI_HOLDING</OriginatingObjectType>
        <RelatedObjectType tc=""6""">OLI_PARTY</RelatedObjectType>
        <RelationRoleCode tc=""8""">OLI_REL_OWNER</RelationRoleCode>
      </Relation>
      <Relation OriginatingObjectID=""9393d31c-7e09-4f4b-a27d-83dafdd542c3""
RelatedObjectID=""e0eb6365-fb16-480e-b21f-7ac4bcbff929""">
        <OriginatingObjectType tc=""4""">OLI_HOLDING</OriginatingObjectType>
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
        <RelatedObjectType tc=""6"">OLI_PARTY</RelatedObjectType>
        <RelationRoleCode tc=""34"">OLI_REL_BENEFICIARY</RelationRoleCode>
      </Relation>
    </OLifE>
  </TXLifeRequest>
</TXLife>";

            String token = GetToken("IT", "00000000-0000-0000-0000-000000000001");
            SignatureRequestModel objResult = new SignatureRequestModel();
            try
            {
                HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

                request.ContentType = "text/xml";
                request.Headers.Add("authorization", "Bearer " + token);
                request.Method = "POST";

                Byte[] contentBytes = Encoding.UTF8.GetBytes(body);
                request.ContentLength = contentBytes.LongLength;

                // Send request
                using (Stream post = request.GetRequestStream())
                {
                    post.Write(contentBytes, 0, contentBytes.Length);
                }

                using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
                {
                    StreamReader reader = new StreamReader(response.GetResponseStream());
                    string res = reader.ReadToEnd();

                    XmlDocument doc = new XmlDocument()
                    {
                        InnerXml = res
                    };

var nsmgr = new System.Xml.XmlNamespaceManager(doc.NameTable);
                    nsmgr.AddNamespace("accord", "http://ACORD.org/Standards/Life/2");

                    if (doc.GetElementsByTagName("ResultCode").Item
(0).Attributes.GetNamedItem("tc").Value == "1")
                    {
   result.Signers = new List<Signer>();
                        result.Value = doc.GetElementsByTagName("TransRefGUID").Item
(0).InnerText;
System.Xml.XmlNodeList elemList = doc.GetElementsByTagName("Party");

foreach (System.Xml.XmlNode partyNode in elemList)
                {
                    string fName = partyNode.SelectSingleNode
("accord:Person/accord:FirstName", nsmgr).InnerXml;
                    string lName = partyNode.SelectSingleNode
("accord:Person/accord:LastName", nsmgr).InnerXml;
                    string signer = partyNode.SelectSingleNode
("accord:OLifEExtension/accord:Signer", nsmgr).InnerXml;
                    string url = partyNode.SelectSingleNode
("accord:OLifEExtension/accord:Url", nsmgr).InnerXml;
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
                    Signer signerInfo = new Signer(fName, lName, signer, url);
                    result.Signers.Add(signerInfo);
                }

                    }
                }
            }
            catch (WebException ex)
            {
                if (ex.Response != null && ex.Response.ContentLength > 0)
                {
                    using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
()))
                    {
                        string message = s.ReadToEnd();
                        Console.WriteLine(message);
                    }
                }
                throw;
            }
            return View(result);
        }


public class SignatureRequestModel
    {
        public SignatureRequestModel()
        {
            CurrentGuid = Guid.NewGuid();
        }
        public string Value { get; set; } = String.Empty;
        public Guid CurrentGuid { get; set; }
        public List<Signer> Signers { get; set; }
    }

public class Signer
    {
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public string SignerType { get; set; }
        public string Url { get; set; }

        public Signer(string firstName, string lastName, string signerType, string url)
        {
            FirstName = firstName;
            LastName = lastName;
            SignerType = signerType;
            Url = url;
        }
    }
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

SubmissionConfirmation

HTTP POST to submit a submission confirmation for an activity. The request/response payload is client-specific for their implementation.

URL

{rootURL}/Activity/SubmissionConfirmation

Parameters

A json string with inputs required by the integration provider implementing the call.

Response

A json string with outputs generated by the integration provider implementing the call.

Submit

HTTP POST to submit an application for the user associated to the auth token (IS4) that was passed in the header.

Requires an API Token with an associated user.

URL

{rootURL}/Activity/Submit

Parameters

- **activityId**: string ***required**

  The unique ID of the activity to submit

Response

The Submit method returns a Boolean. This only returns true if the application is not submitted and transitions to submitted.

Example

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Submit

```
string url = "https://firelight.insurancetechnologies.com/EGApp/api/Activity/
Submit?activityID=" + activityId;
String token = GetToken();
try
{
    HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

    request.ContentType = "application/json"; request.Headers.Add("authorization", "Bearer "
+ token); request.Method = "Post";

    using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
    {
        StreamReader reader = new StreamReader(response.GetResponseStream()); string data =
reader.ReadToEnd();
        Console.WriteLine(data);

        GeneralAPIResponse result = JsonConvert.DeserializeObject(data, typeof
        (GeneralAPIResponse)) as GeneralAPIResponse;
    }
}
catch (WebException ex)
{
    if (ex.Response != null && ex.Response.ContentLength > 0)
    {
        using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
        ()))
        {
            string message = s.ReadToEnd(); Console.WriteLine(message);
        }
    }
    throw;
}
```

UnlockActivity

HTTP GET to unlock any specific activity.

Requires an API Token with an associated user.

URL

{rootURL}/Activity/Unlock?activityID=XXX

Parameters

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **activityID**: *string* ***required**
    - The unique ID of the activity to be unlocked.

Response

The GeneralAPIResponse model will be returned. The ResultCode in model will indicate if the request was successful or not. This model will return error message, if there is a failure.

Example

UnlockActivity

```
Example
            string url = "https://firelight.insurancetechnologies.com/EGApp/api/Activity/
Unlock?activityID=" + activityId;
            String token = GetToken();
            try
            {
                HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

                request.ContentType = "application/json";
                request.Headers.Add("authorization", "Bearer " + token);
                request.Method = "GET";

                using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
                {
                    StreamReader reader = new StreamReader(response.GetResponseStream());
                    string data = reader.ReadToEnd();
                    Console.WriteLine(data);

                    GeneralAPIResponse result = JsonConvert.DeserializeObject(data, typeof
(GeneralAPIResponse)) as GeneralAPIResponse;
                }
            }
            catch (WebException ex)
            {
                if (ex.Response != null && ex.Response.ContentLength > 0)
                {
                    using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
()))
                    {
                        string message = s.ReadToEnd();
                        Console.WriteLine(message);
                    }
                }
                throw;
            }
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

HTTP GET that attempts to change a specified activity's status from Locked to Unlocked.

Requires an API Token with an associated user.

URL

{rootURL}/Activity/Unlock?activityID=XXX

Parameters

- **activityID**: *string***required**
    - The unique ID of the activity to be unlocked.

Response

The standard WebAPIResponseModel is returned. If successful returns a ResultCode of 1. If unsuccessful returns an error model.

Example

UnlockActivity

```
Activity result = null;
string url =
"https://firelight.insurancetechnologies.com/EGApp/api/Activity/GetActivityDetails?activityI
d="
+ activityId;
String token = GetToken();
try
{
    HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);
    request.ContentType = "application/json";
    request.Headers.Add("authorization", "Bearer " + token);
    request.Method = "GET";
    using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
    {
        StreamReader reader = new StreamReader(response.GetResponseStream());
        string data = reader.ReadToEnd();
        Console.WriteLine(data);
    }
}
catch (WebException ex)
{
    if (ex.Response != null && ex.Response.ContentLength > 0)
    {
        using (StreamReader s = new StreamReader(ex.Response.GetResponseStream()))
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
        {
            string message = s.ReadToEnd();
            Console.WriteLine(message);
        }
    }
    throw;
}
```

UploadDocument

HTTP POST to upload document(s) for the activity with the specified request body.

Requires an API Token with an associated user.

URL

{rootURL}/Activity/UploadDocument

Request body *required

UploadDocumentRequest Model. The request body needed to upload document for the activity.

Response

The GeneralAPIResponse model will be returned. The ResultCode in model will indicate if the request was successful or not.

Example

UploadDocument

```
string url =
"https://firelight.insurancetechnologies.com/EGApp/api/Activity/UploadDocument";
string body = "{'Id': 'be3bb20a-236f-482f-9336-3e0afb8ded55',
'Attachments': [{'AttachmentName':'TestDoc','AttachmentType': 0, 'DocumentType': 0,
              'DocumentFormat': 3, 'DocumentTag': '',
'AttachmentData64Binary': '<BINARY ATTACHMENT DATA>'}]}";

string token = GetToken();

try
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
{
    HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

    request.ContentType = "application/json";
    request.Headers.Add("authorization", "Bearer " + token);
    request.Method = "POST";

    Byte[] contentBytes = Encoding.UTF8.GetBytes(body);
    request.ContentLength = contentBytes.LongLength;

    // Send request
    using (Stream post = request.GetRequestStream())
    {
        post.Write(contentBytes, 0, contentBytes.Length);
    }

    using (HttpWebResponse response = (HttpWebResponse) request.GetResponse())
    {
        StreamReader reader = new StreamReader(response.GetResponseStream());
        string result = reader.ReadToEnd();
        Console.WriteLine(result);
    }
}
catch (WebException ex)
{
    if (ex.Response != null && ex.Response.ContentLength > 0)
    {
        using (StreamReader s = new StreamReader(ex.Response.GetResponseStream()))
        {
            string message = s.ReadToEnd();
            Console.WriteLine(message);
        }
    }
}
```

ValidateActivityData

/Activity/ValidateActivityData

HTTP post that validates the given activity data, sending back a list of errors if necessary.

Requires API Token, but does not need an API Token with an associated user.

Request body ActivityValidationRequest Model ***required**

A list of data items used for the activity along with a unique ID to identify the request:

- Id
- CUSIP
- Jurisdiction
- CarrierCode
- AcordType
- DataItems
- ReturnUpdatedDataItems

Response

The response contains:

- Id
- ResultCode
- Messages
- ExceptionError
- ValidationMessages
- DataItems (if ReturnUpdatedDateItems is true)

Example

ValidateActivityData

```
Example
    string url =
"https://firelight.insurancetechnologies.com/EGApp/api/Activity/ValidateActivityData";
    string body = “{
  "Id": "a043fc18-7e99-4f72-9877-eb80dd2440cb",
  "CUSIP": "FSEFA",
  "Jurisdiction": "37",
  "CarrierCode": "IT",
  "AcordTypeCode": "111",
  "AcordSubTypeCode": "11101",
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
"ReturnUpdatedDataItems": "false",
"DataItems": [
  {
    "DataItemId": "FLI_APPLICATION_NAME",
    "Value": "TC_1_E2E_PPR 8-19"
  },
  {
    "DataItemId": "Owner_Type",
    "Value": "Individual"
  },
  {
    "DataItemId": "FLI_ISSUED_STATE_CODE",
    "Value": "37"
  },
  {
    "DataItemId": "FLI_DTCC_ASC_FIRM_ID",
    "Value": "3625"
  },
  {
    "DataItemId": "FLI_ORGANIZATION_DTCC_MEMBER_ID",
    "Value": "0725"
  },
  {
    "DataItemId": "Annuity_TaxQualification",
    "Value": "1"
  },
  {
    "DataItemId": "FLI_PRODUCT_CUSPID",
    "Value": "FSEFA"
  },
  {
    "DataItemId": "FLI_PRODUCT_CARRIER_CODE",
    "Value": "IT"
  },
  {
    "DataItemId": "Initial_AlternativeAST_ASAA",
    "Value": "100"
  },
  {
    "DataItemId": "Annuity_Rider_GuaranteedLifetimeWithdrawalBenefit",
    "Value": "1"
  },
  {
    "DataItemId": "FLI_ROLE_CODE",
    "Value": "AGENT"
  },
  {
    "DataItemId": "Agent_FirstName",
    "Value": "InsTech"
  },
  {
    "DataItemId": "Agent_LastName",
    "Value": "Advisor 1"
  },
  {
    "DataItemId": "Agent_Primary_EmailAddress",
    "Value": "Sample@insurancetechnologies.com"
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
    },
    {
      "DataItemId": "Agent_SSN",
      "Value": "321324321"
    },
    {
      "DataItemId": "Agent_FirmName",
      "Value": "DEMO - Insurance (Custom Setup - not for General Use)"
    },
    {
      "DataItemId": "Agent_Primary_PhoneNumber",
      "Value": "7194426401"
    },
    {
      "DataItemId": "FLI_AGENT_ID_NUMBER",
      "Value": "232838"
    },
    {
      "DataItemId": "Owner_DOB",
      "Value": "11/10/1968"
    },
    {
      "DataItemId": "Owner_Relationship",
      "Value": "Primary"
    },
    {
      "DataItemId": "Owner_FirstName",
      "Value": "Premiere"
    },
    {
      "DataItemId": "Owner_LastName",
      "Value": "HDINY"
    },
    {
      "DataItemId": "Owner_Gender",
      "Value": "1"
    },
    {
      "DataItemId": "Owner_MaritalStatus",
      "Value": "0"
    },
    {
      "DataItemId": "Owner_EmpDetails_TimeEmployed",
      "Value": "0"
    },
    {
      "DataItemId": "Owner_ResidentialAddress1",
      "Value": "201 North Broad Street"
    },
    {
      "DataItemId": "Owner_ResidentialAddress_City",
      "Value": "NYC"
    },
    {
      "DataItemId": "Owner_ResidentialAddress_State",
      "Value": "37"
    },
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
    {
      "DataItemId": "Owner_ResidentialAddress_Zipcode",
      "Value": "10001"
    },
    {
      "DataItemId": "JointOwner_Add",
      "Value": "N"
    },
    {
      "DataItemId": "ContingentOwner_Add",
      "Value": "N"
    },
    {
      "DataItemId": "ContingentAnnuitant_Add",
      "Value": "N"
    },
    {
      "DataItemId": "Annuitant_DOB",
      "Value": "11/10/1968"
    },
    {
      "DataItemId": "Annuitant_Relationship",
      "Value": "Primary"
    },
    {
      "DataItemId": "Annuitant_FirstName",
      "Value": "Premiere"
    },
    {
      "DataItemId": "Annuitant_LastName",
      "Value": "HDINY"
    },
    {
      "DataItemId": "Annuitant_Gender",
      "Value": "1"
    },
    {
      "DataItemId": "Annuitant_MaritalStatus",
      "Value": "0"
    },
    {
      "DataItemId": "Annuitant_EmpDetails_TimeEmployed",
      "Value": "0"
    },
    {
      "DataItemId": "Annuitant_SameAsOwner",
      "Value": "Yes"
    },
    {
      "DataItemId": "Annuitant_ResidentialAddress1",
      "Value": "201 North Broad Street"
    },
    {
      "DataItemId": "Annuitant_ResidentialAddress_City",
      "Value": "NYC"
    },
    {
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "DataItemId": "Annuitant_ResidentialAddress_State",
      "Value": "37"
    },
    {
      "DataItemId": "Annuitant_ResidentialAddress_Zipcode",
      "Value": "10001"
    },
    {
      "DataItemId": "JointAnnuitant_Add",
      "Value": "N"
    }
  ]
}}”;
    String token = GetToken();

    ValidateActivityDataResult model;
    try
    {
        HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

        request.ContentType = "application/json";
        request.Headers.Add("authorization", "Bearer " + token);
        request.Method = "POST";

        Byte[] contentBytes = Encoding.UTF8.GetBytes(body);
        request.ContentLength = contentBytes.LongLength;

        // Send request
        using (Stream post = request.GetRequestStream())
        {
            post.Write(contentBytes, 0, contentBytes.Length);
        }

        using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
        {
            StreamReader reader = new StreamReader(response.GetResponseStream());
            string result = reader.ReadToEnd();
            //Console.WriteLine(result);

            model = JsonConvert.DeserializeObject(result, typeof
(ValidateActivityDataResult)) as ValidateActivityDataResult;
        }
    }
    catch (WebException ex)
    {
        if (ex.Response != null && ex.Response.ContentLength > 0)
        {
            using (StreamReader s = new StreamReader(ex.Response.GetResponseStream()))
            {
                string message = s.ReadToEnd();
                Console.WriteLine(message);
            }
        }
        throw;
    }

    return View(model);
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
}
Client Side Code:
<h4 class="display-2 text-center">Validate Activity Data</h4>
<br /><br />
@foreach (var item in Model.ValidationMessages)
{
<dl>
            <dt>@item.DataItemId</dt>
               <dd>@item.Message</dd>
</dl>
}
```

Example Response

Example Response

```
Bad Response with Updated DataItems:
{
  "ValidationMessages": [
    {
      "DataItemid": "FLI_PRODUCT_CUSPID",
      "SenderId": 0,
      "Operation": 0,
      "Message": "Invalid CUSIP or Product Carrier Code.",
      "Flags": 0,
      "OfficeUseOnly": 0
    }
  ],
  "DataItems": [
    {
      "DataItemId": "FLI_APPLICATION_NAME",
      "Value": "TC_1_E2E_PPR 8-19",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_Type",
      "Value": "Individual",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "Version": null
    },
    {
      "DataItemId": "FLI_ISSUED_STATE_CODE",
      "Value": "37",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "FLI_DTCC_ASC_FIRM_ID",
      "Value": "3625",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "FLI_ORGANIZATION_DTCC_MEMBER_ID",
      "Value": "0725",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuity_TaxQualification",
      "Value": "1",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "FLI_PRODUCT_CUSPID",
      "Value": "",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
    {
      "DataItemId": "FLI_PRODUCT_CARRIER_CODE",
      "Value": "IT",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Initial_AlternativeAST_ASAA",
      "Value": "100",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuity_Rider_GuaranteedLifetimeWithdrawalBenefit",
      "Value": "1",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "FLI_ROLE_CODE",
      "Value": "AGENT",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Agent_FirstName",
      "Value": "IT",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Agent_LastName",
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "Value": "Advisor 1",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Agent_Primary_EmailAddress",
      "Value": "Sample@InsuranceTechnologies.com",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Agent_SSN",
      "Value": "321324321",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Agent_FirmName",
      "Value": "DEMO - Insurance (Custom Setup - not for General Use)",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Agent_Primary_PhoneNumber",
      "Value": "7194426401",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "FLI_AGENT_ID_NUMBER",
      "Value": "232838",
      "IsEnabled": false,
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_DOB",
      "Value": "11/10/1968",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_Relationship",
      "Value": "Primary",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_FirstName",
      "Value": "Premiere",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_LastName",
      "Value": "HDINY",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_Gender",
      "Value": "1",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_MaritalStatus",
      "Value": "0",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_EmpDetails_TimeEmployed",
      "Value": "0",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_ResidentialAddress1",
      "Value": "201 North Broad Street",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_ResidentialAddress_City",
      "Value": "NYC",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_ResidentialAddress_State",
      "Value": "37",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_ResidentialAddress_Zipcode",
      "Value": "10001",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "JointOwner_Add",
      "Value": "N",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "ContingentOwner_Add",
      "Value": "N",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "ContingentAnnuitant_Add",
      "Value": "N",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_DOB",
      "Value": "11/10/1968",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
    },
    {
      "DataItemId": "Annuitant_Relationship",
      "Value": "Primary",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_FirstName",
      "Value": "Premiere",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_LastName",
      "Value": "HDINY",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_Gender",
      "Value": "1",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_MaritalStatus",
      "Value": "0",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "DataItemId": "Annuitant_EmpDetails_TimeEmployed",
      "Value": "0",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_SameAsOwner",
      "Value": "Yes",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_ResidentialAddress1",
      "Value": "201 North Broad Street",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_ResidentialAddress_City",
      "Value": "NYC",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_ResidentialAddress_State",
      "Value": "37",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_ResidentialAddress_Zipcode",
      "Value": "10001",
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "JointAnnuitant_Add",
      "Value": "N",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    }
  ],
  "Id": "a043fc18-7e99-4f72-9877-eb80dd2440cb",
  "ResultCode": 5,
  "Messages": [],
  "ExceptionError": null
}

Good Response:
{
  "ValidationMessages": null,
  "Id": "a043fc18-7e99-4f72-9877-eb80dd2440cb",
  "ResultCode": 1,
  "Messages": [],
  "ExceptionError": null
}

Good Response with DataItems:
{
  "ValidationMessages": null,
  "DataItems": [
    {
      "DataItemId": "FLI_APPLICATION_NAME",
      "Value": "TC_1_E2E_PPR 8-19",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_Type",
      "Value": "Individual",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "FLI_ISSUED_STATE_CODE",
      "Value": "37",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "FLI_DTCC_ASC_FIRM_ID",
      "Value": "3625",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "FLI_ORGANIZATION_DTCC_MEMBER_ID",
      "Value": "0725",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuity_TaxQualification",
      "Value": "1",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "FLI_PRODUCT_CUSPID",
      "Value": "FSEFA",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "Version": null
    },
    {
      "DataItemId": "FLI_PRODUCT_CARRIER_CODE",
      "Value": "IT",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Initial_AlternativeAST_ASAA",
      "Value": "100",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuity_Rider_GuaranteedLifetimeWithdrawalBenefit",
      "Value": "1",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "FLI_ROLE_CODE",
      "Value": "AGENT",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Agent_FirstName",
      "Value": "IT",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
    {
      "DataItemId": "Agent_LastName",
      "Value": "Advisor 1",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Agent_Primary_EmailAddress",
      "Value": "sample@InsuranceTechnologies.com",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Agent_SSN",
      "Value": "321324321",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Agent_FirmName",
      "Value": "DEMO - Insurance (Custom Setup - not for General Use)",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Agent_Primary_PhoneNumber",
      "Value": "7194426401",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "FLI_AGENT_ID_NUMBER",
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "Value": "232838",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_DOB",
      "Value": "11/10/1968",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_Relationship",
      "Value": "Primary",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_FirstName",
      "Value": "Premiere",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_LastName",
      "Value": "HDINY",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_Gender",
      "Value": "1",
      "IsEnabled": false,
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_MaritalStatus",
      "Value": "0",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_EmpDetails_TimeEmployed",
      "Value": "0",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_ResidentialAddress1",
      "Value": "201 North Broad Street",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_ResidentialAddress_City",
      "Value": "NYC",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_ResidentialAddress_State",
      "Value": "37",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Owner_ResidentialAddress_Zipcode",
      "Value": "10001",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "JointOwner_Add",
      "Value": "N",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "ContingentOwner_Add",
      "Value": "N",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "ContingentAnnuitant_Add",
      "Value": "N",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_DOB",
      "Value": "11/10/1968",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_Relationship",
      "Value": "Primary",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_FirstName",
      "Value": "Premiere",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_LastName",
      "Value": "HDINY",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_Gender",
      "Value": "1",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_MaritalStatus",
      "Value": "0",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
    },
    {
      "DataItemId": "Annuitant_EmpDetails_TimeEmployed",
      "Value": "0",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_SameAsOwner",
      "Value": "Yes",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_ResidentialAddress1",
      "Value": "201 North Broad Street",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_ResidentialAddress_City",
      "Value": "NYC",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "Annuitant_ResidentialAddress_State",
      "Value": "37",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      "DataItemId": "Annuitant_ResidentialAddress_Zipcode",
      "Value": "10001",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    },
    {
      "DataItemId": "JointAnnuitant_Add",
      "Value": "N",
      "IsEnabled": false,
      "IsReadonly": false,
      "IsVisible": false,
      "CanBeDeleted": false,
      "Source": 1,
      "Unused": false,
      "Version": null
    }
  ],
  "Id": "a043fc18-7e99-4f72-9877-eb80dd2440cb",
  "ResultCode": 1,
  "Messages": [],
  "ExceptionError": null
}
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Illustration - Methods

Retrieve illustration related information.

GetCalculationResults

/Illustration/GetCalculationResults

HTTP Post that runs illustration calculations for the specified JSON input.

Requires API Token, but does not need an API Token with an associated user.

Request body GetIllustrationRequest Model

A list of data items used for the calculations along with a unique ID to identify the request

Response

If the request was successful, a CalculationResponse Model is returned. If the request could not generate calculation output due to errors from running FireLight rules, you will receive a CalculationErrorResult Model back.

Examples

GetCalculationResults

```
Server Side Code:
public IActionResult GetCalculationResults()
{
    string url
=
"https://firelight.insurancetechnologies.com/EGApp/api/Illustration/GetCalculationResults";
    string body =     "{
    \"Id\": \"1cdb9663-54dd-48fa-ae0c-0dce5550d8a6\",  \"DataItems\": [
    {
      \"DataItemId\": \"FLI_PRODUCT_CARRIER_CODE\",
      \"Value\": \"FLI\",
    },
    {
      \"DataItemId\": \"FLI_PRODUCT_CUSPID\",
      \"Value\": \"FSEFA\"
    },
    {
      \"DataItemId\": \"FLI_ROLE_CODE\",
      \"Value\": \"\"
    },
    {
      \"DataItemId\": \"FLI_PRODUCT_TYPE\",
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
      \"Value\": \"9\"
    },
    {
      \"DataItemId\": \"FLI_PRODUCT_LOB\",
      \"Value\": \"2\"
    },
    {
      \"DataItemId\": \"FLI_ISSUED_STATE_CODE\",
      \"Value\": \"7\"
    },
    {
      \"DataItemId\": \"Agent_FirstName\",
      \"Value\": \"AgentFirst\"
    },
    {
      \"DataItemId\": \"Agent_LastName\",
      \"Value\": \"AgentLast\"
    },
    {
      \"DataItemId\": \"Agent_PhoneNumber\",
      \"Value\": \"719-442-6400\"
    },
    {
      \"DataItemId\": \"Agent_EMail\",
      \"Value\": \"aAgentLast@insurancetechnologies.com\"
    },
    {
      \"DataItemId\": \"Agent_FirmName\",
      \"Value\": \"AgentFirm\"
    },
    {
      \"DataItemId\": \"Agent_FirmAddress1\",
      \"Value\": \"2 S. Cascade\"
    },
    {
      \"DataItemId\": \"Agent_FirmAddress_City\",
      \"Value\": \"Colorado Springs\"
    },
    {
      \"DataItemId\": \"Agent_FirmAddress_State\",
      \"Value\": \"7\"
    },
    {
      \"DataItemId\": \"Agent_FirmAddress_Zipcode\",
      \"Value\": \"80903\"
    },
    {
      \"DataItemId\": \"Owner_Type\",
      \"Value\": \"-1\",
    },
    {
      \"DataItemId\": \"Owner_NonNaturalName\",
      \"Value\": \"d \"
    },
    {
      \"DataItemId\": \"Owner_FirstName\",
      \"Value\": \"PrimOwnFirst\"
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
    },
    {
      \"DataItemId\": \"Owner_LastName\",
      \"Value\": \"PrimOwnLast\"
    },
    {
      \"DataItemId\": \"Owner_DOB\",
      \"Value\": \"11/9/1965 12:00:00 AM\"
    },
    {
      \"DataItemId\": \"Owner_Gender\",
      \"Value\": \"1\"
    },
    {
      \"DataItemId\": \"Owner_TaxRate\",
      \"Value\": \"0\"
    },
    {
      \"DataItemId\": \"Annuitant_LastName\",
      \"Value\": \"Annuitant\"
    },
    {
      \"DataItemId\": \"Annuitant_Type\",
      \"Value\": \"-7\"
    },
    {
      \"DataItemId\": \"Annuity_TaxQualification\",
      \"Value\": \"1\"
    },
    {
      \"DataItemId\": \"Annuity_Premium_Initial_Amount\",
      \"Value\": \"12500\"
    }
  ]
}";
    String token = GetToken();

    CalcResponse model;
    try
    {
        HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

        request.ContentType = "application/json";
        request.Headers.Add("authorization", "Bearer " + token);
        request.Method = "POST";

        Byte[] contentBytes = Encoding.UTF8.GetBytes(body);
        request.ContentLength = contentBytes.LongLength;

        // Send request
        using (Stream post = request.GetRequestStream())
        {
            post.Write(contentBytes, 0, contentBytes.Length);
        }

        using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
        {
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
            StreamReader reader = new StreamReader(response.GetResponseStream());
            string result = reader.ReadToEnd();
            //Console.WriteLine(result);

            model = JsonConvert.DeserializeObject(result, typeof(CalcResponse)) as
CalcResponse;
        }
    }
    catch (WebException ex)
    {
        if (ex.Response != null && ex.Response.ContentLength > 0)
        {
            using (StreamReader s = new StreamReader(ex.Response.GetResponseStream()))
            {
                string message = s.ReadToEnd();
                Console.WriteLine(message);
            }
        }
        throw;
    }

    return View(model);
}
Client Side Code:
<h2 class="display-2 text-center">Calculation Results</h2>
<br /><br />

@foreach (CalcModel calcModel in Model.FireLightCalcOutputs.Models)
{
<h4>Caption = @calcModel.Caption</h4>
       <table class="table table-striped">
           @foreach (CalcTable calcTable in calcModel.Tables)
       {
                  <thead>
                         <tr>
                            @foreach (CalcField calcField in calcTable.Fields)
                            {
                                   <td>@calcField.Caption</td>
                            }
</tr>
</thead>
<tbody>
@for (int i = 0; i < calcTable.RowCount; i++)
{
<tr>
@foreach (CalcField calcField in calcTable.Fields)
{
<td>@calcField.Values[i]</td>
}
</tr>
}
</tbody>
<caption>@calcTable.Caption</caption>
}
</table>
<br />
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
<hr />
<br />
}
```

GetIllustrationPDF

/Illustration/GetIllustrationPDF

HTTP post that generates the illustration output for the specified JSON input.

Requires API Token, but does not need an API Token with an associated user.

Request body GetIllustrationRequest Model ***required**

A list of data items used for the calculations along with a unique ID to identify the request

Response

The GetIllustrationPDF method returns a ReportResponse model. Model is returned. If the request could not generate calculation output due to errors from running FireLight rules, you will receive a CalculationErrorResult Model back.

Example

GetIllustrationPDF

```
Server Side Code:
public IActionResult GetIllustrationPDF()
{
    string url =
"https://firelight.insurancetechnologies.com/EGApp/api/Illustration/GetIllustrationPDF";
    string body = "{
    \"Id\": \"1cdb9663-54dd-48fa-ae0c-0dce5550d8a6\",  \"DataItems\": [
    {
      \"DataItemId\": \"FLI_PRODUCT_CARRIER_CODE\",
      \"Value\": \"FLI\",
    },
    {
      \"DataItemId\": \"FLI_PRODUCT_CUSPID\",
      \"Value\": \"FSEFA\"
    },
    {
      \"DataItemId\": \"FLI_ROLE_CODE\",
      \"Value\": \"\"
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
    },
    {
      \"DataItemId\": \"FLI_PRODUCT_TYPE\",
      \"Value\": \"9\"
    },
    {
      \"DataItemId\": \"FLI_PRODUCT_LOB\",
      \"Value\": \"2\"
    },
    {
      \"DataItemId\": \"FLI_ISSUED_STATE_CODE\",
      \"Value\": \"7\"
    },
    {
      \"DataItemId\": \"Agent_FirstName\",
      \"Value\": \"AgentFirst\"
    },
    {
      \"DataItemId\": \"Agent_LastName\",
      \"Value\": \"AgentLast\"
    },
    {
      \"DataItemId\": \"Agent_PhoneNumber\",
      \"Value\": \"719-442-0000\"
    },
    {
      \"DataItemId\": \"Agent_EMail\",
      \"Value\": \"aAgentLast@insurancetechnologies.com\"
    },
    {
      \"DataItemId\": \"Agent_FirmName\",
      \"Value\": \"AgentFirm\"
    },
    {
      \"DataItemId\": \"Agent_FirmAddress1\",
      \"Value\": \"2 S. Cascade\"
    },
    {
      \"DataItemId\": \"Agent_FirmAddress_City\",
      \"Value\": \"Colorado Springs\"
    },
    {
      \"DataItemId\": \"Agent_FirmAddress_State\",
      \"Value\": \"7\"
    },
    {
      \"DataItemId\": \"Agent_FirmAddress_Zipcode\",
      \"Value\": \"80903\"
    },
    {
      \"DataItemId\": \"Owner_Type\",
      \"Value\": \"-1\",
    },
    {
      \"DataItemId\": \"Owner_NonNaturalName\",
      \"Value\": \"d \"
    },
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
    {
      \"DataItemId\": \"Owner_FirstName\",
      \"Value\": \"PrimOwnFirst\"
    },
    {
      \"DataItemId\": \"Owner_LastName\",
      \"Value\": \"PrimOwnLast\"
    },
    {
      \"DataItemId\": \"Owner_DOB\",
      \"Value\": \"11/9/1965 12:00:00 AM\"
    },
    {
      \"DataItemId\": \"Owner_Gender\",
      \"Value\": \"1\"
    },
    {
      \"DataItemId\": \"Owner_TaxRate\",
      \"Value\": \"0\"
    },
    {
      \"DataItemId\": \"Annuitant_LastName\",
      \"Value\": \"Annuitant\"
    },
    {
      \"DataItemId\": \"Annuitant_Type\",
      \"Value\": \"-7\"
    },
    {
      \"DataItemId\": \"Annuity_TaxQualification\",
      \"Value\": \"1\"
    },
    {
      \"DataItemId\": \"Annuity_Premium_Initial_Amount\",
      \"Value\": \"12500\"
    }
  ]
}";
    String token = GetToken();

    IllustrationPDF model;
    try
    {
        HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

        request.ContentType = "application/json";
        request.Headers.Add("authorization", "Bearer " + token);
        request.Method = "POST";

        Byte[] contentBytes = Encoding.UTF8.GetBytes(body);
        request.ContentLength = contentBytes.LongLength;

        // Send request
        using (Stream post = request.GetRequestStream())
        {
            post.Write(contentBytes, 0, contentBytes.Length);
        }
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
        using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
        {
            StreamReader reader = new StreamReader(response.GetResponseStream());
            string result = reader.ReadToEnd();
            //Console.WriteLine(result);

            model = JsonConvert.DeserializeObject(result, typeof(IllustrationPDF)) as
IllustrationPDF;
        }
    }
    catch (WebException ex)
    {
        if (ex.Response != null && ex.Response.ContentLength > 0)
        {
            using (StreamReader s = new StreamReader(ex.Response.GetResponseStream()))
            {
                string message = s.ReadToEnd();
                Console.WriteLine(message);
            }
        }
        throw;
    }

    return View(model);
}

Client Side Code:
<div>
    <h4 class="display-2 text-center">Illustration PDF</h4>
    <embed style="width:100%;height:650px" src="@String.Format("data:application/pdf; base64,
{0}",
    Model.Reports)" type="application/pdf" />
</div>
```

Review Queue Methods

Retrieves Review Queue related information.

GetReviewQueueStates

HTTP POST to get application state in review queue.

Requires an API Token with an associated user.

URL

{rootURL}/ReviewQueue/GetReviewQueueStates

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Request body *required

GetReviewQueueStatesRequest Model.

The request body needed to get an application state in review queue.

Response

The GetReviewQueueStates method returns a ReviewQueueGetStatusResponseModel model

Example

GetReviewQueueStates - POST

```
 ReviewQueueGetStatusResponseModel result = null;
 string url =
"https://firelight.insurancetechnologies.com/EGApp/api/ReviewQueue/GetReviewQueueStates "
 string body = { 'AppCode': 'a8667fdc-d1a2-4669-8235-b6fe8eafe701',
                    'CarrierCode': 'IT'} ";

            String token = GetToken();
            try
            {
                HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

request.ContentType = "application/json";
                request.Headers.Add("authorization", "Bearer " + token);
                request.Method = "POST";

             Byte[] contentBytes = Encoding.UTF8.GetBytes(body);
                request.ContentLength = contentBytes.LongLength;
// Send request
    using (Stream post = request.GetRequestStream())
    {
        post.Write(contentBytes, 0, contentBytes.Length);
    }


                using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
                {
                    StreamReader reader = new StreamReader(response.GetResponseStream());
                    string data = reader.ReadToEnd();
                    Console.WriteLine(data);

                    result = JsonConvert.DeserializeObject(data, typeof
(ReviewQueueGetStatusResponseModel)) as ReviewQueueGetStatusResponseModel;
                }
            }
            catch (WebException ex)
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
            {
                if (ex.Response != null && ex.Response.ContentLength > 0)
                {
                    using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
()))
                    {
                        string message = s.ReadToEnd();
                        Console.WriteLine(message);
                    }
                }
                throw;
            }
```

UpdateReviewQueueState

HTTP POST to get update application state in review queue (Approve/Reject).

Requires an API Token with an associated user.

URL

{rootURL}/ReviewQueue/UpdateReviewQueueState

Request body *required

UpdateReviewQueueStateRequest Model.

The request body needed to get an application state in review queue.

Response

The UpdateReviewQueueState method returns a ReviewQueueUpdateStatusResponseModel model

Example

UpdateReviewQueueState - POST

```
          ReviewQueueUpdateStatusResponseModel result = null;
            string url =
"https://firelight.insurancetechnologies.com/EGApp/api/ReviewQueue/UpdateReviewQueueState"

    string body =” { 'AppCode': 'dcb1955b-c173-4221-9073-bddb0312af90',
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
              'CarrierCode': 'IT', 'NewState': '2', 'QueueNumber':100, 'Remarks': 'Test
comments'}” ;

            String token = GetToken();
            try
            {
                HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

request.ContentType = "application/json";
                request.Headers.Add("authorization", "Bearer " + token);
                request.Method = "POST";

             Byte[] contentBytes = Encoding.UTF8.GetBytes(body);
                request.ContentLength = contentBytes.LongLength;
// Send request
    using (Stream post = request.GetRequestStream())
    {
        post.Write(contentBytes, 0, contentBytes.Length);
    }


                using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
                {
                    StreamReader reader = new StreamReader(response.GetResponseStream());
                    string data = reader.ReadToEnd();
                    Console.WriteLine(data);

                    result = JsonConvert.DeserializeObject(data, typeof
(ReviewQueueUpdateStatusResponseModel)) as ReviewQueueUpdateStatusResponseModel;
                }
            }
            catch (WebException ex)
            {
                if (ex.Response != null && ex.Response.ContentLength > 0)
                {
                    using (StreamReader s = new StreamReader(ex.Response.GetResponseStream
()))
                    {
                        string message = s.ReadToEnd();
                        Console.WriteLine(message);
                    }
                }
                throw;
            }
```

Tele-Sign - Methods

Endpoints to interact with an application’s signature during the Tele-Sign process.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

CompleteSignature

/TeleSign/CompleteSignature

HTTP post that validates input data during a Tele-Sign request.

Requires an API Token with an associated user.

Request body CompleteSignatureRequest ***required**

A list of data items used to validate a signer during the Tele-Sign process.

Response CompleteSignatureResponse

Success response code or unsuccessful response code and an error message.

Example

CompleteSignature

```
*IVRId can be either an IVR call record ID or a contact ID.

Server Side Code:
public IActionResult CompleteSignature()
{
    string url =
"https://firelight.insurancetechnologies.com/EGApp/api/TeleSign/CompleteSignature";
    string body = "{
    TeleSignAppId: \”12345\”,
    ANI: \”8888888888\”,
    DNIS: \”8888888888\”,
    IVRId: \”1111111\”,
    FireLightAppId: \”00000000-0000-0000-0000-000000000000\”,
    CarrierCode: \”IT\”,
    CompletionDate: \“2011-10-05T14:48:00.000Z\”, Declined: false }”;
    String token = GetToken();

    CompleteSignatureResponse model;
    try
    {
        HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

        request.ContentType = "application/json";
        request.Headers.Add("authorization", "Bearer " + token);
        request.Method = "POST";

        Byte[] contentBytes = Encoding.UTF8.GetBytes(body);
        request.ContentLength = contentBytes.LongLength;

        // Send request
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
        using (Stream post = request.GetRequestStream())
        {
            post.Write(contentBytes, 0, contentBytes.Length);
        }

        using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
        {
            StreamReader reader = new StreamReader(response.GetResponseStream());
            string result = reader.ReadToEnd();

            model = JsonConvert.DeserializeObject(result, typeof(CompleteSignatureResponse))
as CompleteSignatureResponse;

            if ((int)model.ResultCode == 1 || (int)model.ResultCode == 2)
            {
                // Request was completed successfully
            }
            else
            {
                // One or more errors occurred
                var errors = model.Messages;
                var fireLightAppId = model.FireLightAppId;
            }
        }
    }
    catch (WebException ex)
    {
      // Request was not successful
        if (ex.Response != null && ex.Response.ContentLength > 0)
        {
            using (StreamReader s = new StreamReader(ex.Response.GetResponseStream()))
            {
                string message = s.ReadToEnd();
                Console.WriteLine(message);
            }
        }
        throw;
    }
}
```

ValidateSignature

/TeleSign/ValidateSignature

HTTP post that validates input data during a Tele-Sign request.

Requires API Token, but does not need an API Token with an associated user.

Request body ValidateSignatureRequest ***required**

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

A set of data items used to validate a signer during the Tele-Sign process.

Response ValidateSignatureResponse

A JSON object containing the validity of the verification information, as well as the FireLight application ID.

Example

ValidateSignature

```
Server Side Code:
public IActionResult ValidateSignature()
{
    string url =
"https://firelight.insurancetechnologies.com/EGApp/api/TeleSign/ValidateSignature";
    string body = "{ TeleSignAppId: \"12345\", TeleSignPIN: \"1234\", DateOfBirth:
\"01/01/1990\", CarrierCode: \"IT\" }";
    String token = GetToken();

    ValidateSignatureResponse model;
    try
    {
        HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);

        request.ContentType = "application/json";
        request.Headers.Add("authorization", "Bearer " + token);
        request.Method = "POST";

        Byte[] contentBytes = Encoding.UTF8.GetBytes(body);
        request.ContentLength = contentBytes.LongLength;

        // Send request
        using (Stream post = request.GetRequestStream())
        {
            post.Write(contentBytes, 0, contentBytes.Length);
        }

        using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
        {
            StreamReader reader = new StreamReader(response.GetResponseStream());
            string result = reader.ReadToEnd();

            model = JsonConvert.DeserializeObject(result, typeof(ValidateSignatureResponse))
as ValidateSignatureResponse;

            if (model.Valid)
            {
                //Validation was successful, all data was verified
                var fireLightAppId = model.FireLightAppId;
            }
            else
            {
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
                //Validation was not successful
                var errors = model.Messages;
            }
        }
    }
    catch (WebException ex)
    {
        if (ex.Response != null && ex.Response.ContentLength > 0)
        {
            using (StreamReader s = new StreamReader(ex.Response.GetResponseStream()))
            {
                string message = s.ReadToEnd();
                Console.WriteLine(message);
            }
        }
        throw;
    }
}
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

FireLight Embedded Client API

Step through the following topics to find information about the FireLight Embedded Client.

Overview

The FireLight embedded client API can be used to integrate FireLight wizards directly into your application. This functionality allows you to re-use FireLight wizards that you build in the FireLight Admin tool within your own application outside of the traditional FireLight frame. With our embedded client API, you are able to control how and where the wizards are embedded within your application. This is not a class iFrame solution. Our embedded client API uses our client side view engine to natively render the FireLight wizard for you into your user's browser's document object model (DOM). This approach gives you the ability to brand the wizard using CSS and subscribe to events so that your application can react to actions within the wizard.

Font Awesome Installation

In order for icons to show in embedded FireLight, the Font Awesome icon library must be present on the hosting page. FireLight uses the Pro version of Font Awesome 5. Embedded users may load the Font Awesome 5 Pro stylesheet directly from FireLight, but the icons may only be used in the FireLight embedded application - hosting applications are prohibited from using Font Awesome 5 Pro icons without purchasing their own license.

If the hosting application already owns Font Awesome 5 Pro, you may continue to use your existing stylesheet and fonts. As long as they are loaded on the hosting page, FireLight can make use of them. If a hosting application would like to use Font Awesome 5 Pro icons outside of the FireLight embedded application, the hosting application must purchase a license directly from Font Awesome at https://fontawesome.com/plans.

In order to show Font Awesome icons in FireLight embedded, please utilize the following stylesheet (font files will be loaded automatically as needed):

Stylesheet URL: {Base URL}/Content/FontAwesome/css/all.css

Example:
https://firelight.insurancetechnologies.com/EGApp/Content/FontAwesome/css/all.css

Getting Started

IMPORTANT! It is highly recommended to link to the FireLight embedded bundle that is hosted on our web site as described below. If your organization is hosting the FireLight bundle content, and you do not update the version hosted on your website with every

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

FireLight release and patch, it will be stale, missing new functionality and bug fixes, and it also will be incompatible with the backend FireLight code hosted on our servers.

To get started with the embedded API, there are a few things that need to occur. The steps listed below allow you to establish a session with FireLight and embed a FireLight wizard within your application.

1. Obtain a token by connecting to our token service described here.

2. Create an activity using the CreateActivity method.

3. Link to the FlexBoxGrid CDN for layout styling.

    - https://cdnjs.cloudflare.com/ajax/libs/flexboxgrid/6.3.1/flexboxgrid.min.css

    - Example: <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/flexboxgrid/6.3.1/flexboxgrid.min.css" type="text/css">

4. Link to the following JavaScript resources (Adjust BASE URL to match target environment):

    a. Embedded Illustration

        i. Format: {BASE URL}/Resource/Scripts/illustration

        ii. Example: https://firelight.insurancetechnologies.com/EGApp/Resource/Scripts/illustration

    b. Embedded Application

        i. Format: {BASE URL}/Resource/Scripts/eApp

        ii. Example: https://firelight.insurancetechnologies.com/EGApp/Resource/Scripts/eApp

        Optionally, to specify the name of the div where embedded FireLight will be rendered, use the “destination” query parameter. Example:

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

https://firelight.insurancetechnologies.com/EGApp/Resource/Scripts/eApp?destination=myCustomDiv

c. E-Signature

i. Format: {BASE URL}/Resource/Scripts/eSign

ii. Example: https://firelight.insurancetechnologies.com/EGApp/Resource/Scripts/eSign

Optionally, to specify the name of the div where embedded FireLight will be rendered, use the “destination” query parameter.

Example: https://firelight.insurancetechnologies.com/EGApp/Resource/Scripts/eSign?destination=myCustomDiv

d. Multiple Resources (i.e. Application and E-Signature)

i. Format: {BASE URL}/Resource/Scripts/eApp~eSign

ii. Example: https://firelight.insurancetechnologies.com/EGApp/Resource/Scripts/eApp~eSign

NOTE: When multiple module resources are loaded loadModule must be used to determine which module to load and display.

Optionally, to specify the name of the div where embedded FireLight will be rendered, use the “destination” query parameter.

Example: https://firelight.insurancetechnologies.com/EGApp/Resource/Scripts/eApp~eSign?destination=myCustomDiv

5. Link to the following CSS resources (Adjust BASE URL to match target environment):

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

a. Embedded Illustration

   i. Format: {BASE URL}/Resource/CSS/illustration

   ii. Example: https://firelight.insurancetechnologies.com/EGApp/Resource/CSS/illustration

b. Embedded Application

   i. Format: {BASE URL}/Resource/CSS/eApp

   ii. Example: https://firelight.insurancetechnologies.com/EGApp/Resource/CSS/eApp

   By default, all available styles, including fonts and colors, will be included in the CSS returned by the calls above. To return only the root styles that provide the framework for rendering the embedded UI, add “rootOnly=true” to the query string of the request. This will reduce the payload of the CSS response by ~30KB and will make setting your own fonts and colors easier.

c. Embedded E-Signature

   i. Format: {BASE URL}/Resource/CSS/eSign

   ii. Example: https://firelight.insurancetechnologies.com/EGApp/Resource/CSS/eSign

6. Create a div element in your web page called fireLightMainDiv.

7. Once the fireLightLoaded event has fired, an instance of a FireLight wizard can be loaded using the init method of the Client API along with the activityId that was returned from the CreateActivity method called above.

Common API Interface

Step through the following to find information about the Common API Interface events and methods.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Events

Events

Event	Defined in / Description
fireLightLoaded	Subscribe to this event to be notified when the FireLight eApp script has completed loading. Once this event has fired, initialization of the module should be done. See init, loadModule and loadEmbeddedESign for details on initialization.
errorEvent	Subscribe to this event to be notified when FireLight has encountered an error. Scenarios where this event could be raised include script errors, issues communicating with the server, or exceptions thrown by the server. This event returns an ErrorEvent model.
navigationEvent	Subscribe to this event to be notified when an action that may affect the navigation graph has completed. This event returns a NavigationEvent model.
statusEvent	Subscribe to this event to be notified when the status of the activity has changed. This event also gets fired after the init method has been called and before the application has completed loading. The status is reflected as a percentage of fields that do not have validation error messages associated with them. This event returns a StatusEvent model.
ruleMessagesEvent	Subscribe to this event to be notified when the activity has received validation error messages. The response will be a RuleMessagesEvent model, keyed by data item ID. Use getRuleMessages to obtain the actual messages.
notificationsEvent	Subscribe to this event to be notified when a notification is raised within FireLight. This event returns a NotificationsEvent model.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

getRuleMessages

Gets all rule messages for the current activity. The response will be a map of error message arrays, keyed by data item ID.

Parameters None

Returns Message Model *object*

init

Initializes embedded mode for a given activity for Data Entry. This method assumes the specified activity has already been created. A new session will be created as a part of the initialization process. This is supported by eApp and Illustration.

NOTE: If resources for multiple modules are loaded loadModule must be called prior to init in order to load the correct module.

Parameters

- **token:** *string*

  The string returned from the API Token Service that uniquely identifies the current call to the client API (for tracking purposes). The string that uniquely identifies the current call to the client API (for tracking purposes).

- **activityId:** *string*

  The GUID of the activity to be loaded.

- **isNewActivity:** *Boolean*

  A Boolean (true/false) value to indicate if the current init operation is for a new activity or an existing activity.

- **enableErrorHandling:** *Boolean*

  A Boolean (true/false) value to indicate whether FireLight should show its own error dialogs when system errors occur. If you set this to false, you should handle errors in your embedding code.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **enableAuthProxy:** *Boolean* – defaults to true

  A Boolean (true/false) value which indicates whether or not the Auth Proxy should be used. The Auth Proxy is used to store cookies for a session so the browser does not have to, letting users load embedded while using browsers that don't support third party cookies (including Safari and Firefox).

Returns *void*

loadModule

Initializes the specified module. This method is only needed when multiple module resources are loaded. It must be called prior to calling init or loadEmbeddedESign.

Parameters

- **module:** *string*

  The string returned from the API Token Service that uniquely identifies the current call to the client API (for tracking purposes). The string that uniquely identifies the current call to the client API (for tracking purposes).

  Valid values: EApp, illustration, eSign

navigateToPage

Navigates to the specified page within the current activity.

Parameters

- **pageNumber:** *number*

  The page number (0-based).

Returns *void*

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

navigateToPageNext

This API loads up the client's application and navigates to the next page if one exists. If one does not exist, the button directs to the last possible page in the application. Navigates to the specified page within the current activity.

Parameters

Returns *void*

The navigateToPageNext and navigateToPagePrevious API calls are intended to be used with client's own Previous/Next navigation buttons. To gain a feel of how these buttons will operate, the client can use the regular FireLight process and experiment with the FireLight existing Previous/Next buttons.

NOTE:

- “last possible page” means the last page on the application that has a control on it.
- “first possible page” means the first page on the application that has a control on it.

If the client would like to navigate to a specific page, use the existing navigateToPage API and simply pass in the specific page the users would like to navigate to.

navigateToPagePrevious

This API loads up the client's application and navigates to the previous page if one exists. If one does not exist, the button directs to the first possible page in the application.

Parameters

Returns *void*

renderPDF

Renders the PDF for the current activity. FireLight will be responsible for rendering the PDF, so no response will be returned by this method.

Parameters None

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Returns *void*

save

Saves the current activity to permanent storage within FireLight.

Parameters None

Returns *void*

setShowAllValidationMessages

Shows or hides all validation messages.

Parameters

- **show:** *boolean*

  Whether all validation messages should be visible.

Returns *void*

showUnlockDialog

Shows the dialog that is used for unlock and set application to data entry mode.

Parameters None

Returns *void*

toggleShowAllValidationMessages

Toggles showing or hiding of all validation messages.

Returns *void*

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Code Sample

Common API Interface Code Sample

```
<div>
    <div style="display:flex;">
        <div style="margin:0px 5px;">
            <button id="btnRenderPDF" class="btn btn-primary">Render PDF</button>
        </div>
        <div style="margin:0px 5px;">
            <button id="btnSave" class="btn btn-primary">Save app</button>
        </div>
 <div style="margin:0px 5px;">
            <button id="btnReload" class="btn btn-primary">Reload</button>
        </div>
    </div>
    <br />
    <div>
        <label for="currentPageNumber">Current Page Number</label>
        <input id="currentPageNumber" type="text" disabled="disabled" value="0" />
    </div>
    <br />
    <div id="btnNavigatePage">
        <label for="pagelbl">Go to page: </label>
        <input id="pagelbl" type="number" />
        <button id="goToPage">Go</button>
    </div>
    <br />
    <div id="btnGetRuleMessages">
        <button>Get Rule Messages</button>
        <div id="ruleMessages"></div>
    </div>
</div>
<p>
    Percent Complete: <div id="activityStatus" class="percentComplete"></div>
</p>

<br /><hr /><br />

<p id="errMessage"></p>

<div class="contents" id="fireLightMainDiv">
    Loading...
</div>

<script type="text/javascript">
    const token = '@(ViewBag.access_token)';
    const activityId = '@(ViewBag.ActivityId)';
    const isNew = '@(ViewBag.activity_isNew)' === 'true';
    const activityStatus = document.getElementById('activityStatus');
    const mainDiv = document.getElementById('fireLightMainDiv');
    const reloadDiv = document.getElementById('btnReload');
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
    const saveDiv = document.getElementById('btnSave');
    const getRuleMsgsDiv = document.getElementById('btnGetRuleMessages');
    const navigatePageDiv = document.getElementById('goToPage');
    const renderPdfDiv = document.getElementById('btnRenderPDF');

    mainDiv.addEventListener('fireLightLoaded', (api) => {
        FireLightAPI.init(token, activityId, isNew);

        FireLightAPI.errorEvent.subscribe(handleErrorEvent);
        FireLightAPI.navigationEvent.subscribe(navigationChange);
        FireLightAPI.statusEvent.subscribe(statusChange);
        FireLightAPI.ruleMessagesEvent.subscribe(ruleMessageChange);
        FireLightAPI.notificationsEvent.subscribe(notificationRaised);

    });

    function handleErrorEvent(err) {
        var msg = err.error.ApiSource + ' - ' + err.error.ErrorMessage;
        document.getElementById('errMessage').innerHTML = 'Error message = ' + msg;
        console.log('errorEvent = ' + msg);
    }

    var statusChange = function (status) {
        console.log('statusEvent Field: percentComplete = ' + status.percentComplete + ',
Field: lastDataItemUpdated = ' + status.lastDataItemUpdated);
        activityStatus.innerText = status.percentComplete + '% Complete';
    }

    var navigationChange = function (nav) {
        console.log(JSON.stringify(nav));
        console.log('navigationChange = Field: module = ' + nav.module + ', Page = ' +
nav.pageNumber);
        document.getElementById('currentPageNumber').value = nav.pageNumber;
    }

    var notificationRaised = function (notification) {
        console.log(JSON.stringify(notification));
    }


    reloadDiv.addEventListener('click', (api) => {
        FireLightAPI.load();
        console.log('reloadDiv clicked - FireLightAPI.load() method called');

        FireLightAPI.statusEvent.subscribe((status) => {
            console.log('reloadDiv click event - statusEvent = ' + JSON.stringify(status));
            statusChange(status);
        });

        FireLightAPI.errorEvent.subscribe((status) => {
            handleErrorEvent(status);
        });
    });

    saveDiv.addEventListener('click', (api) => {
        FireLightAPI.save();
        console.log('reloadDiv clicked - FireLightAPI.save() method called');
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
        FireLightAPI.errorEvent.subscribe((err) => {
            handleErrorEvent(status);
        });
    });

    getRuleMsgsDiv.addEventListener('click', (api) => {
        var elem = document.getElementById('ruleMessages');
        elem.innerHTML += '';

 var msgs = FireLightAPI.getRuleMessages();
        $.each(msgs, function (item, value) {
            $.each(value.Errors, function (key, value2) {
                elem.innerHTML += 'Field: ' + value2.DataItemid + ' - Message: ' +
value2.Message + ' - Page: ' + value.PageNumber + '<br />';
            });
        });
    });

    navigatePageDiv.addEventListener('click', (api) => {
        FireLightAPI.navigateToPage(document.getElementById('pagelbl').value);

        FireLightAPI.errorEvent.subscribe((status) => {
            handleErrorEvent(status);
        });
    });

    renderPdfDiv.addEventListener('click', (api) => {
        FireLightAPI.renderPDF();

        FireLightAPI.errorEvent.subscribe((err) => {
            handleErrorEvent(status);
        });
    });
</script>
<script
src="https://firelight.insurancetechnologies.com/EGApp/Resource/Scripts/eApp"></script>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

E-Application Client API Interface

Step through the following topics to find information about the E-Application Client API Interface events and methods.

Events

Events

Event	Defined in / Description
signatureStatusEvent	Subscribe to this event to be notified of information regarding the status of locking the application and proceeding to signatures. This event returns a SignatureStatusEvent model.

loadEmbeddedEApp

Initializes embedded mode for a given activity for EApp. This method assumes the specified activity has already been created. loadEmbeddedEApp is only supported when the EApp module is loaded. See Getting Started and loadModule to see how to load multiple module resources.

NOTE: If resources for multiple modules are loaded loadModule must be called prior in order to load the correct module.

Parameters

- **activityId:** *string*

  The GUID of the activity to be loaded.

Returns: *void*

proceedToSignatures

Locks the electronic application and proceeds to the signature process. Subscribe to the signatureStatusEvent (see Events) to receive the redirectURL needed to navigate to FireLight Signatures.

Parameters None

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Returns *void*

showDocumentsDialog

Shows the dialog that is used for uploading documents that may be required or optional for the e-application.

Parameters None

Returns *void*

Code Sample

E-Application API Code Sample

```
<p>
    Percent Complete: <div id="activityStatus" class="percentComplete"></div>
</p>
<div style="display:flex;">
    <div>
        <button id="btnProceedToSignatures">Proceed to Signatures</button>
    </div>
    <div>
        <button id="btnShowDocumentsDialog">Show Documents Dialog</button>
    </div>
   <div>
        <button id="btnUnlockApplication">Unlock Application</button>
    </div>
</div>

<br /><hr /><br />

<p id="errMessage"></p>

<div id="fireLightMainDiv">
    Loading...
</div>
<script type="text/javascript">
    const token = '@(ViewBag.access_token)';
    const isNew = '@(ViewBag.activity_isNew)' === 'true';
    const activityId = '@(ViewBag.ActivityId)';

    const btnProceedToSignatures = document.getElementById('btnProceedToSignatures');
    const btnShowDocumentsDialog = document.getElementById('btnShowDocumentsDialog');
    const btnUnlockApplication = document.getElementById('btnUnlockApplication');
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
    btnProceedToSignatures.disabled = true;

    $('#fireLightMainDiv').on('fireLightLoaded', function ()
    {
        FireLightAPI.init(token, activityId, isNew);

        FireLightAPI.statusEvent.subscribe(statusChange);
        FireLightAPI.signatureStatusEvent.subscribe(signatureStatusChange);
        FireLightAPI.errorEvent.subscribe(handleErrorEvent);
    });

    var statusChange = function (status) {
        console.log('statusEvent - status = ' + JSON.stringify(status));
       console.log('statusEvent = Field: percentComplete = ' + status.percentComplete);

       if (status.percentComplete === 100) {
           btnProceedToSignatures.disabled = false;
           btnUnlockApplication.disabled = false;

       }
       else {
           btnProceedToSignatures.disabled = true;
           btnUnlockApplication.disabled = true;

       }

activityStatus.innerText = status.percentComplete + '% Complete';
    }

    function signatureStatusChange(sig) {
        if (sig.succeeded) {
            window.location.href = sig.redirectUrl;
        }
        else {
            console.log('Signature status message = ' + sig.message);
        }
    }

    function handleErrorEvent(err) {
        var msg = err.error.ApiSource + ' - ' + err.error.ErrorMessage;
        document.getElementById('errMessage').innerHTML = 'Error message = ' + msg;
        console.log('errorEvent = ' + msg);
    }

    btnProceedToSignatures.addEventListener('click', () => {
        FireLightAPI.proceedToSignatures();
    });

    btnShowDocumentsDialog.addEventListener('click', () => {
        FireLightAPI.showDocumentsDialog();
    });

    btnUnlockApplication.addEventListener('click', () => {
        FireLightAPI.showUnlockDialog();
    });
</script>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
<script src="@Url.Content
("https://firelight.insurancetechnologies.com/EGApp/Resource/Scripts/eApp")"></script>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

E-Sign Client API Interface

Step through the following topics to find information about the E-Sign Client API Interface events and methods.

Events

Events

Event	Defined in / Description
eSignStatusEvent	Subscribe to this event to be notified when the status of ESign has changed. This event returns an ESignStatusEvent model.

getAvailableActions

Returns an AvailableActions model that contains information about which actions can be taken during the current ESign step.

Parameters None

Returns *void*

loadEmbeddedESign

Initializes embedded mode for a given activity for E-Signature. This method assumes the specified activity has already been created and is ready for E-Signature. loadEmbeddedESign is only supported when the eSign resource is loaded. See Getting Started and loadModule to see how to load multiple module resources.

NOTE: If resources for multiple modules are loaded, loadModule must be called pior in order to load the correct module.

Parameters

- **activityId:** *string*

  The GUID of the activity to be loaded.

Returns: *void*

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Code Sample

E-Application API Code Sample

```
<p>
    Percent Complete: <div id="activityStatus" class="percentComplete"></div>
</p>
<div style="display:flex;">
    <div>
        <button id="btnProceedToSignatures">Proceed to Signatures</button>
    </div>
    <div>
        <button id="btnShowDocumentsDialog">Show Documents Dialog</button>
    </div>
   <div>
        <button id="btnUnlockApplication">Unlock Application</button>
    </div>
</div>

<br /><hr /><br />

<p id="errMessage"></p>

<div id="fireLightMainDiv">
    Loading...
</div>
<script type="text/javascript">
    const token = '@(ViewBag.access_token)';
    const isNew = '@(ViewBag.activity_isNew)' === 'true';
    const activityId = '@(ViewBag.ActivityId)';

    const btnProceedToSignatures = document.getElementById('btnProceedToSignatures');
    const btnShowDocumentsDialog = document.getElementById('btnShowDocumentsDialog');
    const btnUnlockApplication = document.getElementById('btnUnlockApplication');

    btnProceedToSignatures.disabled = true;

    $('#fireLightMainDiv').on('fireLightLoaded', function ()
    {
        FireLightAPI.loadModule('EApp');
        FireLightAPI.init(token, activityId, isNew);

        FireLightAPI.statusEvent.subscribe(statusChange);
        FireLightAPI.signatureStatusEvent.subscribe(signatureStatusChange);
        FireLightAPI.errorEvent.subscribe(handleErrorEvent);
    });

    var statusChange = function (status) {
        console.log('statusEvent - status = ' + JSON.stringify(status));
       console.log('statusEvent = Field: percentComplete = ' + status.percentComplete);

       if (status.percentComplete === 100) {
           btnProceedToSignatures.disabled = false;
           btnUnlockApplication.disabled = false;
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
        }
        else {
            btnProceedToSignatures.disabled = true;
            btnUnlockApplication.disabled = true;

        }

activityStatus.innerText = status.percentComplete + '% Complete';
    }

    function signatureStatusChange(sig) {
        if (sig.succeeded) {
            FireLightAPI.loadModule('eSign');
            FireLightAPI.loadEmbeddedESign(activityId);
        }
        else {
            console.log('Signature status message = ' + sig.message);
        }
    }

    function handleErrorEvent(err) {
        var msg = err.error.ApiSource + ' - ' + err.error.ErrorMessage;
        document.getElementById('errMessage').innerHTML = 'Error message = ' + msg;
        console.log('errorEvent = ' + msg);
    }

    btnProceedToSignatures.addEventListener('click', () => {
        FireLightAPI.proceedToSignatures();
    });

    btnShowDocumentsDialog.addEventListener('click', () => {
        FireLightAPI.showDocumentsDialog();
    });

    btnUnlockApplication.addEventListener('click', () => {
        FireLightAPI.showUnlockDialog();
    });
</script>
<script src="@Url.Content
("https://firelight.insurancetechnologies.com/EGApp/Resource/Scripts/eApp~eSign")"></script>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Illustration Client API Interface

Step through the following topics to find information about the Illustration Client API Interface linked application events and methods.

Events

Events

Event	Defined in / Description
linkedApplicationEvent	Subscribe to this event to be notified of information regarding the status of the linked application. This event returns a LinkedApplicationEvent model.

createApplication

Creates a new e-application from the current illustration and links them together. Values (input and calculated) and PDF output from the illustration will be uploaded into the e-application.

Parameters: None

Returns *void*

openQuickView

QuickView is a dialog that runs calculations for the illustration and displays them to the user. This method opens the dialog and requests calculations.

Parameters: None

Returns *void*

updateApplication

Updates the linked application with values and PDF output from the current illustration.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Parameters: None

Returns *void*

Code Sample

Code Sample Illustration Client API

```
<div style="display:flex;">
    <div id="btnCreateApp">
        <button>Create App</button>
    </div>

    <div id="btnUpdateApp">
        <button>Update App</button>
    </div>

    <div id="btnOpenQuickView">
        <button>Open Quick View</button>
    </div>
</div>
<br /><hr /><br />
<p id="errMessage"></p>
<div id="fireLightMainDiv">
    Loading...
</div>
<script type="text/javascript">
    const token = '@(ViewBag.access_token)';
    const isNew = '@(ViewBag.activity_isNew)' === 'true';
    const activityId = '@(ViewBag.ActivityId)';
    const btnCreateApp = document.getElementById('btnCreateApp');
    const btnUpdateApp = document.getElementById('btnUpdateApp');
    const btnOpenQuickView = document.getElementById('btnOpenQuickView');

    $('#fireLightMainDiv').on('fireLightLoaded', function ()
    {
        FireLightAPI.init(token, activityId, isNew);
        FireLightAPI.linkedApplicationEvent.subscribe(linkedApplicationChange);
        FireLightAPI.errorEvent.subscribe(handleErrorEvent);
    });

    /** Event Handler for LinkedApplication Change event */
    function linkedApplicationChange(linkedApp) {
        console.log('linkedApplicationChange - linkedApp = ' + JSON.stringify(linkedApp));
        if (linkedApp.LastActionSuccess) {
            console.log('Success! The last action: ' + linkedApp.LastAction + ' occurred on
' + linkedApp.LastActionDate);
        }
        else {
            console.log('Failure! The last action: ' + linkedApp.LastAction + ' occurred on
' +   linkedApp.LastActionDate + '. The error message it generated was: ' +
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
linkedApp.LastActionError);
        }
    }

    function handleErrorEvent(err) {
        var msg = err.error.ApiSource + ' - ' + err.error.ErrorMessage;
        document.getElementById('errMessage').innerHTML = 'Error message = ' + msg;
        console.log('errorEvent = ' + msg);
    }

    btnCreateApp.addEventListener('click', () => {
        FireLightAPI.createApplication();
    });

    btnUpdateApp.addEventListener('click', () => {
        FireLightAPI.updateApplication();
    });

    btnOpenQuickView.addEventListener('click', () => {
        FireLightAPI.openQuickView();
    });

</script>
<script src="@Url.Content
("
https://firelight.insurancetechnologies.com/EGApp/Resource/Scripts/illustration")"></script>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

JSON Models

Following are the JSON models that are used with FireLight API.

Rest API Models

Following are the Rest API Models used with FireLight API.

ActivitiesRequest

Type: object

Required:

- **DataItems**

Properties:

- **DataItems** array of ActivityDataItem

  A list of data items whose values required with activities.

Activity

Type: object

Properties:

- **ActivityId** string

  The unique identifier for the activity

- **TransactionType** integer

  The type of transaction - view Transaction Types list

- **Enum** array

  The Transaction Types name

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **Status** integer

  The Status Types that the activity is in

- **DisplayStatus** integer

  The Display Status Types of the activity

- **ActivityName** string

  The name of the activity

- **AuditDate** string, format *date*
- **Audits** array
- **CreateDate** string, format *date*
- **LastUpdate** string, format *date*
- **DataItems** array of ActivityDataItem

ActivityAttachment

Type: object

Properties:

- **AttachmentName** string

  The name of the attachment

- **AttachmentType** int (OLI_LU_ATTACHMENTTYPE): optional

  The type of the attachment

- **DocumentType** int (ActivityAttachmentDocumentType)
- **DocumentFormat** int (ActivityAttachmentDocumentFormat): default 3

  The file format of the attachment

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **DocumentTag** string
- **AttachmentData64Binary** string

  Attachment data in Base64 binary format

ActivityAttachmentDocumentFormat

Type: enum

Properties:

- **0:** XPS
- **1:** DOC
- **2:** DOCX
- **3:** PDF
- **4:** JPG
- **5:** GIF
- **6:** TIFF
- **7:** PNG
- **8:** BMP
- **9:** NONE

ActivityAttachmentDocumentType

Type: enum

Properties:

- **1:** Application
- **2:** Compliant Illustration
- **3:** Medical Questionnaire
- **4:** Supporting Document

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **5:** Audit Report
- **6:** Additional Doc With Signature
- **7:** Reviewer Uploaded Document
- **8:** Signing Ceremony Only Document
- **9:** Needs Determination
- **10:** Annuity Intelligence Report
- **11:** Linked Illustration
- **12:** Account Opening

ActivityAuditEntity

Type: object

Properties:

- **ActivityId** string, format *uuid*

  The unique identifier for the activity

- **ActivityAuditId** integer

  The unique ID of the activity audit

- **CreateDate**string, format *date*
- **AuditType** integer
- **AuditTypeDescription** string
- **UserName** string
- **Message** string
- **MessageData** string
- **DisplayStatus** integer

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

ActivityDataItem

Type: object

Properties:

- **DataItemId** string
- **Value** string
- **IsEnabled** boolean
- **IsReadonly** boolean
- **IsVisible** boolean
- **CustomActionClicked** boolean

ActivityDetailRequest

Type: object

Required:

- **Id**
- **DataItems**

Properties:

- **Id** guid

  Application Id to get activity details

- **DataItems**array of ActivityDataItem

  A list of data items whose values required with activity details.

ActivityPDFResponse

Type: object

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Properties:

- **ApplicationId** string

  The activityId for the document

- **DocumentId** guid

  The unique id of the document

- **DocumentType** int (ActivityAttachmentDocumentType)

- **FormNumber** string

  The form number for the document, if applicable

- **FormName** string

  The form name for the document, if applicable

- **Parameter:** DutyType

  The Duty Type for the document (Required/Optional)

- **PDFStream** stream

  The stream containing the PDF document

ActivityValidationRequest

Required:

- **CUSIP**
- **Jurisdiction**
- **AcordTypeCode**
- **AcordSubTypeCode**
- **CarrierCode**

Optional

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **ReturnUpdatedDataItems**

Properties:

- **CUSIP** string

  Matches the CUSIP that is configured in the FireLight admin tool

- **Jurisdiction** integer - view Jurisdictions list

  Identifies the jurisdiction type of the activity you are creating

- **AcordTypeCode** integer - view AcordTypeCode & AcordSubTypeCode list

  Identifies the type of activity to create. Note that some activities require AcordSubTypeCode as well as AcordTypeCode.

- **AcordSubTypeCode** integer - view AcordTypeCode & AcordSubTypeCode list

  Identifies the type of activity to create. Used in conjunction with AcordTypeCode to identify some activities.

- **CarrierCode** string

  The three letter carrier code for the product specified by the CUSIP

- **ReturnUpdatedDataItems** boolean

  Determines if the updated DataItems should be returned

- **DataItems** array of ActivityDataItem

  A collection of data items that can be used to populate the wizard that is created during the request. The name of the data item should match the name of the element in the wizard.

AvailableActionsReturn

Type: object

Properties:

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **IsLockAvailable** Boolean
- **IsDeleteAvailable** Boolean
- **IsESignatureAvailable** Boolean
- **IsSubmitAvailable** Boolean
- **IsUnlockAvailable** Boolean

AvailableProductResponse

Type: object

Properties:

- **CUSIP:** string

  String used to identify product.

- **ID:** Guid

  ID Related to Request

- **ProductName:** string

  Name of the Product

- **ProductOrganizationName:** string

  The org name defined at the product level (if defined) or the org name in Organization profile

- **CarrierCode:** string

  The Carrier associated with the product

- **Jurisdictions:** List<long>

  List of Approved states

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **Locale:** String

  The Locale of the product, i.e., en-US

- **Type:** Int32

  The Type of the Product, i.e., Term, Life, etc.

CalculationErrorResult

Type: Object

Properties:

- **Id**: string
- **ResultCode**: int

  A value from the Return Status list to indicate if the call was a success or not.

- **ValidationMessages**: array of WizardErrorMessage

  If there are errors with any of the fields, this will be populated with data

CalculationResponse

Type: Object

Properties:

- **Id** *string*

  The unique ID of the request that was used for this response.

- **ResultCode** *integer*

  A value from the Return Status list to indicate if the call was a success or not.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **ExceptionError** *string*

  The exception message from the server (null if no exception)

- **Messages** ResponseMessage model object array

  A collection of ForeSight UI messages that is part of the response.

- **FireLightCalcOutputs** *array of* OutputSet models

  The output set for FireLight specific outputs.

- **SystemVersion** *string*

  Current version of the system

CompleteSignatureRequest

Type: object

Properties:

- **TeleSignAppId:** String

  The ID of the Tele-Sign application.

- **ANI:** String

  The “calling from” phone number.

- **DNIS:** String

  The “calling to” phone number.

- **IVRId:** String

  The IVR call record ID OR the contact ID.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **FireLightAppId:** Guid

  The ID of the application in FireLight. See Return Status list.

- **CarrierCode:** String

  The Carrier Code of the application in FireLight.

- **CompletionDate:** DateTime

  The date/time in UTC the signature process was completed (in ISO format).

- **Declined:** bool

  Whether or not the client declined the signature.

CompleteSignatureResponse

Type: object

Properties:

- **ResultCode:** Return Status

  Code related to Failure or success.

- **ExceptionError:** *ReturnException*

  Information About the Exception, Including Message, CorrelationId, and TimeStamp

CreateActivityRequest

Type: object

Required:

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **CUSIP**
- **Jurisdiction**
- **AcordTypeCode**
- **AcordSubTypeCode**
- **CarrierCode**

Optional

- **ActivityName**

Deprecated

- **TransactionType (Use AcordTypeCode & AcordSubTypeCode)**

Properties:

- **CUSIP** string

  Matches the CUSIP that is configured in the FireLight admin tool

- **Jurisdiction** integer – view Jurisdictions list

  Identifies the jurisdiction type of the activity you are creating

- **ActivityName** string

  The name given to the Activity. (Optional) If not provided a default name is assigned.

- **AcordTypeCode** integer – view AcordTypeCode & AcordSubTypeCode list

  Identifies the type of activity to create. Note that some activities require AcordSubTypeCode as well as AcordTypeCode.

- **AcordSubTypeCode** integer – view AcordTypeCode & AcordSubTypeCode list

  Identifies the type of activity to create. Used in conjunction with AcordTypeCode to identify some activities.

- **TransactionType** integer - view Transaction Typeslist

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Identifies the transaction type of the activity you are creating.

Note: This is deprecated but still functional for most requests. Use AcordTypeCode & AcordSubTypeCode instead.

- **Forms** array

  List of forms to accompany the application

- **CarrierCode** string

  The three letter carrier code for the product specified by the CUSIP

- **DataItems** array of ActivityDataItem

  A collection of data items that can be used to populate the wizard that is created during the request. The name of the data item should match the name of the element in the wizard.

- **Attachments** array of ActivityAttachment

  A collection of data items that can be used to populate the wizard that is created during the request. The name of the data item should match the name of the element in the wizard.

CreateActivityResponse

Type: object

Properties:

- **ActivityId** guid
- **Id** guid
- **ResultCode** int of type
- **Messages** array of strings
- **ExceptionError**

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

DocumentInfoDocumentType

Type: enum

Properties:

- **0:** Form
- **1:** Document
- **2:** Report
- **3:** Application
- **4:** Addendum
- **5:** Quote

DocumentInfoResponse

Type: object

Properties:

- **ActivityId** string

  The activityId for the document

- **DocumentId** guid

  The unique id of the document

- **DocumentName** string

  The name of the document

- **DocumentType** int (DocumentInfoDocumentType)
- **FormGuid** guid
- **FormIndex** int
- **FormName** string
- **FormNumber** string

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

The form number for the document, if applicable

- **Edition** string

  The edition number

- **Included** bool
- **IsDisabled** bool

  The metadata of the document

- **Order** int

  The relative order of the document within the documents list

- **Parameter** string (Application Form | Optional Form)

  The parameter specifying additional information for the document ("Application Form," "Optional Form," etc.)

DutyType

Type: enum

Properties:

- **0:** Optional
- **1:** Required
- **2:** Conditional
- **3:** Print-Only
- **4:** Submit Only
- **5:** Input-Only: Conditional
- **6:** Input-Only: Required
- **7:** Signing Ceremony Only
- **8:** Singing Ceremony: Conditional

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

DutyType

```
/// <summary>
/// Form Duty/Obligation/Necessity in a package
/// </summary>
public enum DutyType
{
    /// <summary>
    /// Required in the package
    /// </summary>
    Required = 1,

    /// <summary>
    /// Optional in the package
    /// </summary>
    Optional = 0,

    /// <summary>
    /// Conditional in the package
    /// </summary>
    Conditional = 2,

    /// <summary>
    /// Included (required) only when printing, at beginning of document
    /// </summary>
    [Description("Print Only")]
    PrintOnly = 3,

    /// <summary>
    /// Included only when submitting, at end of document
    /// </summary>
    [Description("Submit Only")]
    SubmitOnly = 4,

    /// <summary>
    /// Included only when submitting, at end of document
    /// </summary>
    [Description("Input Only: Conditional")]
    InputOnlyConditional = 5,

    /// <summary>
    /// Included only when submitting, at end of document
    /// </summary>
    [Description("Input Only: Required")]
    InputOnlyRequired = 6,

    /// <summary>
    /// Included only for signing ceremony and print pdf
    /// </summary>
    [Description("Signing Ceremony Only")]
    SigningCeremonyOnly = 7,

    /// <summary>
    /// Included only for signing ceremony and print pdf
    /// Conditional in the package
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
        /// </summary>
        [Description("Signing Ceremony Only: Conditional")]
        SigningCeremonyConditional = 8,
    }
```

GeneralAPIResponse

Type: object

Properties:

- **Id** string in guid format
- **ResultCode** int - A value from the Return Status to indicate if the call was a success or not.
- **Messages** array of strings
- **ExceptionError**

GetActivities

Type: object

Properties:

- **Id** string in guid format
- **ListOfActivities** list of Activity Model
- **ResultCode** int representing Return Status
- **Messages** array of strings
- **ExceptionError** string

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

GetActivityDetails

Type: object

Properties:

- **Dataitems** array of type GetActivityDetailsDataItem
- **Audits** array of type Audit
- **ActivityName** string
- **CreateDate** DateTime
- **LastUpdate** DateTime
- **Status** int

  A value from ActivityStatus (Status Types) that indicates the current status of the activity.
- **Id** string in guid format
- **ResultCode** int
- **Messages** array of strings
- **ExceptionError**

GetActivityDetailsDataItem

Type: object

Properties:

- **DataItemId** string
- **Value** object
- **IsEnabled** boolean

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **IsReadonly** boolean
- **IsVisible** boolean
- **CanBeDeleted** boolean
- **Source** int
- **Unused** boolean
- **Version** string version number

GetActivityPDFRequest

Type: object

Properties:

- **ActivityId:** Guid

  Activity ID to retrieve documents for

- **DocumentIds:** List<guid>

  List of documents to obtain by id. If null or empty, will return all documents: optional

- **ExcludeOptionalDocuments:** boolean

  Determines whether optional documents are excluded. If null or empty, will return all documents: optional

GetActivityPDFResponse

Type: object

Properties:

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **Documents:** List<ActivityPDFResponse>

  List of Activity Documents.

GetDocumentListResponse

Type: object

Properties:

- **Documents:** List<DocumentInfoResponse>

  List of Activity Documents.

GetIllustrationRequest

Type: object

Properties:

- **id** string

  GUID that is a unique identifier for the request

- **DataItems** array of ActivityDataItem

GetReviewQueueStatesRequest

Type: object

Required:

- **AppCode** string in guid format
- **CarrierCode** string

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Properties:

- **AppCode** guid

  Application Id to get review queue application state

- **CarrierCode**

  Carrier code values required with application id

OutputField

Type: object

Properties:

- **Values** array of string

  A zero-based array of field values

- **Name** string

  The name of the output field

- **Caption** string

  The display caption of the output field

OutputModel

Type: object

Properties:

- **ModelType** integer

  The model type for the output model

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **Name** string

  The name of the output model

- **Caption** string

  The display caption for the output model

- **Tables** array

  The array of OutputTable

OutputSet

Type: object

Properties:

- **CalculationRequestId** integer

  A value to indicate which calculation request these outputs are for.

- **Name** string

  The name of the output set

- **Caption** string

  The caption for outputs

- **Models** array of OutputModel models

  The array of output models

- **Messages** array of ResponseMessage models

  The array of calculation messages

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

OutputTable

Type: object

Properties:

- **RowCount** integer

  The number of rows in the table

- **Name** string

  The name of the output table

- **Caption** string

  The display caption for the output table

- **Fields** array

  An array of OutputField models for the table

ProductAvailabilityRequestModel

Type: object

Properties:

- **AcordType:** string (AcordTypeCode & AcordSubTypeCode list)

  Filters the results to product types that match the given AcordTypeCode (optional). The value must be either the AcordTypeCode (i.e., “103”) or the AcordTypeCode and AcordSubTypeCode concatenated with a hyphen (i.e., “111-11101”).

- **CarrierCode:** string

  Limits the product results to only those available for the given Carrier Code (optional)

- **JurisdictionIds:** List<integer>

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Filters the results to products available for the supplied Jurisdictions IDs (optional)

- **RoleCode:** string

  Filters the results to products available to the supplied Role (optional)

ProductAvailabilityResponse

Type: object

Properties:

- **Products:** List<AvailableProductResponse>

  List of Product Information.

- **ID:** Guid

  ID Related to Request

- **ResultCode:** Return Status

  Code related to Failure or success.

- **Messages:** array of strings

  List of Return Messages associated to the response

- **ExceptionError:** ReturnException

  Information about the Exception, including Message, CorrelationId, and TimeStamp

ReportResponse

Type: object

Properties:

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **Reports**: string

  Base64 encoded representation of the report

- **Id** string

  The unique ID of the request that was used for this response.

- **ResultCode** integer

  A value from the Return Status to indicate if the call was a success or not.

- **ExceptionError** string

  The exception message from the server (null if no exception)

- **Messages** ResponseMessage model object array

  A collection of ForeSight UI messages that is part of the response.

- **FireLightCalcOutputs** array of OutputSet models

  The output set for FireLight specific outputs.

- **SystemVersion** string

  Current version of the system

ResponseMessage

Type: object

Properties:

- **Severity** integer

  The severity of the message

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **MessageText** string

  The text of the message

ReviewQueueGetStatusResponseModel

Type: object

Properties:

- List of ReviewQueueState
- **Id** string in guid format
- **ResultCode** int of type
- **Messages** array of strings
- **ExceptionError**

ReviewQueueState

Type: object

Properties:

- **QueueName** String
- **QueueNumber** String
- **State** QueueState

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

ReviewQueueUpdateStatusResponseModel

- **UpdatedCount**
- **Id** string in guid format
- **ResultCode** int
- **Messages** array of strings
- **ExceptionError**

TokenResponse

Type: object

Properties:

- **access_token** *string*

  This is the string text value of the token. It will need to be passed along when used, as defined in 'Using the Token' below.

- **expires_in** *integer*

  This is the amount of time, in seconds, that the token will remain active. After this amount of time, the token is considered expired and no longer valid to be used.

- **token_type** *string*

  This is the type of token. It needs to be passed along when the token is used, as defined in 'Using the Token' below. It is expected to return as "Bearer".

UpdateReviewQueueStateRequest

Required

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **CarrierCode** string
- **AppCode** string in guid format
- **QueueNumber** string
- **NewState** QueueState

Optional

- **Remarks** string
- **SourceSystem** string

UploadDocumentRequest

Type: object

Properties:

- **Id** string in guid format

  Activity Guid for which document(s) to be uploaded.

- **Attachments** array of ActivityAttachment

  A collection of attachments those are needed to upload.

ValidateActivityDataResult

Type: Object

Properties:

- **Id**: string
  ID of the request

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **ResultCode**: int
  Code to identify the result of the request
- **ExceptionError**:
- **Messages**: array of strings
- **ValidationMessages**: array of ValidationMessage

  If there are errors with any of the fields, this will be populated with data

ValidateSignatureRequest

Type: object

Properties:

- **TeleSignAppId:** String

  The ID of the Tele-Sign application.
- **TeleSignPIN:** String

  The PIN used to verify caller identity.
- **DateOfBirth:** DateTime

  The date of birth of the signer, view Return Status list
- **CarrierCode:** String

  The carrier code for the application.

ValidateSignatureResponse

Type: object

Properties:

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **FireLightAppId:** Guid

  The ID of the application in FireLight.

- **Valid:** bool

  Whether or not the submitted information matches the expected values.

- **State:** string

  The US state the application was created in.

- **ResultCode:** Return Status

  Code related to Failure or success.

- **Messages:** array of strings

  List of Return Messages associated to the response

- **ExceptionError:** ReturnException

  Information About the Exception, Including Message, CorrelationId, and TimeStamp

- **ApplicantFirstName:** string

  The first name of the applicant/signer which includes the middle name or initial if entered

- **ApplicantLastName:** string

  The last name of the applicant/signer.

ValidationMessage

Type: object

Properties:

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **DataItemId**: string
- **SenderId**: string
- **Operation**: int
- **Message**: string
- **Flags**: int
- **OfficialUseOnly**: int

WizardErrorMessage

Type: Object

Properties:

- **DataItemId**: string

  The DataItemID associated with this validation message

- **SenderId**: int

  The ID of the rule that sent this message

- **Operation**: int

  ID of operation, if any

- **Message**: string

  Message the field generated

- **Flags**: int

  Flags associated with message

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Embedded Client API Event Models

Following are the JSON models that are used with the FireLight Embedded Client API events.

ErrorEvent

Type: object

Properties:

- **error**: Error Model

  The description of the error returned from the API

- **type**: string

  The API method call that generated this event

ESignStatusEvent

Type: object

Properties:

- **page string**

  The current page/route of ESign. Will only be populated if the current status is RouteChange.

- **message string**

  Any error message return by the status event

- **status int** (E-Sign Status Type)

  The current status of ESign. One of E-Sign Status Types.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

LinkedApplicationEvent

Type: object

Properties:

- **LastAction:** enum

  The type of the last action that was performed – view list

- **LastActionDate:** date

  Date of last action that was performed

- **LastActionSuccess**: Boolean

  Flag to indicate the success of the last action

- **LastActionError**: string

  Error message, if any, the last action generated

- **ApplicationId**: string

  Guid in string format of the linked application's ID

NavigationEvent

Type: object

Properties:

- **activityID**: Guid

  The unique identifier for the activity

- **module** string

  The name of the page that was navigated to

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **pageNumber** int

  The page number that was navigated to, 0 based

- **type** string

  The API method call that generated this event

- **navigationGraph** Array of ActivityBundle model

NotificationsEvent

Type: object

Properties:

- **timestamp** int

  The timestamp (browser time) indicating when the notification was raised.

- **severity** int

  The severity of the notification. Corresponds to the Notification Severity Types enumeration.

- **title** string

  A summary of the purpose of the message.

- **message** string

  The full message.

- **messageContainsMarkup** boolean

  Indicates whether the message text contains markup that needs to be parsed.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

RuleMessagesEvent

Type: object

Properties:

- **DataItemId**: string

  The item id of the field that generated the message

- **Errors**: Array of MessageError model

  An object to hold the data for the errors associated with this particular message

- **IsTemplate**: Boolean

  True if the application is a template.

- **LocationText**: string

  The name of the application that generated this message

- **PageNumber**: int

  The page number the item that generated the message can be found on within the application

SignatureStatusEvent

Type: object

Properties:

- **succeeded:** boolean

  True if the application is complete and can proceed to gathering signatures

- **message**: string

  Information regarding the attempt to proceed to gather signatures

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **redirectUrl**: string

  URL of the address to gather signatures.

  NOTE: This URL is only usable when enableAuthProxy is set to false when calling Common API init function. When enableAuthProxy is true use the Token SSO sign on method to redirect to FireLight E-Signature.

The API method call that generated this event.

StatusEvent

Type: object

Properties:

- **isEditable** Boolean

  Flag to indicate whether the event can be modified

- **lastDataItemUpdated** string

  Value to indicate the last item that was changed with the application

- **percentComplete** int

  An integer value to indicate how much of required application's field have been completed

- **type** string

  The API method call that generated this event

Embedded Client API Models

Following are the Client API Models that are used with FireLight API.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

ActivityBundle

Type: object

Properties:

- **FormGuid**: string

  The unique ID of the underlying form or wizard.

- **FormIndex**: int

  The index of the underlying form/wizard within the package.

- **BundleName**: string

  The name of the bundle. Corresponds to the name of the form/wizard.

- **Pages**: Array of BundlePage model

  The pages within the form/wizard

- **IsSelected**: boolean

  Indicates if the bundle is currently selected. This is true if one of bundle's pages is selected.

- **IsEnabled**: boolean

  Indicates if the bundle is available to be selected. This is true unless the form/wizard is marked as excluded or hidden from data entry.

- **IsVisible**: boolean

  Indicates if the bundle should be displayed to the user. This is true unless the form/wizard is marked as excluded or hidden from data entry.

- **HasErrors**: boolean

  Indicates if any of the pages within the bundle have error messages associated with them.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

AvailableActions

Type: object

Properties:

- **IsUnlockAvailable:** boolean

  Returns whether the application can currently be unlocked. When this is false, a call to showUnlockDialog will return an error message instead of showing the dialog.

BundlePage

Type: object

Properties:

- **PageNumber**: int

  The index (0-based) of the page within the entire set of available pages.

- **PageName**: string

  The page name presented to the user.

- **IsSelected**: boolean

  Indicates if the page is currently selected.

- **IsVisible**: boolean

  Indicates if the bundle should be displayed to the user.

- **IsEnabled**: boolean

  Indicates if the page is available to be selected. If IsEnabled = false and IsVisible = true the page should be visible but not selectable.

- **SectionName**: string

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

The name of the Workflow Section. If empty, the BundlePage is not part of a Workflow Section.

- **HasErrors**: boolean

  Indicates if the page has error messages.

- **IsForm**: boolean

  Indicates if the page is a form. If false, the page is a wizard.

- **DocumentGuid**: Guid

  The unique ID of the uploaded document (used only if page is an uploaded form)

Error

Type: object

Properties:

- **ApiSource**: string

  The API method that threw this error

- **CorrelationID**: string

  The ID of the item this error is correlated with

- **Environment**: string

  The name of the environment (Token Service Endpoint) that this error was thrown in

- **ErrorMessage**: string

  The descriptive text of the error message that was thrown

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **ErrorSource**: string

  The API method name where this error was generated from

- **ErrorType**: int

  The enum value of the error message that was generated

Message

Type: object

Properties:

- **Count**: int

  The number of messages (objects) contained within the array

- **DataItem**: object

  The information for the data field that generated the message

- **DataItemId**: string

  The item id of the field that generated the message

- **ErrorCount**: int

  The number of errors associated with this message

- **Errors**: Array of MessageError model

  An object to hold the data for the errors associated with this particular message

- **IsTemplate**: Boolean

  True if the application is being used as a template.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **LocationText**: string

  The name of the application that generated this message

- **PageNumber**: int

  The page number the item that generated the message can be found on within the application

MessageError

Type: object

Properties:

- **DataItemid**: string

  The item id of the field that generated the error message

- **Flags**: int

  The enum value representing all of the ???

- **Message**: string

  The descriptive text of the error message that was thrown

- **Operation**: int

  The API operation that threw this error

- **SenderId**: int

  The ID associated with the sender???

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Lists

Following are lists that are used with FireLight API.

AcordTypeCode & AcordSubTypeCode

AcordTypeCode & AcordSubTypeCode

ACORDTypeCode	AcordSubTypeCode	Activity
103		Application
106		Account Opening
111		Illustration
111	11101	Guided Illustration
113		Generic Post Issue
116	11600	Needs Determination
116	11602	Pre-Suitability
121	12100	E-Delivery
1241		Fee Inquiry
129		Licensing
161		Agent Assignment
212		Quote
212	21211	Quote Shell
310		Print Forms
410		Appointments
508		Sub Pay

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Display Status Types

Display Status Types

Name	Value	Description
Unknown	0	Unknown
DataEntry	1000	Data Entry
ClientRequestPending	1100	Pending Client Request
ClientRequestComplete	1200	Client Request Complete
CheckedOut	1300	Checked Out
AppGenerated	1400	Application Generated
The following e-Delivery statuses occur before the app is locked:		
eDeliveryPending	1500	Pending
eDeliveryAgentReviewPending	1510	Pending Agent Review
eDeliveryPendingEDelivery	1520	Pending E-Delivery
eDeliveryClientReviewPending	1540	Pending Client Review
eDeliveryClientReview	1550	In Client Review
eDeliveryClientReviewApproved	1560	Client

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

		Approved
eDeliveryAgentRequestChange	1570	Change Requested by Agent
eDeliveryClientRequestChange	1580	Change Requested by Client
eDeliveryClientRequestChangeClientDirected	1581	Change Requested by Client
The following e-Delivery statuses are used for other apps when one of the children requested change or declined signature—need separate request status in 1122, but the display status text is the same as the agent's case.		
eDeliveryPendingClientRequestChange	1590	Pending Agent Review
eDeliveryPendingClientRequestChangeClientDirected	1591	Pending Agent Review
eDeliveryPendingAgentReviewDeclineSign	1592	Pending Agent Review
The following needs a separate display status because display is change request but 1122 status is in error:		
eDeliveryDeclineSignClientDirected	1593	Change Requested by Client
Used for bounced email process agent directed—case with email issue:		
AgentReviewUndeliverable	1594	Pending Agent Review
Used for bounced email process agent directed—not case with email issue:		
AgentReviewUndeliverableOther	1595	Pending Agent

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

		Review
Used for long running async processes that must be complete before an app can move on to signatures:		
PendingResponse	1600	Pending Response
ResponseReceived	1610	Response Received
Locked	2000	Locked
Used for locking related illustration:		
AppLocked	2200	Application Locked
Signature	3000	Signatures
SignaturePending	3100	Pending Signatures
SignatureInProgress	3200	In Signatures
ClientSignaturesComplete	3300	Client Signatures Complete
The agent signing process is started. Created for e-Delivery client apps:		
SignatureAgentPending	3400	Pending Agent Signatures
The Agent signature is in process. Created for e-Delivery client apps:		
SignatureAgentInProcess	3500	In Agent

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

		Signatures
SignatureRejected	3800	In Signatures (keep in signatures for audit history purpose)
eDeliveryClientDirectedSignatureRejected	3801	In Signatures (keep in signatures for audit history purpose)
SignatureFinished	3900	Signatures Complete
eDeliveryComplete	3910	E-Delivery Complete (deliver only complete—client status)
eDeliveryDownloadComplete	3915	Download Complete
eDeliveryPrintRequested	3920	Request Print
eDeliveryPrintRequestedByAgent	3921	Request Print
ManualReview	4000	Pre-Submit Review
ManualReviewPending	4100	Pending Pre-Submit Review
ManualReviewInProgress	4200	In Pre-Submit Review

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

ManualReviewRejected	4800	Pre-Submit Review: Declined
ManualReviewFinished	4900	Pre-Submit Review Complete
ManualReviewApproved	4910	Pre-Submit Review: Approved
ManualReviewMoreInfo	4920	Pre-Submit Review: Info Requested
Terminated	5000	Terminated
Canceled	5100	Policy Canceled
Used for for bounced email process:		
CanceledUndeliverable	5110	Policy Canceled
Used for bounced email process—not the case that bounced:		
CanceledByOtherUndeliverable	5120	Policy Canceled
Expired	5200	Policy Expired
CanceledByCarrier	5300	Policy Canceled
AdminTransfer	5400	Admin Transfer

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

ReviewQueue	6000	Review Queue
ReviewQueuePending	6100	Pending Review
ReviewQueueInProgress	6200	In Review
PendingAgentReview	6300	Pending Agent Review
ReviewQueueRejected	6800	Review: Declined
ReviewQueueFinished	6900	Review Complete
ReviewQueueApproved	6910	Review: Approved
ReviewQueueMoreInfo	6920	Review: Info Requested

Submitted	7000	Submit Requested
PendingTransmission	7100	Pending Transmission
Complete	8000	Complete
Post Submission statuses:		
InCarrierReview	8100	In Carrier Review
CarrierReviewComplete	8200	Carrier Review Complete

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Purged	9000	Purged

E-Sign Status Types

E-Sign Status Types

Name	ID	Description
RouteChange	0	The current Page/Route has changed.
ESignInitialized	1	E-Sign has been Initialized.
ESignAccepted	2	The E-Signature process has been accepted by the user.
ESignDeclined	3	The E-Signature process has been declined by the user.
ESignNowStarted	4	Sign Now has been initiated by a user.
ESignNowComplete	5	A user has finished the Sign Now process.
ESignSignatureDeclined	6	A user has declined to sign
ActivityUnlocked	7	The activity has been unlocked (signatures no longer possible)
ActivitySubmitted	8	The activity has been submitted. Signatures are complete or not needed.
ExitRequest	9	Indicates an internal request to exit the signatures page.
ESignComplete	10	All signatures have been completed.
ESignSharedSignatureCaptured	11	Agent's shared signature has been captured.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Jurisdictions

Jurisdictions

ID	Short Name
1	AL
2	AK
4	AZ
5	AR
6	CA
7	CO
8	CT
9	DE
10	DC
12	FL
13	GA
14	GU
15	HI
16	ID
17	IL
18	IN

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

ID	Short Name
19	IA
20	KS
21	KY
22	LA
23	ME
25	MD
26	MA
27	MI
28	MN
29	MS
30	MO
31	MT
32	NE
33	NV
34	NH
35	NJ
36	NM
37	NY
38	NC

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

ID	Short Name
39	ND
41	OH
42	OK
43	OR
45	PA
46	PR
47	RI
48	SC
49	SD
50	TN
51	TX
52	UT
53	VT
54	VI
55	VA
56	WA
57	WV
58	WI
59	WY

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Last Action Types

Last Action Types

Name	ID	Description
None	1	There has not been an action performed.
Apply	2	Updates were applied to the application.
Update	3	The application or one of its statuses were updated.

Notification Severity Types

Notification Severity Types

Name	ID	Description
Informational	0	The notification provides additional information to the user.
Warning	1	The notification serves as a warning to the user.
Error	3	The notification provides the user with an error message.

QueueState

Type: enum

Properties:

- **0:** Unlocked
- **1:** Locked
- **2:** Approved

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- **3:** Rejected
- **4:** Removed

Return Status

Return Status

Return Status	ID	Description
RESULT_SUCCESS	1	The transaction was a success.
RESULT_SUCCESSINFO	2	The transaction was a success but some condition caused information to be returned. This information is usually informational or a warning.
RESULT_RECDPEND	3	Not used.
RESULT_RECDPENDINFO	4	Not used.
RESULT_FAILURE	5	The transaction was not successful and the reason for the failure was returned.

Status Types

Status Types

Name	ID	Description
ActiveApplication	0	The application is not 100% complete. Data for the application is still being entered or updated.
Locked	1	The application has been locked and no further edits can be made.
PendingSignatures	2	The application is complete and the user has selected the e-Signature option.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Name	ID	Description
PendingApproval	3	The application has been completed and is awaiting approval.
Approved	4	The application has been reviewed and is approved.
Submitted	5	The application is pending submission.
Complete	6	The application process has been completed. This is the final status prior to the application being purged.
PendingReview	7	The application has been completed and is awaiting review. This status is shown for all OSJ post-submission review queues.
UnderReview	8	The application is currently being reviewed.

Transaction Types

Transaction Types

Name	ID	Description
Application	1	Used to start an application
SubPay	2	Subsequent Payment
NeedsDetermination	3	Used when the item needs determination
PostIssue	4	Used to initiate a post issue service
FeeInquiry	5	Used to perform a fee inquiry
Illustration	6	Used to start a new illustration

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Name	ID	Description
GuidedIllustration	7	Used to start a guided illustration
AccountOpening	8	Used to open an account
eDelivery	9	Used to start the E-Delivery process

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Salesforce Code Samples

Following are Salesforce Code Samples for use with FireLight API.

CreateActivity Example

CreateActivity

```
private static String CreateActivity(String token)
    {
        String createActivityUrl =
'https://firelight.insurancetechnologies.com/EGApp/api/Activity/CreateActivity';

       // These values need to be changed to match your environment
       String body = '{ \'CUSIP\': \'FSETerm\', \'Jurisdiction\': 1, \'TransactionType\': 1,
\'CarrierCode\': \'IT\' }';
        HttpRequest req = new HttpRequest();
        req.setEndpoint(createActivityUrl);
        req.setMethod('POST');
        req.setHeader('Authorization', 'Bearer ' + token);
        req.setHeader('Content-Type', 'application/json');
        req.setBody(body);
        Http http = new Http();
        HTTPResponse res = http.send(req);
        String result = res.getBody();
        //System.debug('CreateActivity - result =' + result);
        if (result.contains('ActivityId')) {
             result = result.substring(15).substring(0, result.substring(15).indexOf('"'));
             String activityId = result;
        }
        return result;
    }
```

GetToken Example

GetToken

```
global static String GetToken(String data)
    {
        string org = ORG_CODE;  // Three letter FireLight Character Code
        string tokenUrl =
'https://firelight.insurancetechnologies.com/EGApp/api/Security/GetToken';
        DateTime currDate = DateTime.now();
        String secret = TOKEN_SECRET;  //Web Token Secret from FireLight admin tool
        String dateStr = currDate.formatGMT('MMddyyyyHHmmss');
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
        string xml1228 = EncodingUtil.base64Encode(Blob.valueof(BuildTX1228('')));
        String textToSign = secret + dateStr + org + xml1228;
        String code = EncodingUtil.convertToHex(Crypto.generateDigest('SHA-256',
Blob.valueOf(secret))).toUpperCase();

       // Certificate should be stored under the key 'mycert'
      // The public key of this cert needs to be added to the FireLight admin tool
        String signedData = EncodingUtil.urlEncode(EncodingUtil.base64Encode
(System.Crypto.signWithCertificate('RSA-SHA256', Blob.valueOf(textToSign), 'mycert')), 'UTF-
8');
        xml1228 = EncodingUtil.urlEncode(xml1228, 'UTF-8');
        string body = 'grant_
type=hashsig&id='+org+'&secret='+code+'&sig='+signedData+'&xml='+xml1228;
         HttpRequest req = new HttpRequest();
         req.setEndpoint(tokenUrl);
         req.setMethod('POST');
         req.setHeader('Date', DateTime.now().formatGMT('EEE, dd MMM yyyy HH:mm:ss z'));
         req.setHeader('Content-Type', 'text/plain');
         req.setBody(body);
         Http http = new Http();
         HTTPResponse res = http.send(req);

         string result = res.getBody();
         if (result.contains('access_token'))
         {
              String token = result.substring(17).substring(0, result.substring(17).indexOf
('"'));
              result = token;
         }
        return result;
    }
```

VisualForce Page Code

VisualForce Page Code

```
<apex:page standardController="Contact" extensions="UserDescriberContact_
FLEmbedded,FirelightCntrlEmbedded"
 standardStylesheets="false" cache="false" deferLastCommandUntilReady="false">
    <apex:stylesheet value="{!$Resource.bootstrapCSS}" />
    <apex:stylesheet
value="https://firelight.insurancetechnologies.com/EGApp/Resource/CSS/illustration"/>
    <apex:includeScript value="{!$Resource.jquery}" />
    <apex:includeScript value="{!$Resource.jqueryUI}" />
    <apex:includeScript value="{!$Resource.BootstrapJS}" />

    <style>
        ...custom page styling...
    </style>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
    <div id="divLoader" class="col-xs-12 loader" style="display:none;margin-left:45%"></div>

    <div class="col-xs-12" id="flOuterDiv" style="display:none;">
        <apex:outputPanel id="flIllust">
            <apex:pageBlock id="flIllustPb">
                <div class="panel panel-primary">
                    <div class="panel-body">
                        <button id="btnRenderQV" class="btn btn-primary">Open Quick
View</button>
                        <button id="btnRenderPDF" class="btn btn-primary">Render
PDF</button>
                        <p id="errMessage"></p>
                        <div class="contents" id="fireLightMainDiv">
                            Loading...
                        </div>
                    </div>
                </div>
            </apex:pageBlock>
        </apex:outputPanel>
    </div>

    <script type="text/javascript">
        jQuery.noConflict();
        var appToken = '';
        var appId = '';

        window.FireLightAPI={};
        window.FireLightAPI.renderDestination='fireLightMainDiv';

        jQuery(document).ready(function(){
            submitData('{!SerializedMappedUsers}')
        });

        const renderQuickView = document.getElementById('btnRenderQV');
        const renderPdfDiv = document.getElementById('btnRenderPDF');
        const mainDiv = document.getElementById('fireLightMainDiv');

        renderQuickView.addEventListener('click', () => {
            console.log('renderQuickView.addEventListener');
            FireLightAPI.openQuickView();
        });

        renderPdfDiv.addEventListener('click', (api) => {
            console.log('renderPdfDiv.addEventListener');
            FireLightAPI.renderPDF();
        });

        mainDiv.addEventListener('fireLightLoaded', (api) => {
            console.log('fireLightLoaded');

            jQuery('#divLoader').hide();
            jQuery('#flOuterDiv').show();

            var valid = (appId == '');
            if ((appToken != '') && (appId != ''))
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
            {
                //console.log('before init call');
                FireLightAPI.init(appToken, appId, valid);

                //console.log('after init call');
                FireLightAPI.errorEvent.subscribe(handleErrorEvent);

                //console.log('after subscribecall');
                FireLightAPI.statusEvent.subscribe((status) => {
                    //console.log('statusEvent fired');
                    if (status.percentComplete < 100) {
                        console.log('% complete = ' + status.percentComplete);
                    }
                    else {
                        console.log('100 % complete');
                    }
                });
            }

        });

        function handleErrorEvent(err) {
            //console.log('handleErrorEvent');
            var msg = err.error.ApiSource + ' - ' + err.error.ErrorMessage;
            //console.log('errorEvent = ' + msg);
            //document.getElementById('errMessage').innerHTML = 'Error message = ' + msg;
            //alert('Error message = ' + msg);
        };

        String.prototype.lpad = function(padString, length) {
            var str = this;
            while (str.length < length)
                str = padString + str;
            return str;
        };

        function submitData() {
            //parse the item string passed in
            var data = '{!SerializedMappedUsers}';
            var users = JSON.parse(data.replace(/[\n\r]/g, ''));

            //alert('submitData - data = ' + data);

            //show the correct items on the screen
            jQuery('#btnSubmitOuterDiv').hide();
            jQuery('#divLoader').show();

            //build json string of contacts - double check line coming in, maybe able to
simplify
            var contactInfo = buildContactJSONString(users);
            //console.log('contactInfo = ' + contactInfo);

            //calls to API
            GetToken(contactInfo)

            return true;
        };//end of submitData
```

```
        function buildContactJSONString(userList) {
            var result = [];

            //build user object array

            return result;
        };//end of buildContactJSONString

        function GetToken(data) {
            Visualforce.remoting.Manager.invokeAction(
                '{!$RemoteAction.FirelightCntrlEmbedded.GetToken}',
                data,
                true,
                function(result, event) {
                    if (event.status) {
                        //console.log('res = ' + result);
                        //console.log('received token');

                        if (result.indexOf('remote') > 0){
                            //submitData();
                            //error happened
                            alert('error - ' + result);

                            //show the correct items on the screen
                            jQuery('#btnSubmitOuterDiv').show();
                            jQuery('#divLoader').hide();
                        }
                        else{
                            appId = result.substr(0, result.indexOf(';'));
                            appToken = result.substr(result.indexOf(';')+1);

                            console.log('appId = ' + appId);
                            console.log('appToken = ' + appToken);

                            var scriptUrl =
"https://firelight.insurancetechnologies.com/EGApp/Resource/Scripts/eApp";

                            jQuery.ajax({
                                url: scriptUrl,
                                dataType: "script",
                                cache: false,
                                success: function(jsonResponse, textStatus, jqXHR)
                                {
                                    alert('script loaded');

                                    //show the correct items on the screen
                                    jQuery('#btnSubmitOuterDiv').show();
                                    jQuery('#divLoader').hide();
                                },
                                error: function(jqXHR, textStatus, errorThrown)
                                {
                                    alert('script failed to load - ' + exception + ' - ' +
settings + ' - ' + JSON.stringify(jqxhr));

                                    //show the correct items on the screen
                                    jQuery('#btnSubmitOuterDiv').show();
```

```
                                    jQuery('#divLoader').hide();
                                }
                            });
                        }
                    }
                },
                { 'escape':false }
            );
        };

    </script>
</apex:page>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Embedded Workflows

Following are some embedded workflows using FireLight APIs.

Declining e-Signature and Capturing Wet-Signed Documents

In the regular FireLight user interface, the user is given an option to decline electronic signature. When a user declines electronic signature, the system provides instructions on how to upload wet signed documents. This same workflow does not exist within the embedded, but the following steps can be followed to recreate the experience in your own application.

1. Start by configuring the group option to Disable E-Sign Decline. This will instruct the signing ceremony to not ask the user if they consent to electronic signature. This assumes that the question will be asked outside of FireLight.
2. Create a page in your application that asks the user if they would like to do electronic signature. This is a consent form that allows the user to accept or decline the use of electronic signature. Based on the answer, set the FLI_DECLINED_ESIGN value to true if they decline the use of electronic signature or false if they accept the use of electronic signature.
3. If FLI_DECLINED_ESIGN is set to false, and the group option has been enabled, redirect the user into the FireLight signing ceremony. The user will not be asked again for consent to electronic signature. Refer to proceedToSignatures to learn how to redirect the user to the signing ceremony.
4. If FLI_DECLINED_ESIGN is set to true, implement the UI functions to allow users to upload documents. Use the following APIs to implement this function.

    - GetActivityPDF to get the PDF to sign.
    - UploadDocument to upload documents and associate with the activity.
    - showDocumentsDialog to show our documents dialog that would allow the user to upload the wet signed documents. This will require the embedded session and client libraries to be loaded.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

E-Sign Workflows

Steps for Using FireLight E-Sign within an Embedded Application

1. Enable the proxy server from within your embedded application.

    a. FireLightAPI.init(token, activityId, isNew, enableErrorHandling, useAuthProxy)

2. (User fills out the embedded application.)

3. Implement an application completion process. For example, check that the application is complete by using FireLight.statusEvent.subscribe(), and checking status.percentComplete in the callback.

4. Call API.proceedToSignatures(). This verifies the application is ready for signatures and raises a signatureStatusEvent. Check status.succeeded to continue.

5. Navigate to FireLight E-Sign using SSO. See the SSO information on how this may be done. Using the Token SSO approach is recommended because a token has already been retrieved. This approach would require setting the following query string parameters: appId & relayState=eSign.

Steps for Using Embedded E-Sign within an Embedded Application

1. Load the Embedded E-App application:

    a. Call FireLightAPI.loadModule('EApp')

    b. Call FireLightAPI.init(token, activityId, isNew, enableErrorHandling, useAuthProxy)

2. (User fills out the embedded application.)

3. Implement an application completion process. For example, check that the application is complete by calling the FireLight API's IsComplete method.

4. Call API.proceedToSignatures(). This verifies the application is ready for signatures and raises a signatureStatusEvent. Check status.succeeded to continue.

5. Load the Embedded E-Sign application:

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

   a. Call FireLightAPI.loadModule('eSign')

   b. Call FireLightAPI.loadEmbeddedESign(activityId)

6. When signatures are complete the user should be navigated away from eSign. For example navigate back to the eApp module.

   a. Call FireLightAPI.loadModule(‘EApp’)

   b. Call FireLightAPI.loadEmbeddedEApp(activityId)

Features currently missing from Embedded E-Sign:

- Canceling of an Active Signer
  - Re-Signing is still available. The signer must complete the signature process and then use Re-Sign.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Common Issues Using the FireLight Embedded System

Issue: The system does not load the embedded application. I get a 403 error when trying to access eApp.

Resolution: Cross-Site Scripting may be preventing you from accessing the FireLight servers from within another website. For security reasons, most browsers require you to specify which external websites may be accessed from within your website. Ensure the following:

- Add your client server name (eg, embedded.mywebsite.com) to the list of CORS Origins within FireLight Admin (Organization Page -> Security Tab)
- Add a refence to whichever FireLight server you are attempting to access (such as uat.firelighteapp.com) to the CSP header request used to call the Embedded FireLight API from within your website. See https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP for more information.

Issue: The system does not load the embedded application. I get a 500 error when trying to access CSS.

Resolution: Cross-Site Scripting may be preventing you from accessing the FireLight servers from within another website. Enable the FireLight API proxy:

- Use 'true' for the fifth parameter of Init
  - `FireLightAPI.init(token, activityId, isNew, enableErrorHandling, useAuthProxy)`
- Use 'true' for the second parameter of startSession
  - `FireLightAPI.startSession(token, useAuthProxy, callback)`

Issue: I do not see a "Continue" button within my embedded app. How do I proceed to signatures?

Resolution: You need to add your own process for continuing to signatures. Usually this is done by adding a button to your web page that contains the embedded application, such as

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

at the top of the page. Note that users are not allowed to navigate to signatures until all required fields have been completed on the application. You are in charge of handling this, such as by enabling your “Proceed to Signatures” button when the application is complete, by displaying a popup message if a user clicks the button while the application is incomplete, etc. Both functions will return a status object containing information about the current state of the application. When status.percentComplete equals 100, the application is complete and you should allow proceeding to signatures – for example, by enabling your own button that calls API.proceedToSignatures() when clicked. See section [todo] for more information on using E-Sign within your embedded app.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Sign On Methods

Introduction

FireLight supports two sign on methods: a POST of an ACORD Tx1228 to the passive FireLight endpoint (external integration) and Single Sign On (SSO).

FireLight provides support for an ACORD Tx1228 during the sign on process. This ACORD request can be used to provide login credentials and agent information as well as specify firm information and product availability.

FireLight External Integration

Summary

FireLight supports five possible integration scenarios from an external system:

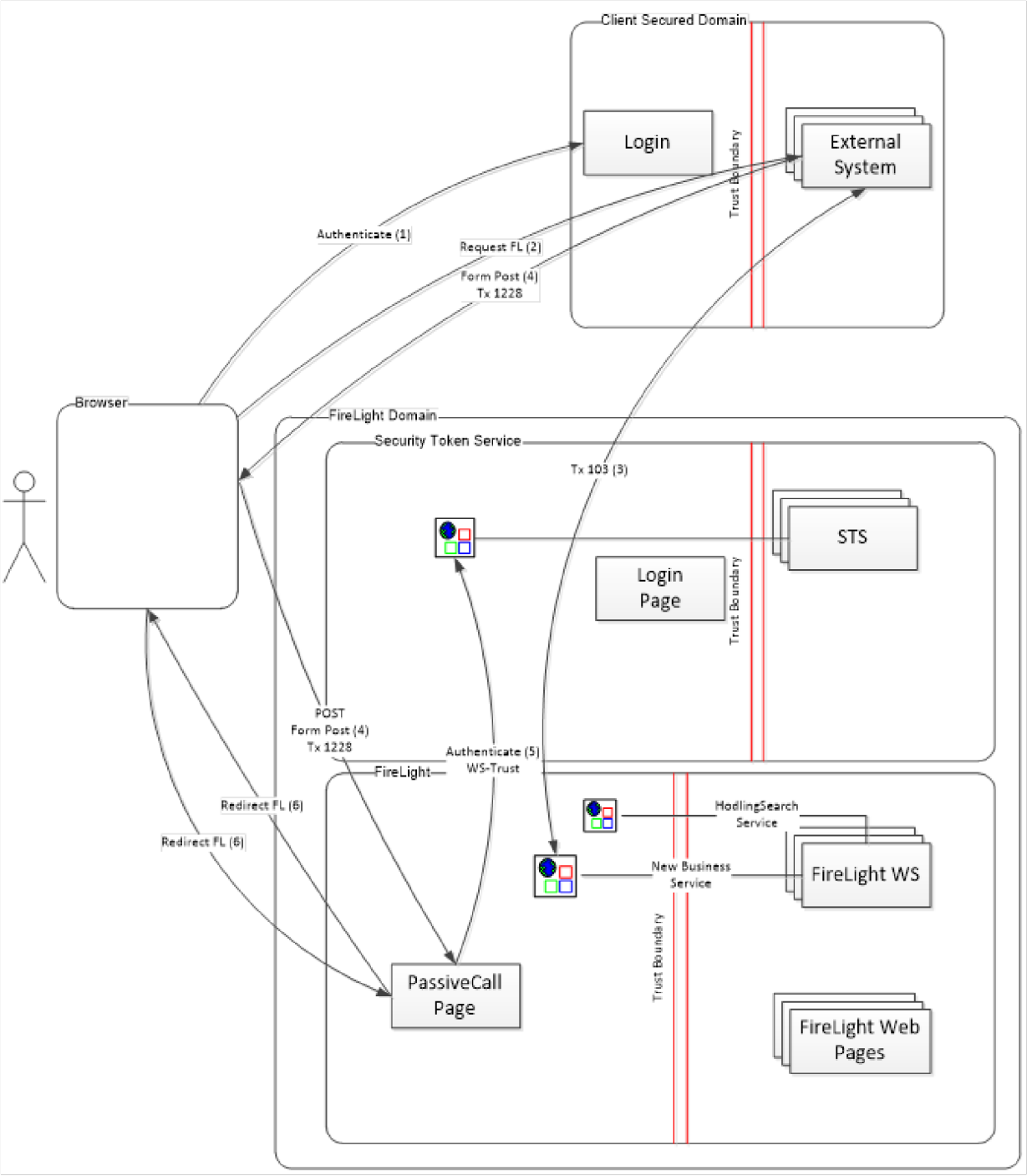
1. The creation of a new application from data submitted from the external system.
2. The ability to update an existing application with new data from the external system.
3. The ability to launch directly into FireLight to edit an existing application.
4. The ability to launch directly into the FireLight dashboard.
5. The ability to launch directly into the FireLight dashboard with retained use of 103 data.

The topics in this section describe in detail the steps an external system must take for each scenario.

Create New Application

To create a new application within FireLight from data captured from an external system requires a two-step process. First, the external system will call the FireLight New Business web service, passing in an ACORD Tx103 transaction. Second, the external system will send over a specially crafted response to the browser that will post over to FireLight an ACORD Tx1228 to authenticate and authorize the user for access to FireLight. If the authentication is successful, FireLight will redirect the user to the Edit Application view so the user can immediately start to work on the newly created application. The following diagram illustrates the steps in this process.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.



The scenario assumes the user is authenticated and authorized prior to interfacing with the FireLight system. Once the user is authenticated, the external system will provide a link the user will click on to request the creation of a new application in FireLight, invoking the following sequence:

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

1. The user is authenticated and authorized by the external system.

2. The user clicks on a link to launch into FireLight, creating a new application.

3. The external system will construct an ACORD Tx103 transaction to send over to FireLight. The response form FireLight is checked by the external system to ensure the transaction was received successfully.

4. The external system sends back to the user a specially crafted response to force the browser to POST an ACORD Tx1228 transaction within a hidden form field to the FireLight passive endpoint. Included within this request is a “refid” query string with a value of the TransRefGUID sent over from step 3.

5. FireLight authenticates with the STS using a predefined shared user login and password set up specifically for the implementation. If the credentials are valid, FireLight will impersonate the user identified within the 1228 for the remainder of the session. If the credentials are not valid, the user will be redirected to a generic error page within FireLight.

6. FireLight will send back a redirect to the Edit Application view with a query string containing the TransRefGUID value sent over from step 5.

7. FireLight will look up the Tx103 based on the TransRefGUID, create a new application, and then display the Edit Application view.

ACORD TX103 Transaction Request

The following exemplifies the minimalistic version of a TX103 that can be accepted by FireLight. See ACORD Tx103 Requirements and Usage for additional information on 103 requirements.

ACORD TX103 Transaction Request

```
<TXLife xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsl:"http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://ACORD.org/Standards/Life/2">
  <TXLifeRequest>
    <TransRefGUID>e74f5211bb4244e88042e6fc222f3937</TransRefGUID>
    <TransType tc="103">OLI_TRANS_NBSUB</TransType>
    <TransExeDate>2011-10-15</TransExeDate>
    <TransExeTime>08:41:17.3373644-06:00</TransExeTime>
    <OLifE>
      <Holding id="9393d31c-7e09-4f4b-a27d-83dafdd542c3">
        <HoldingTypeCode tc="2">OLI_HOLDTYPE_POLICY</HoldingTypeCode>
        <HoldingName>Policy Changes</HoldingName>
        <Policy>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
          <ProductCode>XXX DEFINED PRODUCT CODE</ProductCode>
           <CarrierCode>XXX DEFINED CARRIER Code</CarrierCode>
          <Jurisdiction tc="7">OLI_USA_CO</Jurisdiction>
        </Policy>
      </Holding>
      <Party id="Party_1">
        <PartyTypeCode tc="1">OLI_PT_PERSON</PartyTypeCode>
        <Person>
          <FirstName>Jane</FirstName>
          <LastName>Smith</LastName>
        </Person>
        <Producer>
          <CarrierAppointment PartyID="Party_1">
            <CompanyProducerID>XXX00001</CompanyProducerID>
            <CarrierCode>XXX</CarrierCode>
          </CarrierAppointment>
        </Producer>
      </Party>
      <Relation OriginatingObjectID="Holding_1" RelatedObjectID="Party_1">
        <OriginatingObjectType tc="4">OLI_HOLDING</OriginatingObjectType>
        <RelatedObjectType tc="6">OLI_PARTY</RelatedObjectType>
        <RelationRoleCode tc="37">OLI_REL_PRIMAGENT</RelationRoleCode>
      </Relation>
    </OLifE>
  </TXLifeRequest>
</TXLife>
```

Submitting Compliant Illustration

You can add to the transaction a compliant illustration as a base64 encoded string under the <Attachment> element within the <Holding> aggregate. The original format of the document must be PDF. The document will be included within the final package submitted to the carrier for processing.

The table below lists the required elements that must be part of the <Attachment> aggregate.

Required Elements for the <Attachment> Aggregate

Element	Description
<Description>	A short 255 characters or less description of the document.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Element	Description
<AttachmentData64Binary>	The base64 encoded string value of the PDF document.
<AttachmentType>	“tc” value of “7” for OLI_ATTACH_ ASORIGSOLDILLUS.
<ImageType>	“tc” value of “4” for OLI_IMAGE_PDF.
<AttachmentLocation>	“tc” value of “1” for OLI_INLINE

The following is a sample <Attachment> aggregate:

<Attachment> Aggregate

```
<Attachment>
  <Description>Compliant Illustration</Description>
  <AttchmentData64Binary>JVBERi0xLjQKJeLjz9MKMiAwIG9iaiA8PC9...</AttchmentData64Binary>
  <AttchmentType tc="7">OLI_ATTACH_ASORIGSOLDILLUS</AttachmentType>
  <ImageType tc="4">OLI_IMAGE_PDF</ImageType>
  <AttachmentLocation tc="1">OLI_INLINE</AttachmentLocation>
 </Attachment>
```

Creating a Fee Inquiry Transaction

NOTE: The fee inquiry transaction is used with the FireLight fees, expenses and commissions repository, which requires a licensing fee.

A fee inquiry transaction can be created by sending over a 103 with the transaction type of 1241. This will land the user in the product selection screen for the fee inquiry transaction type. The following sample contains the minimum requirements for the fee inquiry 103 post-over:

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Fee Inquiry Transaction

```
<?xml version="1.0"?>
<TXLife xmlns="http://ACORD.org/Standards/Life/2">
  <TXLifeRequest xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
    <TransRefGUID>2F5B9603-1364-4653-8547-B3C0E2AA6AB0</TransRefGUID>
     <TransType tc="1241">OLI_TRANS_TRNFEEEXPENSE</TransType>
    <OLifE>
      <Holding id="Holding_3b5b9652-97cd-4a5b-8e4e-79141039fd14">
        <HoldingTypeCode tc="2">OLI_HOLDTYPE_POLICY</HoldingTypeCode>
        <Policy>
          <CarrierCode>IT</CarrierCode>
        </Policy>
      </Holding>
        <Party id="Party_c5b61954-69eb-4ba5-8d1d-e3bc241a2c5d">
        <Producer>
          <CarrierAppointment id="CarrierAppointment_4407eb59-45c7-4b54-8ef0-19a8856968f6">
            <CompanyProducerID>IT02000</CompanyProducerID>
          </CarrierAppointment>
        </Producer>
      </Party>
      <Relation id="Relation_94a85ae3-c7ee-42b6-b787-
a259f6acd722" OriginatingObjectID="Holding_3b5b9652-97cd-4a5b-8e4e-
79141039fd14" RelatedObjectID="Party_c5b61954-69eb-4ba5-8d1d-e3bc241a2c5d">
       <OriginatingObjectType tc="4">OLI_HOLDING</OriginatingObjectType>
        <RelatedObjectType tc="6">OLI_PARTY</RelatedObjectType>
        <RelationRoleCode tc="37">OLI_REL_PRIMAGENT</RelationRoleCode>
      </Relation>
    </OLifE>
  </TXLifeRequest>
</TXLife>
```

Submitting a Third Party Needs Determination

You can attach a needs determination transaction completed outside FireLight using attachments. A needs determination attachment differs from other attachments in that it cannot be deleted or modified in the newly created application. It can be used to replace a needs determination template in the package, allowing it to be included in the edit application view and in the signing ceremony. An Upload Document type of Needs Determination must be added to the organization's list profile to support this workflow.

The following is a sample needs determination <Attachment> aggregate:

Needs Determination <Attachment> Aggregate

```
<Attachment>
  <Description>NeedsDetermination</Description>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
  <AttachmentData64Binary>JVBERi0xLjQKJeLjz9KMiAwIG9Ia8PC9…</AttachmentData64Binary>
   <ImageType tc="4" />
    <OLifEExtension VendorCode="25">
    <DocumentTag>NeedsDetermination</DocumentTag>
    </OLifEExtension>
</Attachment>
```

Submitting TX103 Transaction

Typically, you will want to create a proxy from the FireLight web service wsdl. The proxy is then used to communicate with the web service. At a minimum, the sequence is to call the web service, and then check the response to determine if the request succeeded. The following example shows what a typical transaction will look like on the wire.

TX103 Transaction

```
<s:Envelope xmlns:a="http://www.w3.org/2005/08/addressing"
smlns:s="http://www.w3.org/2003/05/soap-envelope">
  <s:Header>
    <a:Action
s:mustUnderstand="1">http://InsuranceTechnologies.com/Acord/ServiceContracts/V2_23_
0/NewBusiness/SubmitNewBusinessApplication</a:Action>
    <a:MessageID>urn:uuid:8e84c184-a95f-4007-aea7-5a1177883a50</a:MessageID>
    <ActivityId CorrelationId="80f526c4-d313-49bf-8e9f-
19a495c045f0"
 xmlns="http://schemas.microsoft.com/2004/09/ServiceModel/Diagnostics">8e3ea05f-2e0e-4ece-
8c17-e99c53d7c8e5</ActivityId>
    <a:ReplyTo>
      <a:Address>http://www.w3.org/2005/08/addressing/anonymous</a:Address>
    </a:ReplyTo>
  </s:Header>
  <s:Body> xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
    <SubmitNewBusinessApplication
xmlns="http://InsuranceTechnologies.com/Acord/ServiceContracts/V2_23_0/">
      <request>
        <TXLife smlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/SMLSchema-
Instance" smlns="http://ACORD.org/Standards/Life/2:>
          <TXLifeRequest>
            <TransRefGUID>7fe5d746-bed5-47c8-8e40-cab1d04f9a58</TransRefGUID>
            <TransType tc="103">OLI_TRANS_NBSUB</TransType>
            <TransExeDate>2011-08-24</TransExeDate>
            <TransExeTime>17:53:03.4134493-06:00</TransExeTime>
            <OLifE>
              <Holding id="Holding_1">
                <HoldingTypeCode tc="2">OLI_HOLDTYPE_POLICY</HoldingTypeCode>
                <Policy>
                  <ProductCode>XXX DEFINED PRODUCT CODE</ProductCode>
                  <CarrierCode>XXX DEFINED CARRIER CODE</CarrierCode>
                  <Jurisdiction tc="28">OLI_USA_MN</Jurisdiction>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
                </Policy>
              </Holding>
              <Party id="Party_1">
                <PartyTypeCode tc="1">OLI_PT_PERSON</PartyTypeCode>
                <Person>
                  <FirstName>Jane</FirstName>
                  <LastName>Smith</LastName>
                </Person>
                <Producer>
                  <CarrierAppointment PartyID="Party_1">
                    <CompanyProducerID>XXX AGENT ID</CompanyProducerID>
                    <CarrierCode>XXX DEFINED CARRIER CODE</CarrierCode>
                  </CarrierAppointment>
                </Producer>
              </Party>
              <Relation OriginatingObjectID="Holding_1" RelatedObjectID="Party_1">
                <OriginatingObjectType tc="4">OLI_HOLDING</OriginatingObjectType>
                <RelatedObjectType tc="6">OLI_PARTY</RelatedObjectType>
                <RelationRoleCode tc="37">OLI_REL_PRIMAGENT</RelationRoleCode>
              </Relation>
            </OLifE>
          <TXLifeRequest>
        </TXLife>
      </request>
    </SubmitNewBusinessApplication>
  </s:Body>
</s:Envelope>
```


FireLight will respond back with a TxLifeResponse indicating if the transaction succeeded or failed. The following is an example of a successful transaction.

Successful Transaction Example


```
<s:Envelope xmlns:s="http://www.w3.org/2003/05/soap-envelope"
xmlns:a="http://www.w3.org/2005/08/addressing">
  <s:Header>
    <a:Action
s:mustUnderstand="1">http://InsuranceTechnologies.com/Acord/ServiceContracts/V2_23_
0/NewBusiness/SubmitNewBusinessApplicationResponse</a:Action>
    <ActivityId CorrelationId="4ae5304e-814c-4fef-a9f9-
e303dd4cfedd"
 xmlns="http://schemas.microsoft.com/2004/09/ServiceModel/Diagnostics">8e3ea05f-2e0e-4ece-
8c17-e99c53d7c8e5</ActivityId>
    <a:RelatesTo>urn:uuid:8e84c184-a95f-4007-aea7-5a1177883a50</a:RelatesTo>
  </s:Header>
  <s:Body xmlns:xsi="http://www.w3.org/2001/XMLSchema-Instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
    <SubmitNewBusinessApplicationResponse
xmlns="http://InsuranceTechnologies.com/Acord/ServiceContracts/V2_23_0/">
      <SubmitNewBusinessApplicationResult>
        <TXLifeResponse xmlns="http://ACORD.org/Standards/Life/2">
          <TransRefGUID>7115d746-bed5-47c8-8340-deb1d04f9a30</TransRefGUID>
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
          <TransType tc="103"></TransType>
          <TransExeDate>2012-03-03</TransExeDate>
          <TransExeTime>16:08:45.5497374-07:00</TransExeTime>
          <TransResult>
            <ResultCode tc="1">RESULT_SUCCESS</ResultCode>
          </TransResult>
          <OLifE>
            <Holding>
              <HoldingKey>7666b513c9be4c1a9ca1ffaeea51db49</HoldingKey>
            </Holding>
          </OLifE>
        </TXLifeResponse>
      </SubmitNewBusinessApplicationResult>
    </SubmitNewBusinessApplicationResponse>
  </s:Body>
</s:Envelope>
```


The “tc” value within the <TransResult> aggregate is used to determine if the transaction succeeded or failed. Possible values returned from FireLight are shown in the following table.

"tc" Values and Descriptions

Value	Description
1 - RESULT_SUCCESS	The transaction was processed successfully with no additional information.
5 - RESULT_FAILURE	The transaction was not processed. The reason for the failure can be identified in the <ResultInfo> aggregate.

Following is an example of a failed transaction.

Failed Transaction Example


```
<s:Envelope xmlns:s="http://www.w3.org/2003/05/soap-envelope"
xmlns:a="http://www.w3.org/2005/08/addressing">
  <s:Header>
    <a:Ation
s:mustUnderstand="1".http://InsuranceTechnologies.com/Acord/ServiceContracts/V2_23_
0/NewBusiness/SubmitNewBusinessApplicationResponse</a:Action>
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
    <ActivityId CorrelationId="af7eb65e-bc42-42b2-96d0-
54f9550b5848"
 xmlns="http://schemas.microsoft.com/2004/09/ServiceModel/Diagnostics">c0cc87d2-07c3-4a53-
83e3-435d0098e7c3</ActivityId>
  </s:Header>
  <s:Body xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
    <SubmitNewBusinessApplicationResponse
xmlns="http://InsuranceTechnologies.com/Acord/ServiceContracts/V2_23_0/">
      <SubmitNewBusinessApplicationResult>
        <TXLifeResponse xmlns="http://ACORD.org/Standards/Life/2">
          <TransResult>
            <ResultCode tc="5">RESULT_FAILURE</ResultCode>
            <ResultInfo>
              <ResultInfoDesc>Organization missing or not configured within
FireLight.</ResultInfoDesc>
              <ResultInfoSysMessageCode>Diagnostic information.</ResultInfoSysMessageCode>
              <ResultInfoSource>System</ResultInfoSource>
            </ResultInfo>
          </TransResult>
        </TXLifeResponse>
      </SubmitNewBusinessApplicationResult>
    </SubmitNewBusinessApplicationResponse>
  </s:Body>
</s:Envelope>
```


Notice an explanation for the failure is stored within the first <ResultInfo> aggregate within the response. If the transaction failed, the external system should notify the user appropriately and not redirect to FireLight.

ACORD Tx103 Response

An ACORD <TXLifeResponse> aggregate will be returned in response to the 103 request from the external system. Stored within the <HoldingKey> element will be the identifier to the application that will be created in FireLight after step 7 in the sequence listed in Create New Application. This identifier can later be used to retrieve status information about the application, or to launch back into FireLight to edit the existing application at a later date. Refer to Edit Existing Application in FireLight on how to launch back into FireLight to edit an existing application.

Post ACORD TX 1228 Transaction

A specially crafted response by the external system is sent to the browser to force a form POST to FireLight to authenticate and authorize the user. Once authenticated, FireLight will redirect the user to the proper landing page within the system. A hidden form with a name of “POXML” is used to transmit the ACORD 1228 transaction to the FireLight system.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

When the external system needs to assign a Firm ID to an agent and/or limit product availability the extended 1228 is used. In order for FireLight to recognize the Firm ID and product availability, the example schema must be followed. See TX1228 Examples.

"Can Sell" Checks for Agents

1. FireLight has two touch points in the system where it can make a call to a web service for this type of validation check: the first is when the application is created and the second is when the application is locked down for signatures.

2. Typically, an ACORD 228 transaction is used for this purpose.

3. A failed response from the first call results in a soft stop (warning message is displayed, but the user can continue with the app) and a failed response from the second call is a hard stop (they cannot move forward within the workflow).

4. For both calls, we can tailor the message displayed to the user.

Home Office Users Can Enter Applications for an Agent

For Home Office users to enter applications on behalf of an agent, we tweak the SSO request to FireLight to contain the Home Office user's information (Name, Email, etc.), but have it reference in the CompanyProducerID field the agent they are impersonating.

Example:

1. Within the carrier portal, you would have a page for the Home Office users to launch into FireLight. On this page the Home Office user would select the agent they wanted to impersonate.

2. The Home Office user would click a link to launch into FireLight. The page would generate a SSO request with the Party aggregate containing the Home Office user name and the agent's identifier (agent ID).

3. FireLight would create the application under the agent ID, but log all audits with the Home Office user name.

4. The agent could later log into the system to complete the application.

Sample HTML Form

The following is a very simple HTML form that will automatically POST the contents of the form to FireLight.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Sample HTML Form

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0
Transitional//EN" "http://www.w3.org/TR/xhtml1/DTDxhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999.xhtml">
  <head>
    <title>Post to FireLight</title>
  </head>
  <body onload="document.form1.submit();">
    <form name="form1" method="post" action=
"https://staging.firelighteapp.com/egapp/passivecall.aspx?refid=4681a7beafd0462984c27a28c101
00b8">
      <input type="hidden" name="POXML" value="html encoded Tx 1228 transaction" />
    </form>
  </body>
</html>
```

The action within the HTML form will point to the published FireLight endpoint. The optional query string parameter on the action varies based on the launch scenario in the “Summary” section. The possible values are noted below.

Launch Scenarios, Query String Parameters and Descriptions

Scenario	Parameter	Description
The creation of a new application.	refid=TransRefGUID	The value of the “refid” must match the TransRefGUID sent over to FireLight within the 103 transaction.
Update an existing application with new data.	refid=TransRefGUID	The value of the “refid” must match the TransRefGUID of the original 103 that was sent over to create the application in FireLight. An updated 103 with the same TransRefGUID is sent over with updated information to the New Business web service.
Launch directly into FireLight to edit an	appid=ApplicationID	The “appid” value must match the <HoldingKey> value returned by FireLight within the original 103 response.

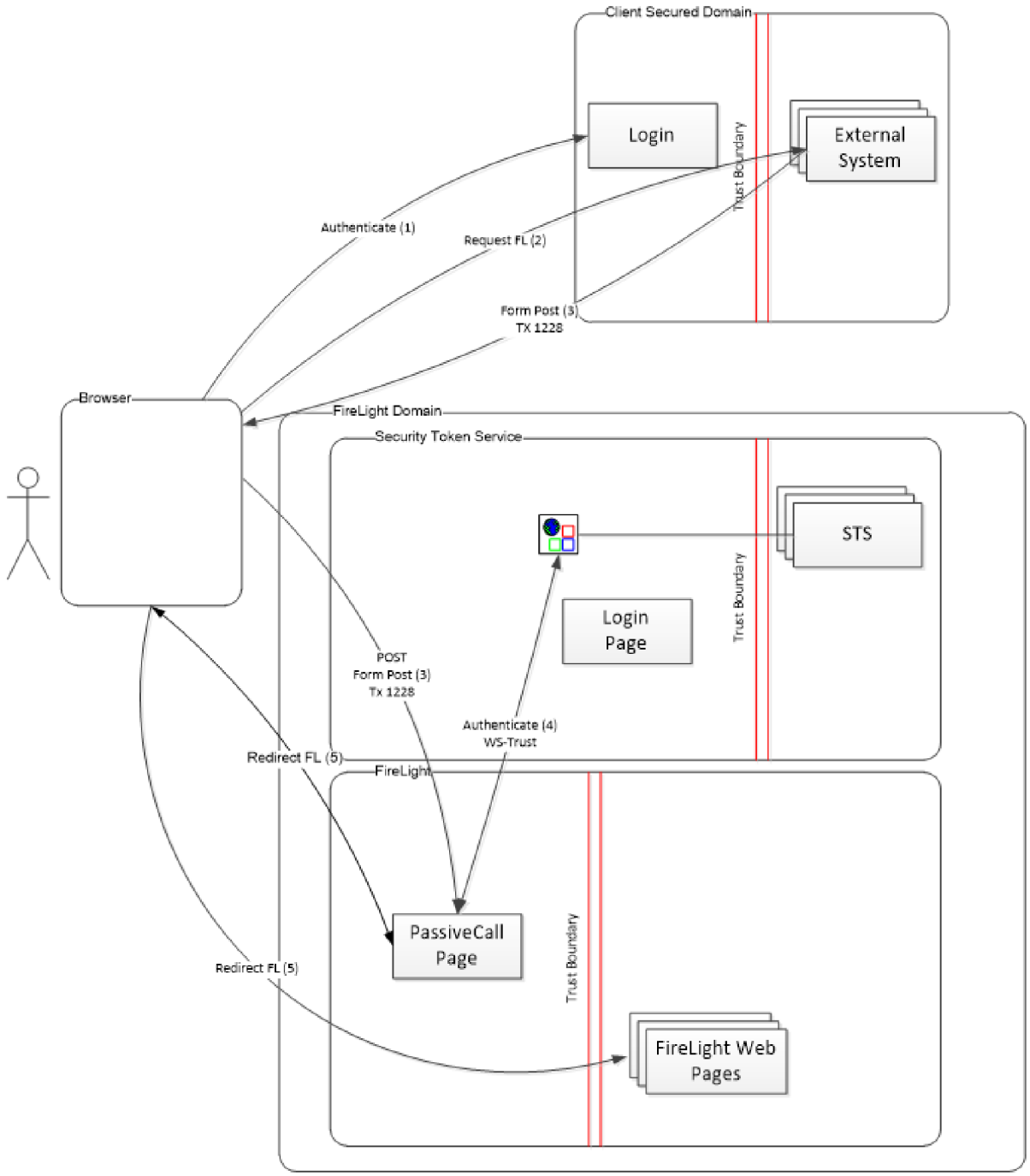
CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Scenario	Parameter	Description
existing application.		
Launch directly into the FireLight dashboard.		No query string parameters are needed.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Edit Existing Application in FireLight

At times, you may want to provide the user the ability to launch directly into the Edit Application view within FireLight without sending over a 103 with update information. The following diagram illustrates the launch sequence for this scenario.



CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

The following is an explanation of each step:

1. The user is authenticated and authorized by the external system.

2. The user clicks on a link to launch into the FireLight dashboard.

3. The external system sends back to the user a specially crafted response to force the browser to POST an ACORD Tx1228 transaction within a hidden form field to the FireLight passive endpoint.

4. FireLight authenticates with the STS using a predefined shared user login and password set up specifically for the implementation. If the credentials are valid, FireLight will impersonate the user identified within the 1228 for the remainder of the session. If the credentials are not valid, the user will be redirected to a generic error page within FireLight.

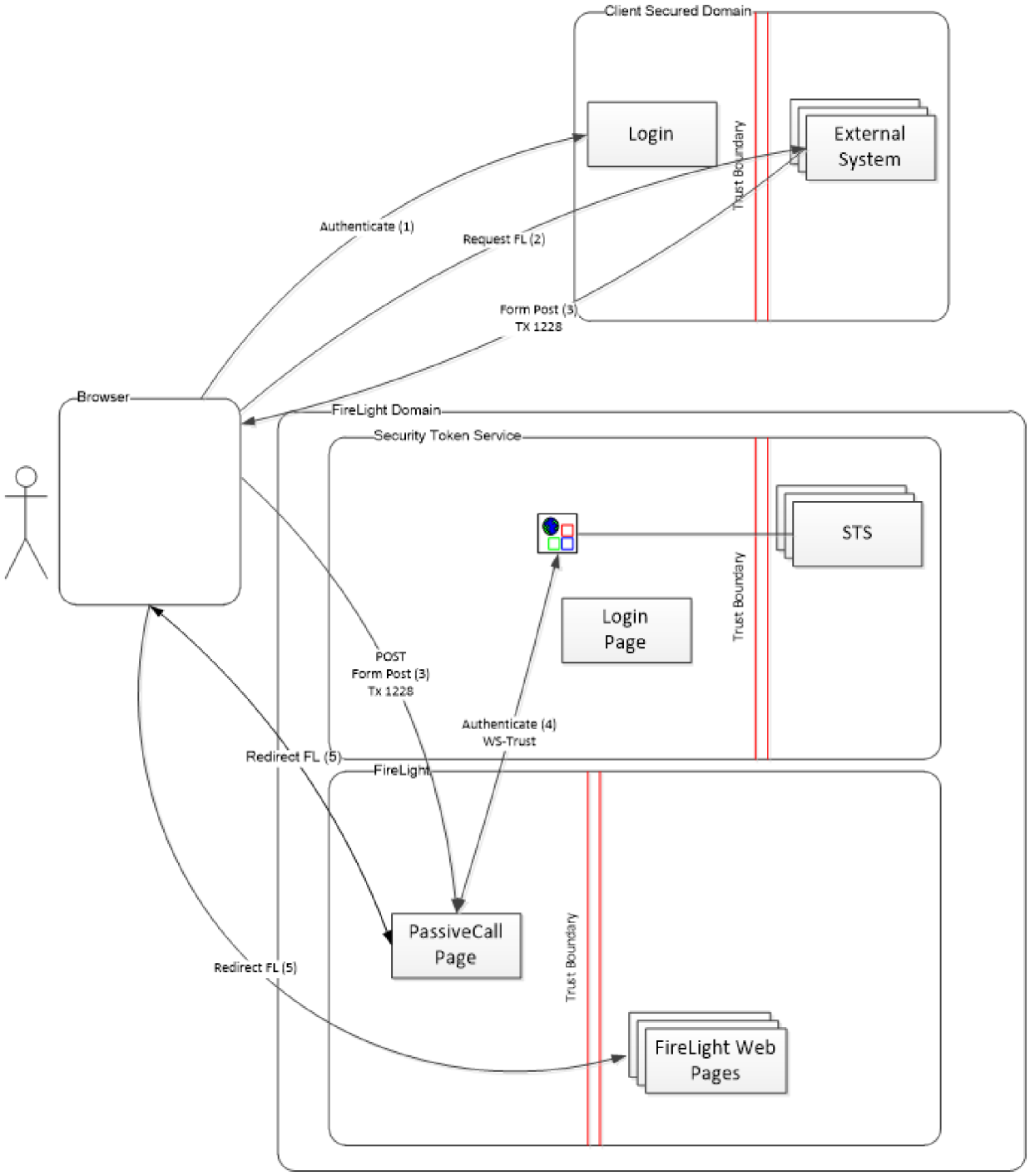
5. FireLight will send back a redirect to the FireLight dashboard.

A query string named “appid” with a value retained from the original <TXLifeResponse>/<HoldingKey> is required to be sent within the action on the form post. See “Sample HTML Form” in Post ACORD TX 1228 Transaction.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Launch into FireLight Dashboard

At times, you may want to launch directly into the FireLight dashboard instead of creating a new application. The following diagram illustrates the launch sequence for this scenario.



CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

The following is an explanation of each step:

1. The user is authenticated and authorized by the external system.

2. The user clicks on a link to launch into the FireLight dashboard.

3. The external system sends back to the user a specially crafted response to force the browser to POST an ACORD Tx1228 transaction within a hidden form field to the FireLight passive endpoint.

4. FireLight authenticates with the STS using a predefined shared user login and password set up specifically for the implementation. If the credentials are valid, FireLight will impersonate the user identified within the 1228 for the remainder of the session. If the credentials are not valid, the user will be redirected to a generic error page within FireLight.

5. FireLight will send back a redirect to the FireLight dashboard.

No query string is needed on the endpoint included within the action element of the Form post. See “Sample HTML Form” in Post ACORD TX 1228 Transaction.

Launch into FireLight Dashboard – Delayed Use of 103

Posting over into FireLight with a 103 that does not contain the product ID and jurisdiction will launch into the FireLight dashboard. Client data from the 103 can be retained and utilized for the duration of the session. The New Business service does additionally require the refid to be included in the URL in order to make use of the delayed 103 functionality. The 103 should also include one of the following parties: insured, annuitant, or owner.

The following is an explanation of each step:

1. The user is authenticated and authorized by the external system.

2. The user clicks on a link to launch into the FireLight dashboard.

3. The external system sends back to the user a specially crafted response to force the browser to POST an ACORD Tx1228 transaction within a hidden form field to the FireLight passive endpoint.

4. FireLight authenticates with the STS using a predefined shared user login and password set up specifically for the implementation. If the credentials are valid, FireLight will impersonate the user identified within the 1228 for the remainder of the

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

session. If the credentials are not valid, the user will be redirected to a generic error page within FireLight.

5. FireLight will send back a redirect to the FireLight dashboard.

6. The user will select the product and jurisdiction for the new application.

7. The system will allow the user to choose whether to use the data from the 103 for the application being created. The checkbox for 'Create the Application for [Insured Name], [Annuitant Name]', or [Owner Name] will default to checked.

8. If the user does not uncheck the box, the associated fields on the forms are pre-filled with the data from the current session's 103. The 103 client information can continue to be used for duration of the session for any applications created.

ACORD Tx103 Requirements and Usage

Overview

The ACORD Tx103 request allows application data to be sent over in the single sign-on request and pre-filled in the corresponding fields on the application. The data in the 103 is either mapped directly over by the provider into the edit application process or saved in the case of the delayed 103. Application data is only available for reuse in the delayed 103 process.

Requirements

The following table describes the elements that are required by FireLight to process the 103 transaction properly. The product code should be omitted in the case of a delayed 103 transaction.

Required Elements and Descriptions

Element	Description
<TransRefGUID>	Transaction Reference GUID. Unique identifier used to uniquely identify this transaction with FireLight.
<ProductCode>	Identifies the product being sold in FireLight. This string must match up to one, and only one, product within FireLight. The value of this field is matched against the CUSPID property within the Product tab of the Admin tool.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Element	Description
<CarrierCode>	Identifies the organization selling the product identified in <ProductCode>. This value must match to one, and only one, organization defined in FireLight. The value is matched against the External Map Code for the organization. Insurance Technologies will provide you with the code for your implementation.
<Jurisdiction>	Identifies the selling jurisdiction for the application. The “tc” value must match one of the ACORD OLI_LU_STATE lookup values.
<Party>	At least one Party object with a relationship type of “37” (OLI_REL_PRIMAGENT) must be present within the transaction. The generated application will be tied to the agent identified within this instance.
<Person>/<LastName>	The name of the created application will be the primary agent’s last name suffixed with “Application”.
<CarrierAppointment>	The contents of this construct are used by FireLight to associate the application to a user created in FireLight to represent the primary agent.
<CompanyProducerID>	A unique identifier of the agent within the organization. The newly created application will be associated with this identifier (aka user). **Note:** the identifier must only be unique within the organization identified in the <CarrierCode> element within the <CarrierAppointment> construct.
<CarrierCode>	Identifies the organization associated with the agent identified in <CompanyProducerID >. Currently in FireLight this should be the same value as the <CarrierCode> within the <Policy> construct.
<Relation>	This construct is used to associate the primary agent’s Party instance to the primary Holding instance.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Element	Description
OriginatingObjectID	Must match the primary Holding instance's ID.
ReleatedObjectID	Must match the primary agent's, Party instance's ID.
<OriginatingObjectType>	The "tc" value must be 4.
<RelatedObjectType>	The "tc" value must be 6.
<RelationRoleCode>	The "tc" value must be 37.

Usage

In addition to application data, the Tx103 request is used to post over the transaction type. The following transaction types are used in the 103 request to initiate a new application: New Business, Subsequent Payment*, Post Issue*, Needs Determination, and Fee Inquiry. Post Issue requires an additional transaction subtype to be sent over in the request to indicate the type of post issue maintenance. Needs Determination requires a transaction type of 116 and a subtype of 11600. Fee Inquiry requires a transaction type of 1241. The transaction type and transaction subtype will be stored in the global dataitems FLI_TRANSTYPE and FLI_TRANSSUBTYPE for use in the rules within the Admin form designer.

To create a new business application, the 103 request should include the following:

<TransType tc="103">OLI_TRANS_NBSUB</TransType>

To create a subsequent payment application, the 103 request should include the following:

<TransType tc="508">OLI_TRANS_PAYMNT</TransType>

To create a post issue application with the maintenance type of Admin changes (changes to name, address, email, phone, owner, or beneficiary), the 103 request should include the following:

<TransType tc="113">OLI_TRANS_POLICYADMIN</TransType>

<TransSubType tc="113"> OLI_TRANS_POLICYADMIN </TransSubType>

To create a post issue application with the maintenance type of withdrawal request, the 103 request should include the following:

<TransType tc="113">OLI_TRANS_POLICYADMIN</TransType>

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

<TransSubType tc="105"> OLI_TRANS_WITHDR </TransSubType>

To create a post issue application with the maintenance type of program additions/changes (automatic withdrawal services, rebalancing, DCA), the 103 request should include the following:

<TransType tc="113">OLI_TRANS_POLICYADMIN</TransType>

<TransSubType tc="107">OLI_TRANS_UPDARR</TransSubType>

To create a fee inquiry transaction, the 103 request should include the following:

<TransType tc="1241"> OLI_TRANS_TRNFEEEXPENSE </TransType>

To create a needs determination transaction, the 103 request should include the following:

<TransType tc="116">OLI_TRANS_NEEDSDET</TransType>

<TransSubType tc="11600">OLI_TRANSSUB_NEEDSDET</TransSubType>

* There is an additional license fee for subsequent payment, post issue and fee inquiry (fees, expenses and commissions repository) support. Existing clients will need an addendum to their contract to support this new feature.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

ACORD Tx1228 Requirements and Usage

Overview

FireLight provides support for an ACORD Tx1228 during the sign on process. This ACORD request can be used to provide login credentials and agent information as well as specify firm information and product availability. When specifying firm information, applications can be tied to a firm so managers who need to review their firm's applications can easily do so.

During SSO login, any parties identified with a relation role code of 120 (OLI_REL_AGENTOF) in the 1228 that have an email address (identified by PrefEmailAddr, primary email, or the first email address if not specified) and a producer registration FirmRegistrationNum value will be maintained and identified within FireLight by the FirmRegistrationNum. When an application lands in the review queue, if a match for the application's Firm identifier is found in the firm contact information maintained in FireLight, the notification will be sent to that firm's email address. If not found, the queue email address will be used as before.

The topics in this section detail the usage and requirements FireLight has for the Tx1228 used in the sign on process.

TX1228 Examples

Simple Example

The simple example is used to provide basic user information.

Tx1228 Simple Example

```
<TXLife xmlns="http://ACORD.org/Standards/Life/2">
  <UserAuthRequest>
    <UserLoginName>XXX SHARED LOGIN NAME</UserLoginName>
    <UserPswd>
      <CryptType>AES</CryptType>
      <CryptPswd>Encrypted Password</CryptPswd>
    </UserPswd>
    <UserDomain>XXX ORGANIZATION CODE</UserDomain>
    <VenderApp>
      <AppName>External System Name</AppName>
    </VenderApp>
  </UserAuthRequest>
  <TXLifeRequest>
    <TransRefGUID>26C1A4FC-C22C-4027-9BF6-575AF38AAE2A</TransRefGUID>
    <TransType tc="1228">OLI_TRANS_TRNPRODINQ</TransType>
    <TransSubType tc="22800">OLI_TRANSSUB_PRODDETALL</TransSubType>
    <OLifE>
      <SourceInfo>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
        <CreationDate>2010-11-02</CreationDate>
        <CreationTime>12:20:03.5876024-06:00</CreationTime>
        <SourceInfoName>Carrier Portal Name</SourceInfoName>
      </SourceInfo>
      <Party id="Agent_Party">
        <PartyTypeCode tc="1">OLI_PT_Person</PartyTypeCode>
        <FullName>Agents Full Name</FullName>
        <EMailAddress>
          <AddrLine>agents_email@insurancetechnologies.com</AddrLine>
        </EMailAddress>
        <Person/>
        <Producer>
          <CarrierAppointment PartyID="Agent_Party">
            <CompanyProducerID>XXX AGENT ID</CompanyProducerID>
            <CarrierCode>XXX DEFINED CARRIER CODE</CarrierCode>
          </CarrierAppointment>
        </Producer>
      </Party>
      <Relation OriginatingObjectID="Agent_Party">
        <RelationRoleCode tc="11">OLI_REL_AGENT</RelationRoleCode>
      </Relation>
    </OLifE>
  </TXLifeRequest>
</TXLife>
```

Extended Example

The extended example is used to optionally associate the agent with one or more firms, as well as optionally specify products available per firm.

Tx1228 Extended Example

```
<TXLife xmlns="http://ACORD.org/Standards/Life/2">
  <UserAuthRequest>
    <UserLoginName>XXX SHARED LOGIN NAME</UserLoginName>
    <UserPswd>
      <CryptType>AES</CryptType>
      <CryptPswd>Encrypted Password</CryptPswd>
    </UserPswd>
    <UserDomain>XXX ORGANIZATION CODE</UserDomain>
    <VenderApp>
      <AppName>External System Name</AppName>
    </VenderApp>
  </UserAuthRequest>
  <TXLifeRequest>
    <TransRefGUID>26C1A4FC-C22C-4027-9BF6-575AF38AAE2A</TransRefGUID>
    <TransType tc="1228">OLI_TRANS_TRNPRODINQ</TransType>
    <TransSubType tc="22800">OLI_TRANSSUB_PRODDETALL</TransSubType>
    <OLifE>
      <SourceInfo>
        <CreationDate>2010-11-02</CreationDate>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
        <CreationTime>12:20:03.5876024-06:00</CreationTime>
        <SourceInfoName>Carrier Portal Name</SourceInfoName>
      </SourceInfo>
      <Party id="Agent_Party">
        <PartyTypeCode tc="1">OLI_PT_Person</PartyTypeCode>
        <FullName>Agents Full Name</FullName>
        <EMailAddress>
          <AddrLine>agents_email@insurancetechnologies.com</AddrLine>
        </EMailAddress>
        <Person/>
        <Producer>
          <CarrierAppointment PartyID="Agent_Party">
            <CompanyProducerID>XXX AGENT ID</CompanyProducerID>
            <CarrierCode>XXX DEFINED CARRIER CODE</CarrierCode>
          </CarrierAppointment>
        </Producer>
      </Party>
      <Party id="Firm_Party_1">
        <PartyTypeCode tc="2">Organization</PartyTypeCode>
        <FullName>Firm Name</FullName>
        <Producer>
          <CarrierAppointment>
            <CarrierCode>XXX DEFINED CARRIER CODE</CarrierCode>
            <DistributionAgreementInfo id="DistributionAgreementID_1ID"
                                       DistributionAgreementID="DistributionAgreementID_1"
/>
          </CarrierAppointment>
          <Registration>
            <FirmName>Firm Name</FirmName>
            <FirmRegistrationNum>XXX FIRM ID</FirmRegistrationNum>
          </Registration>
        </Producer>
      </Party>
      <Party id="Firm_Party_2">
        <PartyTypeCode tc="2">Organization</PartyTypeCode>
        <FullName>Firm Name 2</FullName>
        <Producer>
          <CarrierAppointment>
            <CarrierCode>XXX DEFINED CARRIER CODE</CarrierCode>
            <DistributionAgreementInfo id="DistributionAgreementID_2ID"
                                       DistributionAgreementID="DistributionAgreementID_2"
/>
          </CarrierAppointment>
          <Registration>
            <FirmName>Firm Name 2</FirmName>
            <FirmRegistrationNum>ZZZ FIRM ID</FirmRegistrationNum>
          </Registration>
        </Producer>
      </Party>
      <Relation OriginatingObjectID="Agent_Party">
        <RelationRoleCode tc="11">OLI_REL_AGENT</RelationRoleCode>
      </Relation>
      <Relation OriginatingObjectID="Firm_Party_1" RelatedObjectID="Agent_Party">
        <RelationRoleCode tc="120">Agent of Agency</RelationRoleCode>
        <Sequence>0</Sequence>
      </Relation>
      <Relation OriginatingObjectID="Firm_Party_2" RelatedObjectID="Agent_Party">
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
        <RelationRoleCode tc="120">Agent of Agency</RelationRoleCode>
        <Sequence>1</Sequence>
      </Relation>
      <DistributionAgreement id="DistributionAgreementID_1">
        <PolicyProductInfo id="PolicyProductInfo_1">
          <ProductCode>Product1Code</ProductCode>
          <CarrierCode>XXX DEFINED CARRIER CODE</CarrierCode>
          <OLifEExtension>
            <CusipNum>PRODUCT1_CUSIP</CusipNum>
          </OLifEExtension>
        </PolicyProductInfo>
        <PolicyProductInfo id="PolicyProductInfo_2">
          <ProductCode>Product2Code</ProductCode>
          <CarrierCode>XXX DEFINED CARRIER CODE</CarrierCode>
          <OLifEExtension>
            <CusipNum>PRODUCT2_CUSIP</CusipNum>
          </OLifEExtension>
        </PolicyProductInfo>
      </DistributionAgreement>
      <DistributionAgreement id="DistributionAgreementID_2">
        <PolicyProductInfo id="PolicyProductInfo_3">
          <ProductCode>Product3Code</ProductCode>
          <CarrierCode>XXX DEFINED CARRIER CODE</CarrierCode>
          <OLifEExtension>
            <CusipNum>PRODUCT3_CUSIP</CusipNum>
          </OLifEExtension>
        </PolicyProductInfo>
        <PolicyProductInfo id="PolicyProductInfo_4">
          <ProductCode>Product4Code</ProductCode>
          <CarrierCode>XXX DEFINED CARRIER CODE</CarrierCode>
          <OLifEExtension>
            <CusipNum>PRODUCT4_CUSIP</CusipNum>
          </OLifEExtension>
        </PolicyProductInfo>
      </DistributionAgreement>
    </OLifE>
  </TXLifeRequest>
</TXLife>
```

User Nickname Example

User Nickname Example

```
TxLifeRequest Templete
<TXLife xmlns="http://ACORD.org/Standards/Life/2">
  <UserAuthRequest>
    <UserLoginName>UserName</UserLoginName>
    <UserPswd>
      <CryptType>AES</CryptType>
      <CryptPswd>UserPassword</CryptPswd>
    </UserPswd>
    <UserDomain>IT</UserDomain>
```

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
    <VendorApp>
      <AppName>ForeSight</AppName>
    </VendorApp>
  </UserAuthRequest>
  <TXLifeRequest>
    <TransRefGUID></TransRefGUID>
    <TransType tc="1228"></TransType>
    <TransSubType tc="22800"></TransSubType>
    <OLifE>
      <SourceInfo>
        <CreationDate>2013-09-20</CreationDate>
        <CreationTime>11:21:15.1855895-06:00</CreationTime>
        <SourceInfoName>IT</SourceInfoName>
      </SourceInfo>
      <Party id="Agent_Party">
        <PartyTypeCode tc="1">Person</PartyTypeCode>
        <FullName>IT TESTER FULLNAME</FullName>
        <EMailAddress>
          <AddrLine>tester@insurancetechnologies.com</AddrLine>
        </EMailAddress>
        <Person >
          <NickName>Person Logging in</NickName>
        </Person>
        <Producer>
          <CarrierAppointment>
            <CompanyProducerID>UserId</CompanyProducerID>
            <CarrierCode>IT</CarrierCode>
          </CarrierAppointment>
        </Producer>
      </Party>
      <OLifEExtension VendorCode="25">
        <UserRoleCode tc="10">OverRide</UserRoleCode>
      </OLifEExtension>
    </OLifE>
  </TXLifeRequest>
</TXLife>
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Elements within the Tx1228

The following table describes each element within the Tx1228.

Elements within the Tx1228

Element	Description
<UserAuthRequest>	Required aggregate when the Tx1228 is a POST to the passive FireLight endpoint. When used in SSO this should not be included. It contains the predefined login information used by FireLight to authenticate the external system with FireLight.
<UserLoginName>	The predefined login name set up for this subscription to FireLight. All requests will use the same login name and password.
<UserPwd>	Required aggregate containing the predefined password for authenticating the external system with FireLight.
<CryptType>	Required with a value of “AES”.
<CryptPswd>	Required. This element contains a SHA1 hash of the shared password appended to a SHA1 hash of the <TXLifeRequest> aggregate, which is then signed using AES and the entire value base64 encoded.
<UserDomain>	Required. Used by FireLight to identify the organization subscription to FireLight.
<VendorApp>	Required aggregate used to identify the name of the external system.
<AppName>	External system name.
<TXLifeRequest>	Required aggregate containing information about the actual user of the system.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Element	Description
<TransRefGUID>	Required and must be unique for each transaction.
<TransType>	Required and “tc” value must be “1228” (OLI_TRANS_TRNPRODINQ).
<TransSubType>	Optional, but must have a “tc” value of “22800” (OLI_TRANSSUB_PRODDETALL).
<OLifE>	Required aggregate.
<SourceInfo>	Optional aggregate used by FireLight logging to indicate the actual referring system identifier. For instance, if you have multiple ways into FireLight, you can use the <SourceInfoName> to identify which launch mechanism (e.g. portal) was used.
<Party>	Required aggregate containing information used by FireLight to identify the actual user of the system. Optionally, used to identify Firms the user is associated with.
<FullName>	Optional if <Person><FirstName> and <LastName> are supplied. Required if <Person> aggregate is not used.
<Person>	Required if <FullName> not included within the <Party> aggregate. **Note:** When the <NickName> attribute is used with the <Person> aggregate, it allows a user to log in to FireLight under another user’s account and still maintain accurate case ownership information. The FireLight audit records record the nickname for the logged in user. The owner of the case retains the agent information and it is displayed in various parts of the system.
<Producer>	Required aggregate used to uniquely identify the

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Element	Description
	user within the FireLight system.
<CarrierAppointment>	Required aggregate containing an identifier used by FireLight to identity the user and/or firms distribution agreement.
<CompanyProducerID>	Unique identifier from the external system that will be used by FireLight to uniquely identify the user.
<CarrierCode>	Used by FireLight to identify the organization subscription to FireLight.
<Relation>	Required aggregate used to identify the role of the <Party> aggregate.
<DistributionAgreementInfo>	Optional aggregate to identify the distribution agreement aggregate for that organization.
<Registration>	Optional aggregate to identify the organization firm information.
<FirmName>	Optional aggregate to identify the organization firm name.
<FirmRegistrationNum>	Optional aggregate to identify the organization Firm ID.
<Relation>	Required aggregate used to identify the role of the <Party> aggregates and the relation between Person and Organization <Party>.
Aggregates when specifying distribution agreements and associated firms.	
<DistributionAgreement>	Optional aggregate to identify product availability.
<PolicyProductInfo>	Optional aggregate to identify a product defined in FireLight.
<ProductCode>	External code to help identify product within

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Element	Description
	FireLight.
<OLifEExtension>	Aggregate to extend the ACORD (v2.27) model.
<CusipNum>	External code to help identify a product defined in FireLight.
<DistributionAgreement>	Optional aggregate to identify product availability.

Usage

POST ACORD Tx1228 to FireLight

When the external system performs a POST to FireLight to authenticate and authorize a user the <UserAuthRequest> element is required.

Generating the <CryptPswd> Contents

Requirement:

> Ensure that the value being passed is exactly the same as the value being hashed. If using an XML parser, ensure the string returned from that tool is the same as what was printed to the HTML. For example, the parser might return a shortened form of the empty elements and the XML in the form sent to FireLight might have the long version.

The value stored within the <CryptPswd> element is generated by following these steps:

1. Create a SHA1 hash of the <TXLifeRequest> aggregate contents.
    - Newline and space characters between tags in the TXLifeRequest element should be removed.
    - When creating the hash code from the TXLIfeRequest XML fragment, include the ACORD namespace in the root element of the fragment. So when you pull out the TXLifeRequest node, the <TXLifeRequest> would look like the following:

      <TXLifeRequest xmlns="http://ACORD.org/Standards/Life/2">
2. Convert the hashed byte array to upper-cased hex values.
3. Concatenate the password to the end of the generated hash from step 2.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

4. Sign the contents from step 2 using the AES, symmetrical algorithm. To initialize the algorithm, set the Key and Initialization Vector to a 16 byte value derived from the Shared key provided to you from Insurance Technologies. See the next section on how to derive the actual key.

   If the provider you are using for the encryption allows you to set the blocking mode and padding scheme, select “Cipher Block Chaining (CBC)” for blocking mode and “PKCS5” for the padding mode.

5. Base64 encode the value returned from step 4.

6. Store the result in the <CryptPswd> element.

Deriving the IV from the Shared Key

The minimum key size for the AES algorithm is 16 bytes. The Shared Key provided to you from Insurance Technologies may be less than the 16 bytes minimum for the algorithm. To generate a key with the required length, repeat the characters within the Shared Key until the desired length is achieved. For example, if the Shared Key is “SomeKeyValue”, which has a length of 12. The actual key for the algorithm would be “SomeKeyValueSome”.

Tx1228 in an SSO

When accessing FireLight using an SSO from an external system the Tx1228 is optional. When including the Tx1228 the <UserAuthRequest> element is not required, because the user is authenticated via the SSO.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

FireLight SSO Requirements

FireLight supports both WS-Federation and SAML 2.0 protocols for Single Sign On. This section lists the requirements for both of the protocols.

SAML 2.0

FireLight supports idp-initiated SO using the HTTP Post binding. Listed below are the required and optional assertions supported by FireLight.

Required Assertions

Identifier

The certificate identifier must be included within the <Issuer> element.

Name ID

The user's name must be included within a <NameID> within the <Subject> element.

User Role

The User Role defines the type of user logging into the FireLight system. FireLight uses the User Role to determine what major functional area(s) of the system the user will be allowed to access. Listed below are the current options:

- Agent – Normal agent access, no visibility into the Review Queue capabilities.
- OSJ – Review Queue visibility only.
- Client – Access to the Sign-Later functionality only.

SAMLAttribute Definition

- Name - http://schemas.microsoft.com/ws/2008/06/identity/claims/role
- Friendly Name - USER_ROLE

User Rights

The User Rights defines the permission set the user will be granted during the FireLight session. FireLight uses the permission set to limit the actions a user may perform within each functional area. Listed below are the possible options:

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- Full – Access to all features of the functional area.
- Limited – Limited set of capabilities within the functional area.

SAMLAttribute Definition

- Name – http://schemas.insurancetechnologies.com/2010/01/identity/claims/rights
- Friendly Name – USER_RIGHTS

Organization ID

The Organization ID is used by FireLight to determine the submitting organization.

SAMLAttribute Definition

- Name – http://schemas.insurancetechnologies.com/2010/01/identity/claims/organizationid
- Friendly Name – ORGANIZATION_ID

Name Identifier

The Name Identifier is a key managed by the caller to uniquely identify the user to FireLight. This assertion is optional if the optional SSO Session Info is included within the request.

SAMLAttribute Definition

- Name – http://schemas.xmlsoap.org/ws/2005/05/identity/claims/privatepersonalidentifier
- Friendly Name – NAME_IDENTIFIER

Optional Assertions

The following assertions or claims are optional within the SSO request.

SSO Session Info

This assertion can be included within the SSO request to pass additional information about the agent to FireLight. The content of the assertion is based on the ACORD 1228 transaction. The assertion value is the base64 encoded string representation of the ACORD 1228 transaction.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Refer to the FireLight External Integration document for the structure and requirements for the transaction.

SAMLAttribute Definition

- Name - http://schemas.insurancetechnologies.com/2010/01/identity/claims/ssosessioninfo
- Friendly Name - SSO_SESSION_INFO

SSO New Business Transaction

An optional ACORD TX103 can be included within the SSO request. If the assertion is present, FireLight will create a new application based on the data contained within the transaction and redirect the user to the Edit Application view. The assertion value is the base64 encoded string representation of the ACORD 103 transaction.

Any Attachment aggregates within the transaction will be stored as additional documents tied to the application. A typical use of the Attachment aggregate is to pass a compliant Illustration generated from an external system to FireLight.

Refer to the FireLight External Integration document for the structure and requirements for the transaction.

SAMLAttribute Definition

- Name - http://schemas.insurancetechnologies.com/2010/01/identity/claims/ssonb
- Friendly Name - SSO_NB

External Role Code

This assertion can be included within the SSO request to specify the external role code for the user.

SAMLAttribute Definition

- Name - http://schemas.insurancetechnologies.com/2010/01/identity/claims/externalrolecode
- Friendly Name - EXTERNAL_ROLE_CODE

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Branding

This assertion can be included to override the default branding for the user's FireLight session. The identifier must be agreed upon, and the associated branding content (colors, images, etc.) must also be provided and added to the FireLight application web content.

SAMLAttribute Definition

- Name - http://schemas.insurancetechnologies.com/2010/01/identity/claims/branding
- Friendly Name - BRANDING

Sample SAML 2.0 Response

Following is a sample SAMLResponse with the optional TX1228.

Sample SAML 2.0 Response


```
<samlp:Response ID="_1eae52b0-e66d-4703-8484-8cef9d440780" Version="2.0"
          IssueInstant="2013-04-14T23:37:50.3Z"
         Destination="https://localhost/egapp/idp-
initiatedsso.aspx"          xmlns:samlp="urn:oasis:names:tc:SAML:2.0:protocol">
  <saml:Issuer
xmlns:
saml="urn:oasis:names:tc:SAML:2.0:assertion">https://localhost/egsts/trust</saml:Issuer>
  <samlp:Status>
    <samlp:StatusCode Value="urn:oasis:names:tc:SAML:2.0:status:Success" />
  </samlp:Status>
  <saml:Assertion Version="2.0" ID="_4d645a2f-c5d9-40eb-98b9-998a941601b2"
     IssueInstant="2013-04-
14T23:37:50.303Z"          xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion">
    <saml:Issuer>https://localhost/egsts/trust</saml:Issuer>
    <saml:Subject>
      <saml:NameID>Sample User</saml:NameID>
      <saml:SubjectConfirmation  Method="urn:oasis:names:tc:SAML:2.0:cm:bearer">
        <saml:SubjectConfirmationData
          Recipient="https://localhost/egapp/idp-initiatedsso.aspx" />
      </saml:SubjectConfirmation>
    </saml:Subject>
    <saml:AuthnStatement  AuthnInstant="2013-04-14T23:37:50.305Z">
      <saml:AuthnContext>
        <saml:AuthnContextClassRef>
          urn:oasis:names:tc:SAML:2.0:ac:classes:Password
        </saml:AuthnContextClassRef>
      </saml:AuthnContext>
    </saml:AuthnStatement>
    <saml:AttributeStatement>
      <saml:Attribute

Name
="http://schemas.microsoft.com/ws/2008/06/identity/claims/role"          FriendlyName="USER_
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
ROLE">
        <saml:AttributeValue>Agent</saml:AttributeValue>
      </saml:Attribute>
      <saml:Attribute

Name
="http://schemas.insurancetechnologies.com/2010/01/identity/claims/rights"
          FriendlyName="USER_RIGHTS">
        <saml:AttributeValue>Full</saml:AttributeValue>
      </saml:Attribute>

<saml:Attribute
Name
="http://schemas.insurancetechnologies.com/2010/01/identity/claims/organizationid"
          FriendlyName="ORGANIZATION_ID">
        <saml:AttributeValue>ITB</saml:AttributeValue>
      </saml:Attribute>

<saml:Attribute
Name
="http://schemas.xmlsoap.org/ws/2005/05/identity/claims/privatepersonalidentifier"
          FriendlyName="NAME_IDENTIFIER">
        <saml:AttributeValue>00000000-0000-0000-0000-000000000002</saml:AttributeValue>
      </saml:Attribute>

<saml:Attribute
Name
="http://schemas.insurancetechnologies.com/2010/01/identity/claims/ssosessioninfo"
          FriendlyName="SSO_SESSION_INFO">
        <saml:AttributeValue>
          “base64 encoded string”
        </saml:AttributeValue>
      </saml:Attribute>
    </saml:AttributeStatement>
  </saml:Assertion>
</samlp:Response>
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

IDP-Initiated SSO Endpoint

The following endpoints will accept a SML 2.0 idp-initiated SAMLResponse.

SAML 2.0 IDP-Initiated SSO Endpoints

Environment	SAML 2 IDP-Initiated SSO	Status
External QA	https://flqa.insurancetechnologies.com/egapp/idp-initiatedsso.aspx	Active
UAT	https://uat.firelighteapp.com/egapp/idp-initiatedsso.aspx	Active
Staging	https://staging.firelighteapp.com/egapp/idp-initiatedsso.aspx	Active
Production	https://www.firelighteapp.com/egapp/idp-initiatedsso.aspx	Active

Possible Redirects

After authentication the user will be redirected to one of the following pages within the FireLight system:

1. If the SSO contained a TX103 transaction, the user will be redirected to the Edit Application View.

2. If the URL contains the query string “appId” and the referenced application does not exist within FireLight, the user will be redirected to a page indicating the application is not on file.

3. If the URL contains the query string “appId” and references a valid FireLight application identifier owned by the authenticated user, the user will be redirected to the Edit Application View, unless the RelayState is set to eSign.

4. If the RelayState is Set to eSign and the URL contains the query string “appId” and the appId references a valid FireLight application identifier owned by the authenticated user and the application is Locked or PendingSignature Status, then the User will be redirected to the eSign View.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

5. If the RelayState is Set to eSign and the URL contains the query string “appId” and the appId references a valid FireLight application identifier owned by the authenticated user but the Application is not in a valid status for Signatures, the user will be redirected to an Invalid Request Page.

6. If no TX103 exists and the user is authenticated, the user will be redirected to the FireLight Home page.

7. If the request fails authentication or validation for any reason, the user will be redirected to a FireLight error page.

8. If the form post RelayState field contains one of the following values, the user will redirected to the corresponding page in FireLight:

   Home => Home page

   NewApplication => New Application page

   Applications => All Applications page

   OSJ => Review Queue page

Managing Firms and Product Availability

This is done via FireLight Groups and the user sign on process. It provides an alternative to creating a Group for each Firm and the complexity that comes with managing many groups. A simple scenario should only consist of a Management group and an Agent group, where agents specify the firms they sell for in their sign on and managers specify the firm they manage in their sign on.

Details

- A User can be associated with a single firm. Limiting product availability in the sign on is optional when associated with a single firm.

- A User can be associated with multiple firms. Limiting product availability in the sign on is required if multiple firms are present.

- A product cannot be available under multiple firms for a user.

- A single Firm is tied to an application during creation.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

  - If product availability per firm is specified, the firm is determined from the selected product in the application.
  - If product availability is not specified, the firm is determined from the single firm associated to the user.
- Groups must also be set up in the FireLight Admin tool.

Groups

When Groups, Firms and product availability are specified in the SSO it is possible to simplify groups into two major groups: Managers and Agents. The Agents group would be configured to sell all products. By signing onto FireLight with a 1228 specifying product availability, the agent could create applications with only the specified products. The Managers group would be configured to have access to cases in the Agents group as well as limiting case access to the user's Firm ID. By signing onto FireLight with a 1228 specifying a Firm, the manager would be able to access all applications in the Agents group that are associated with their firm.

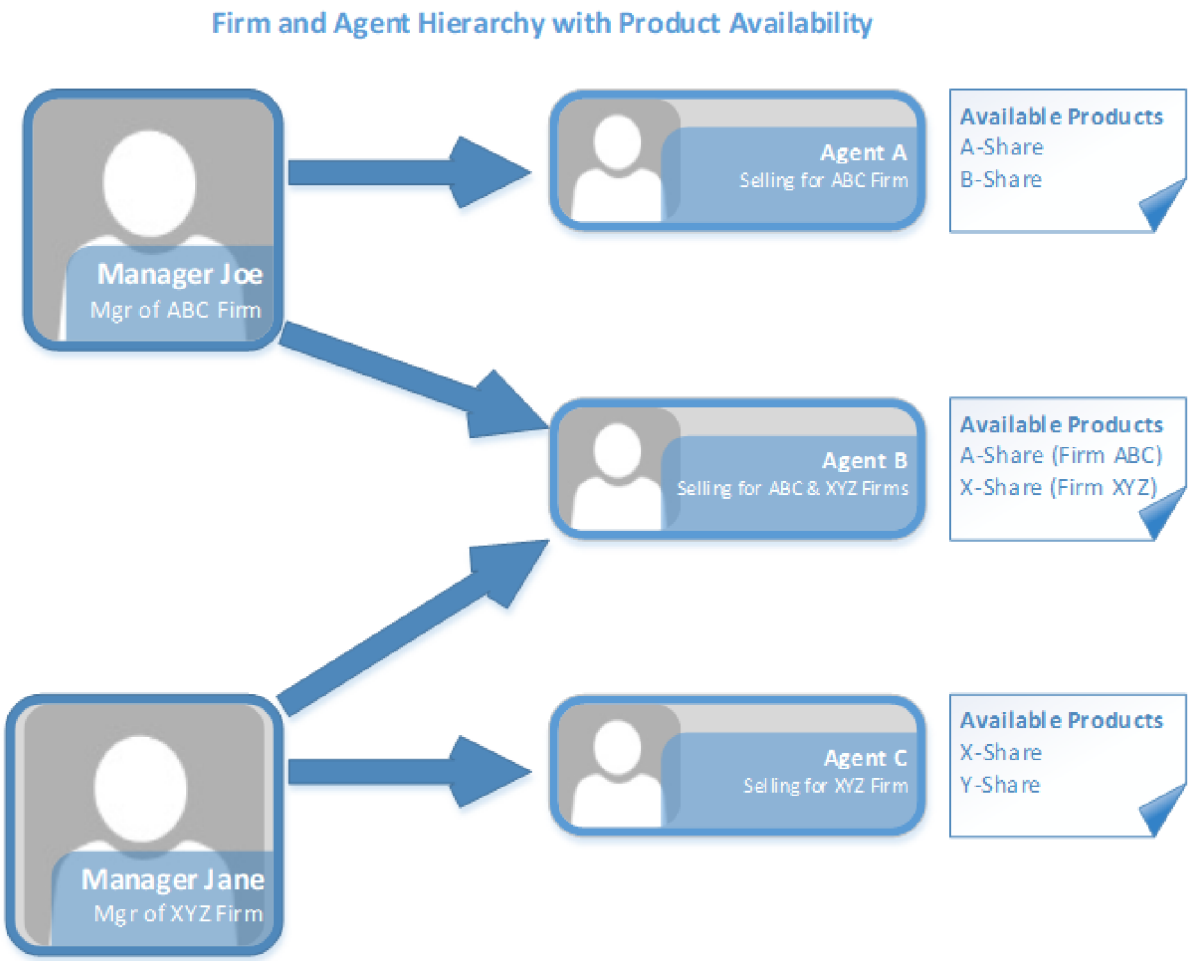
Requirements

Following are the requirements needed to support this functionality. Items in orange require custom integration provider work.

- Set up Group(s) for Managers
  - Case Access to necessary groups
  - Case Access by Firm ID
- Set up Group(s) for Agents
  - Product access can be set up per group
  - or set up as a Super-Set and filtered via 1228
- Users must sign on using a 1228, which assigns a Group and Firm to the user
- Optionally, the 1228 can specify product availability per firm
  - **If product availability is limited via the 1228 then custom work in the integration provider is necessary.**

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

- 1228 should be structured as specified in the FireLight SSO Requirements doc
    - **If the 1228 cannot be structured exactly per specifications then custom work in the integration provider is necessary to restructure per FireLight specifications.**
    - **If the 1228 cannot be sent via SSO or Post Over then FireLight can call an external system to get a valid 1228. This also requires custom work in the integration provider.**



CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

FireLight Groups

- Agent B signs on via SSO under Agents group with a 1228 specifying relation to ABC Firm, XYZ Firm as well as ability to sell products 'A-Share' for ABC Firm and 'X-Share' for XYZ Firm.
- Agent B creates an application using A-Share product. Note: Only A-Share and X-Share products are available.
- The new Application is assigned to Agent B and ABC Firm.
- Agent B creates an application using X-Share product.
- The new application is assigned to Agent B and XYZ Firm.
- Manager Joe signs on via SSO under Managers group with a 1228 specifying relation to ABC Firm.
- Manager Joe views all applications from all owners. This list is filtered to only applications assigned to ABC Firm.
- Manager Jane signs on via SSO under Managers group with 1228 specifying relation to XYZ Firm.
- Manager Jane views all applications from all owners. This list is filtered to only applications assigned to XYZ Firm.

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Sample 1228

The sample below shows a 1228 for Agent B in the diagrams above. This agent is allowed to sell two products. Each product is sold under a separate firm. Applications created with A-Share are visible to Manager Joe and applications created with X-Share are visible to Manager Jane.

Sample 1228


```
<TXLife xmlns="http://ACORD.org/Standards/Life/2">
  <TXLifeRequest>
    <TransRefGUID>26C1A4FC-C22C-4027-9BF6-575AF38AAE2A</TransRefGUID>
    <TransType tc="1228">OLI_TRANS_TRNPRODINQ</TransType>
    <TransSubType tc="22800">OLI_TRANSSUB_PRODDETALL</TransSubType>
    <OLifE>
      <Party id="Agent_Party">
        <PartyTypeCode tc="1">OLI_PT_Person</PartyTypeCode>
        <FullName>Agents B</FullName>
        <EMailAddress>
          <AddrLine>agentB_email@insurancetechnologies.com</AddrLine>
        </EMailAddress>
        <Person/>
        <Producer>
          <CarrierAppointment PartyID="Agent_Party">
            <CompanyProducerID>TEST_AGENT</CompanyProducerID>
            <CarrierCode>XYZ</CarrierCode>
          </CarrierAppointment>
        </Producer>
      </Party>
      <Party id="Firm_Party_1">
        <PartyTypeCode tc="2">Organization</PartyTypeCode>
        <FullName>Firm ABC</FullName>
        <Producer>
          <CarrierAppointment>
            <CarrierCode>XYZ</CarrierCode>
            <DistributionAgreementInfo id="DistributionAgreementID_
1ID" DistributionAgreementID="DistributionAgreementID_1" />
          </CarrierAppointment>
          <Registration>
            <FirmName>Firm ABC</FirmName>
            <FirmRegistrationNum>FirmABC</FirmRegistrationNum>
          </Registration>
        </Producer>
      </Party>
      <Party id="Firm_Party_2">
        <PartyTypeCode tc="2">Organization</PartyTypeCode>
        <FullName>Firm XYZ</FullName>
        <Producer>
          <CarrierAppointment>
            <CarrierCode>XYZ</CarrierCode>
            <DistributionAgreementInfo id="DistributionAgreementID_
2ID" DistributionAgreementID="DistributionAgreementID_2" />
          </CarrierAppointment>
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

```
          <Registration>
            <FirmName>Firm XYZ</FirmName>
            <FirmRegistrationNum>FirmXYZ</FirmRegistrationNum>
          </Registration>
        </Producer>
      </Party>
      <Relation OriginatingObjectID="Agent_Party">
        <RelationRoleCode tc="11">OLI_REL_AGENT</RelationRoleCode>
      </Relation>
      <Relation OriginatingObjectID="Firm_Party_1" RelatedObjectID="Agent_Party">
        <RelationRoleCode tc="120">Agent of Agency</RelationRoleCode>
        <Sequence>0</Sequence>
      </Relation>
      <Relation OriginatingObjectID="Firm_Party_2" RelatedObjectID="Agent_Party">
        <RelationRoleCode tc="120">Agent of Agency</RelationRoleCode>
        <Sequence>1</Sequence>
      </Relation>
      <DistributionAgreement id="DistributionAgreementID_1">
        <PolicyProductInfo id="PolicyProductInfo_1">
          <ProductCode>ASHARE</ProductCode>
          <CarrierCode>XYZ</CarrierCode>
          <OLifEExtension>
            <CusipNum>12345</CusipNum>
          </OLifEExtension>
        </PolicyProductInfo>
      </DistributionAgreement>
      <DistributionAgreement id="DistributionAgreementID_2">
        <PolicyProductInfo id="PolicyProductInfo_3">
          <ProductCode>XSHARE</ProductCode>
          <CarrierCode>XYZ</CarrierCode>
          <OLifEExtension>
            <CusipNum>54321</CusipNum>
          </OLifEExtension>
        </PolicyProductInfo>
      </DistributionAgreement>
    </OLifE>
  </TXLifeRequest>
</TXLife>
```


CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Token SSO

FireLight offers an API Token Service to obtain security JWT tokens for use in a number of API calls. These tokens are also able to be used in an SSO interaction. Details on how to obtain these tokens are found in the API Token Service section. In this interaction, the user 1228 xml is required when requesting a token.

Overview

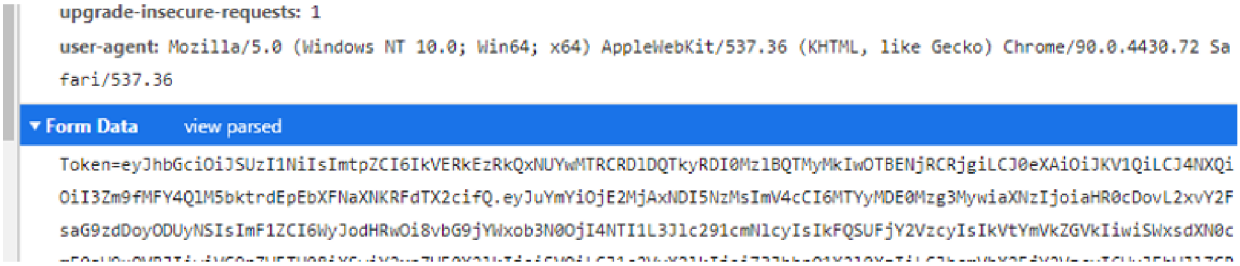
A user's browser may be redirected to the FireLight Token SSO URL, posted along with form data that contains a previously-obtained token from the API Token Service. As long as the token has access to the 'Token SSO' Supported Service (configured in the Admin Organization page, Security tab), the token may be used for redirecting users to the FireLight Token SSO URL. A session will be established for that user at that moment, and FireLight will, by default, redirect the user to the home page (other options exist).

From a high level perspective, a possible interaction would look like a two-step process – an external server system would request a token from the FireLight API Token Service, and then force a redirect with their user's browser, posting form data containing the retrieved token to the FireLight Token SSO URL. Alternatively, in another example, there could be another step in between – the token could be used on the 'CreateActivity' api to first create an activity for the user, and then the user could be redirected with the same token into that exact activity.

Note that content-type for these redirects must be 'application/x-www-form-urlencoded'. The form data must be named 'Token', and will have the url encoded string value of the token. An example of what the form data will look like:

Token=eyJhbGciOiJSUzI1NiIsImtpZCI6IkVERkEzRkQxNUYwMTRCRDlDQT etc….

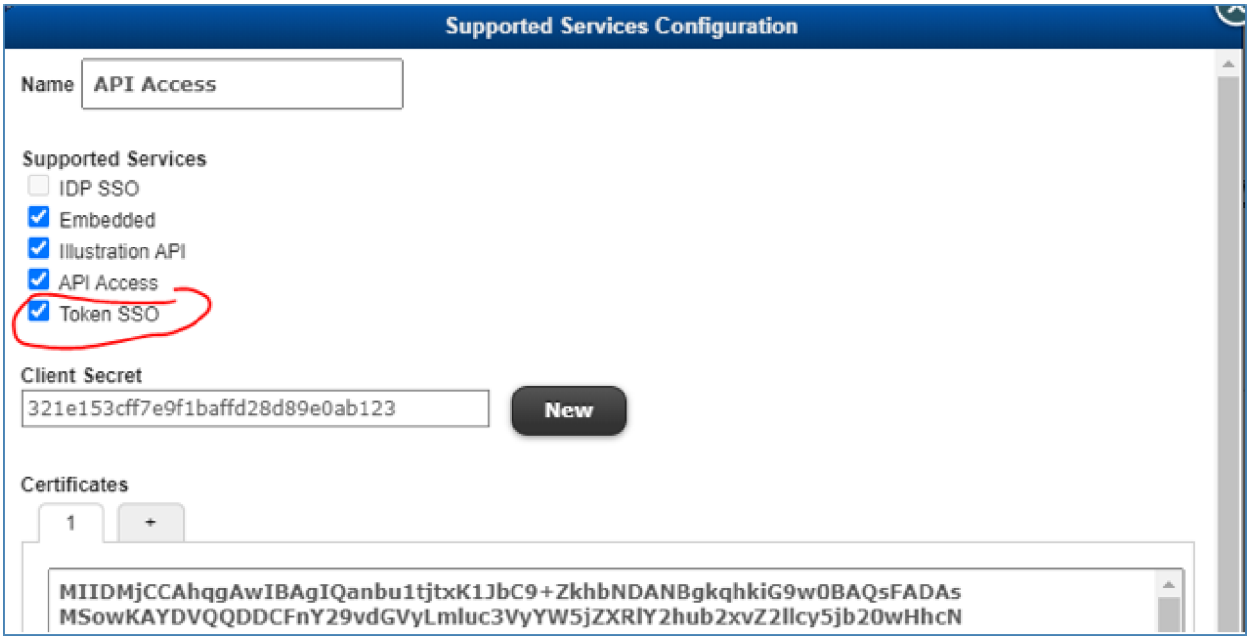
A screenshot of the form data (Chrome):



Admin Setting

All FireLight Tokens must first have a Supported Services Configuration established in order to retrieve them. These configurations are set up in the Admin Tool, located at the Organization page, on the Security tab.

In order for a token to be used on the Token SSO URL, it must have been obtained with a cert/secret configuration that has the 'Token SSO' Supported Service option enabled. The below screenshot shows this.



Redirect Options within FireLight

By default, a user redirected through this route into FireLight will land on the home page. However, two query variable options are available for the purpose of indicating different redirects within FireLight (once authentication and session has been established). These are:

- appid

  This will be accompanied with a specified app/activity's "AppId", a guid that identifies it within the FireLight system. This will redirect the user directly to that app/activity's page, so long as the user has access to that app/activity.

- relayState

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

This variable functions the same way as in the SAML redirect mechanism (also called relayState). The possible options are:

- Home

  Simply redirect the user to the home page of FireLight

- NewApplication

  Redirect the user to the New Application page

- Applications

  Redirect the user to the Applications page

- OSJ

  Redirect the user to the OSJ page

- eSign

  Redirect the user to the E-Signature page – when this option is sent, an 'appid' query variable must also included – the appid will correspond to an application/activity that the token user has access to, and is in a Locked or PendingSignature Status

- embed

  This will set a flag within the FireLight session indicating the UI is Embedded in a client's environment. The key functionality this provides is to hide unwanted UI elements when displayed in Embedded. Primarily the toolbar will no longer be visible. The embed flag is only available for relayState eSign or OSJ.

Examples:

?appid=84604672-acaa-4f5e-a906-70f6bbda2f9b

?relayState=NewApplication

?appid=84604672-acaa-4f5e-a906-70f6bbda2f9b&relayState=eSign&embed=true

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.

Endpoints

Endpoints for Token SSO

Environment	SAML 2 IDP-Initiated SSO	Status
External QA	https://flqa.insurancetechnologies.com/EGApp/SSOGateway/TokenGateway	Active
UAT	https://uat.firelighteapp.com/EGApp/SSOGateway/TokenGateway	Active
Staging	https://staging.firelighteapp.com/EGApp/SSOGateway/TokenGateway	Active
Production	https://www.firelighteapp.com/EGApp/SSOGateway/TokenGateway	Active

CONFIDENTIAL ©2019–2023 Hexure. All rights reserved.